

СПИСОК ИСПОЛНИТЕЛЕЙ

Руководитель НИР,
Профессор, д.ф-м.н

(подпись, дата)

Е.В. Бурнаев
(Введение, заключение)

Старший инженер-
исследователь

(подпись, дата)

М.А.Меньщиков
(разделы 1-4)

Эксперт
Профессор, д.ф-м.н

(подпись, дата)

И.Ю. Тюкин
(раздел 1)

Эксперт
к.т.н.

(подпись, дата)

Д.А. Евсеев
(разделы 2,3)

Старший инженер-
исследователь

(подпись, дата)

М.Е. Искорнев
(разделы 1-4)

Старший инженер-
исследователь
к.т.н.

(подпись, дата)

А.А. Андриянен
(разделы 1-4)

Инженер-исследователь

(подпись, дата)

П.А. Прядохин
(разделы 1-4)

Нормоконтроль

(подпись, дата)

И.А. Небесихин

РЕФЕРАТ

Отчет 99 с., 1 кн., 24 рис., 10 табл., 35 источн.

ГРАФ ЗНАНИЙ, LLM-АГЕНТ, ПАМЯТЬ, ПОИСК, ИЗВЛЕЧЕНИЕ, ТРИПЛЕТ, ТЕЗИС, ВОПРОСНО-ОТВЕТНАЯ СИСТЕМА, ПОИСКОВАЯ ДОПОЛНЕННАЯ ГЕНЕРАЦИЯ, АГЕНТ, СЕМАНТИЧЕСКАЯ БЛИЗОСТЬ, РАНЖИРОВАНИЕ ДОКУМЕНТОВ

Объект исследования НИР - архитектуры и алгоритмы построения и использования графа знаний как внешней памяти для LLM-агентов, ориентированных на решение QA-задач. Цель исследования - разработать подходы к созданию графа знаний и применить их в составе LLM-агента, предназначенного для обработки слабоструктурированной информации и QA-поиска.

Целью данного исследования является разработка подходов к созданию графа знаний с кратко-срочной и долгосрочной памятью, который может быть использован в качестве внешней памяти для таких LLM-агентов Индустриального партнера, как виртуальный тренажер для клиентских менеджеров, а также разработка LLM-агента для обобщения обратной связи от клиентов с целью развития продуктов Индустриального партнера.

Методологическая основа работы включает обзор и анализ современных GraphRAG-подходов, проектирование графа знаний, разработку memorize- и QA-пайплайнов (в том числе темпоральных модификаций), а также экспериментальную оценку на датасетах HotpotQA и RuBQ по метрикам Exact Match и LLM-as-a-Judge.

Научная новизна работы заключается в архитектуре внешней памяти LLM-агента на основе темпорального графа знаний, объединяющей краткосрочную и долгосрочную память и обеспечивающей более точный multi-hop поиск по сравнению с существующими GraphRAG-методами. В итоге разработанное решение позволило превзойти существующие GraphRAG-методы на 15% на HotpotQA-датасете по метрике Exact Match.

Экономическая эффективность. Ожидается, что представленные разработки будут применяться для создания Tutor AI, цифрового наставника, который анализирует коммуникации менеджера с клиентами Индустриального партнера, проверяет информацию на основе своей базы данных (памяти) о продукте и дает рекомендации. Внедрение Tutor AI позволит снизить затраты на очное обучение сотрудников, уменьшить число ошибок консультирования и, как следствие, повысить конверсию продаж и уровень лояльности клиентов, что приведет к росту прибыли Индустриального партнёра.

В перспективе предложенные методы могут быть расширены за счет семантического entity-resolution, поддержки многошагового диалога и адаптации для других предметных областей и типов интеллектуальных ассистентов.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	10
1 Обзор наиболее релевантных (SOTA) методов построения графа знаний для LLM-агента и методов обхода графа знаний для извлечения информации	13
1.1 Описание предметной области.....	13
1.2 Определение GraphRAG-подхода	14
1.3 Обзор GraphGenerator-методов	17
1.3.1 Методология поиска статей	17
1.3.2 Описание существующих методов.....	17
1.3.2.1 EDC.....	17
1.3.2.2 Zep.....	19
1.3.2.3 AutoSchemaKG	20
1.3.2.4 T-GRAG.....	21
1.3.3 Сравнительный анализ	22
1.3.4 Выводы.....	24
1.4 Обзор GraphRetrieval-методов	25
1.4.1 Методология поиска статей	25
1.4.2 Описание существующих методов.....	26
1.4.2.1 ToG	26
1.4.2.2 RoG.....	27
1.4.2.3 PMKGE	29
1.4.2.4 KG-RAG	31
1.4.2.5 GRAG.....	32
1.4.2.6 GNN-RAG	33
1.4.2.7 ToG2.0.....	35
1.4.2.8 DoG.....	36
1.4.2.9 GCR	37
1.4.2.10PDA.....	39
1.4.3 Сравнительный анализ	41
1.4.4 Выводы.....	41
1.5 Выводы	43
2 Разработка LLM агента, использующего разработанную архитектуру графа знаний и разработанный в алгоритм извлечения информации из графа знаний	44
2.1 Разработка архитектуры хранения структурированных и слабо- структурированных данных (графа знаний) и алгоритма, реализующего это решение ..	44
2.1.1 Обработка слабоструктурированных данных.....	45
2.1.2 Интеграция структурированных данных в граф.....	46
2.1.3 Представление знаний в графе	46

2.1.4 Актуализация знаний	47
2.1.5 Сохранение данных в графовую и векторную базы	47
2.1.6 Оценка производительности.....	47
2.1.7 Выводы.....	49
2.2 Алгоритм извлечения информации из памяти LLM-агента на основе графа знаний для решения QA-задач	49
2.2.1 Обработка пользовательского вопроса.....	52
2.2.1.1 Очистка вопроса.....	52
2.2.1.2 Обогащение вопроса	52
2.2.1.3 Декомпозиция вопроса	52
2.2.2 План поиска.....	52
2.2.2.1 Инициализация плана.....	52
2.2.2.2 Корректировка плана	53
2.2.3 Сопоставления сущностей из вопроса и вершин графа	53
2.2.4 Генерация вспомогательных запросов	53
2.2.5 Обход графа	54
2.2.5.1 WaterCircles	54
2.2.5.2 А со звездочкой	55
2.2.5.3 Поиск в ширину	55
2.2.5.4 Лучевой поиск	56
2.2.5.5 Наивный поиск	56
2.2.6 Фильтрация триплетов	57
2.2.7 Получение ответа на под-вопрос	57
2.2.7.1 Генерация ответов на вспомогательные запросы.....	57
2.2.7.2 Суммаризция ответов на вспомогательные запросы.....	58
2.2.7.3 Генерация ответа на под-вопрос	58
2.2.8 Получение ответа на исходный пользовательский вопрос	58
2.2.9 Выводы.....	59
2.3 Количественные оценки эффективности реализованной Retrieve/Rerank- операции.....	59
2.3.1 Постановка исследования	59
2.3.2 Алгоритмические формулировки задач.....	60
2.3.2.1 Одностадийный поиск (SingleStep).....	60
2.3.2.2 Многостадийный поиск (MultiStep)	60
2.3.2.3 Ансамблевый поиск (Ensemble).....	61
2.3.3 Источники данных.....	61
2.3.4 Конфигурации экспериментов.....	61
2.3.5 Результаты экспериментов	63
2.3.6 Выводы.....	66

2.4	Получение количественной оценки качества работы LLM-агента, созданного в ходе реализации алгоритма с использованием открытых публичных данных.	66
2.4.1	Постановка исследования	66
2.4.2	Источники данных	66
2.4.3	Конфигурации экспериментов	67
2.4.4	Результаты экспериментов	68
2.4.4.1	Характеристики построенных графов знаний	68
2.4.4.2	Анализ эффективности QA-пайплайна в различных конфигурациях . .	68
2.4.5	Выводы	69
2.5	Сравнение результатов по методам ToG, RoG, PMKGE, ToG2.0, DoG, PDA на HotpotQA-датасете по метрике Exact Match	69
3	Разработка модели памяти LLM-агента на основе темпорального графа знаний и алгоритма поиска информации в ней	71
3.1	Модель памяти LLM-агента на основе темпорального графа знаний	71
3.1.1	Представление знаний в графе	73
3.1.2	Алгоритм поиска устаревшей информации в графе	75
3.1.3	Выводы	76
3.2	Разработка алгоритма извлечения информации из графа знаний	76
3.2.1	Snapshot QA	79
3.2.2	Timeline/Range	80
3.2.3	Trend/Aggregation	80
3.2.4	Universal QA	81
3.2.5	Выводы	82
4	Направления развития предложенного решения	83
4.1	Технические улучшения	83
4.1.1	Семантическое entity-resolution	83
4.1.2	Дообучение и легковесные LLM	83
4.1.3	Критерии оценки улучшения планирования в QA-пайплайне	84
4.2	Поддержка диалогового взаимодействия	85
4.2.1	Архитектура многошагового взаимодействия с пользователем (диалоговая система)	85
4.2.1.1	Memorize-пайплайн	87
4.2.1.2	QA-пайплайн	88
4.2.2	Поддержка диалоговых структур и контекста	88
4.3	Концептуальные расширения системы	89
4.3.1	Мультиагентные архитектуры	89
4.3.2	Мультимодальность	90
4.3.3	Развитие системы в сторону цифрового наставника	90

4.4 Оценка качества и расширение бенчмарков	91
4.4.1 Новые бенчмарки с учетом темпоральности и доменной вариативности ...	91
4.4.2 Оценка улучшений плана и reasoning	92
4.5 Безопасность и надежность	93
4.6 Выводы	94
ЗАКЛЮЧЕНИЕ	95
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	97

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящем отчёте о НИР применяют следующие понятия, сокращения и обозначения.

Термины и основные понятия

Агент — программный компонент (часто на основе LLM), выполняющий ограниченный набор задач в рамках более общей архитектуры или пайплайна.

Граф знаний — структурированная база знаний в виде графа, где вершины соответствуют сущностям, а ребра обозначают отношения между ними.

Внешняя память агента — структура данных вне параметров модели (например, база документов или граф знаний), используемая для расширения знаний агента.

Вопросно-ответная система (QA-система) — система, принимающая на вход вопрос на естественном языке и возвращающая ответ, как правило с использованием поиска и/или генерации.

Контекстный подграф — подграф, извлеченный из полной структуры графа знаний как контекст для ответа на конкретный запрос.

Маршрут (путь) в графе — последовательность вершин и ребер графа, описывающая цепочку связанных фактов или тезисов.

Многоагентная система (MAS, Multi-Agent System) — система, состоящая из нескольких взаимодействующих агентов, совместно решающих комплексную задачу.

НИР — научно-исследовательская работа.

Обход графа знаний — процесс систематического просмотра всех вершин или рёбер графа для поиска вершин, удовлетворяющих, определённым условиям.

Переранжирование (reranking) — этап повторного упорядочивания уже найденных кандидатов с использованием более сложной модели или дополнительного сигнала качества.

Планирование (planning) в многоагентной системе — процесс разбиения исходной задачи на подзадачи и распределения их между агентами.

Подграф — ограниченная часть графа знаний, содержащая подмножество вершин и ребер, релевантных текущему запросу или задаче.

Поисковая дополненная генерация (RAG, Retrieval-Augmented Generation) — архитектура, в которой генерация текста LLM совмещается с извлечением внешних знаний из базы.

Темпоральный граф знаний — граф знаний, в котором факты и связи снабжены временными метками или интервалами.

Эмбеddинг — Векторное представление какого-либо объекта, чаще всего состоящее из вещественных векторов ограниченной длины, так же может использоваться слово представление с идентичным смыслом.

LLM-модель (Large Language Model) — крупная нейросетевая языковая модель на основе архитектуры трансформер, способная обрабатывать и генерировать текст на естественном языке.

Метрики качества и оценивания

Recall@ k — доля релевантных объектов, оказавшихся в топ- k выдачи по сравнению со всеми релевантными объектами.

Precision@ k — доля релевантных объектов среди первых k элементов выдачи.

F1@ k — гармоническое среднее Precision@ k и Recall@ k , вычисляемое на топ- k результатах.

MAP@ k (Mean Average Precision@ k) — среднее по всем запросам значение средней точности, вычисляемой с учетом порядка появления релевантных объектов в топ- k выдачи. Exact Match (EM) — метрика, равная единице, если сгенерированный ответ в точности совпадает с эталонным, и нулю в противном случае.

LLM-as-a-judge — подход, при котором LLM используется как внешняя оценочная функция для сравнения, ранжирования и валидации ответов или путей рассуждения.

F1-score (F1) — гармоническое среднее между полнотой и точностью на уровне токенов или слов в ответе.

Mean Reciprocal Rank (MRR) — среднее по запросам значение обратного ранга первого релевантного документа.

ROUGE L — метрика для оценки качества решения задачи суммаризации текста с максимально длинной последовательностью совпадения слов.

Accuracy — доля случаев, в которых система выдала полностью корректный ответ.

Coverage (покрытие) — доля запросов, для которых система смогла сформировать ответ с использованием найденного подграфа.

ВВЕДЕНИЕ

Основанием для проведения работ является Техническое задание по теме «Разработка методов и подходов к построению графа знаний как памяти для агентов LLM» (Приложение №1 к Дополнительному соглашению №1 от 28 октября 2025 г. к Договору с Индустриальным партнером №0005283081 от 28 марта 2025 г., далее – Техническое задание).

Необходимость проведения НИР обусловлена задачами Индустриального партнера по созданию надёжных LLM-агентов для анализа коммуникаций с клиентами и развития продуктов, при том что существующие решения на основе стандартной поисково-дополненной генерации (RAG) недостаточно хорошо работают с составными запросами и большими объемами слабоструктурированных данных.

Планируемый научно-технический уровень результатов НИР соответствует современным исследованиям (SOTA) в области GraphRAG-подходов и LLM-агентов: разрабатываемые методы должны обеспечивать качество ответов не ниже уровня известных аналогов и быть пригодными для интеграции в промышленный прототип Tutor AI.

В последнее время для расширения знаний LLM все шире применяется подход поисковой дополненной генерации (RAG), при котором процесс генерации текста с помощью LLM сочетается с извлечением внешней информации из базы знаний. Однако традиционные векторные реализации RAG полагаются исключительно на семантическое сходство и слабоструктурированное представление данных. Несмотря на эффективность в простых задачах, они затрудняют масштабирование и решение составных запросов, поскольку не учитывают глобальный контекст и связи между фактами [1]. Для преодоления этих ограничений в качестве внешней памяти перспективно использование графов знаний – структурированных баз знаний в виде графа, где вершины соответствуют сущностям, а ребра описывают отношения между ними. Графовая память позволяет агенту не только находить отдельные факты, но и понимать контекст и взаимосвязи между ними, что повышает осмысленность и точность выводов модели. Благодаря атомарному представлению информации в графе появляется возможность таргетно извлекать нужные данные и отсеивать нерелевантное. Кроме того, графовая структура в целом универсальна и способна выразить большую часть текстовой информации. Указанные преимущества демонстрируют, что использование графа знаний как памяти особенно актуально для задач, связанных с анализом коммуникаций: в подобных сценариях LLM-агенту необходимо усваивать знания из слабоструктурированных диалогов, выходящих за рамки его краткосрочного контекста. Например, Индустриальный партнер развивает системы анализа коммуникаций с клиентами, включая цифровые тренажеры для обучения клиентских менеджеров и обобщения обратной связи клиентов. В таком проекте LLM-агент должен понимать семантику сообщений и диалогов и обладать механизмом накопления знаний, то есть внешней долговременной

памятью. Исследования показывают, что наиболее эффективным инструментом реализации внешней памяти для LLM-агентов являются именно графы знаний, благодаря способности явно сохранять факты и отношения между ними.

Целью работы является разработка подходов к созданию графа знаний с краткосрочной и долгосрочной памятью, который может применяться в качестве внешней памяти для LLM-агентов индустриального партнера, например, виртуального тренажера для клиентских менеджеров; а также разработка самого LLM-агента, использующего этот граф для обобщения знаний из клиентских коммуникаций. Ожидается, что полученное решение превзойдет существующие GraphRAG-методы, указанные в Разделе 1.4, на 3% по ExactMatch-метрике на HotpotQA-датасете. Для достижения поставленной цели решены следующие задачи:

1. Пункт 2.3.1. ТЗ - Обзор наиболее релевантных (SOTA) методов построения графа знаний для LLM-агента и методов обхода графа знаний для извлечения информации (раздел 1).
2. Пункт 2.3.2. ТЗ - Разработка архитектуры хранения структурированных и слабо-структурированных данных (графа знаний) и алгоритма, реализующего это решение (2.1);
3. Пункт 2.3.3. ТЗ - Разработка алгоритма извлечения информации из графа знаний (раздел 3.2);
4. Пункт 2.3.4. ТЗ - Разработка LLM-агента, использующего разработанную архитектуру графа знаний и разработанный в алгоритм извлечения информации из графа знаний (раздел 2);
5. Пункт 2.3.5. ТЗ - Получение количественной оценки качества работы LLM-агента, созданного в ходе реализации алгоритма с использованием открытых публичных данных. (раздел 2.3).

В рамках проекта разработана архитектура, объединяющая возможности графа знаний и метода RAG для поддержки работы LLM-агента. Для пополнения долгосрочной памяти реализован конвейер memorize-пайплайна, выполняющий преобразование исходных данных (структурированных и неструктурированных) в формат графа знаний. Соответственно, для поиска информации используется QA-пайплайн, который по входному вопросу осуществляет обход графа знаний и извлекает релевантные сведения для генерации ответа. Также разработана версия графа, поддерживающая темпоральную структуру. Факты и связи в таком графе снабжены временными метками (темпоральный граф знаний), что позволяет учитывать динамику знаний во времени (например, отвечать на вопросы, связанные с конкретными датами или периодами). Для эффективного поиска информации предусмотрены два режима работы QA-пайплайна: weak (быстрый, менее ресурсозатратный) и medium (более тщательный и требовательный с точки зрения ресурсов). В weak режиме агент выполняет минимальный необходимый обход графа для получения ответа, тогда как medium включает этап планирования поиска и генерацию вспомогательных под-запросов, повышая точность результата. Оба

режима поддерживают различные алгоритмы обхода графа, такие как лучевой поиск, поиск в ширину и др.

С научной точки зрения, данное исследование обладает высокой степенью новизны. Идея использования внешнего графа знаний вместо классического текстового RAG-подхода уже фигурировала в современных разработках [2], [1], однако существующие решения в основном рассчитаны на относительно простые сценарии (диалоговые боты, автоматический ответ на одиночные вопросы). В данной работе решается принципиально новая задача – построение комбинированного графа знаний, объединяющего структурированную и слабоструктурированную доменную информацию, необходимую для работы специализированных интеллектуальных агентов. Ранее подобная задача не решалась, что позволяет говорить о существенной научной новизне проекта. Более того, предложенный подход открывает возможности для использования специализированных алгоритмов хранения и поиска данных, повышающих степень синергии между памятью и агентом.

Анализ современных методов в области систем поисково-дополненной генерации и графовых баз знаний показал отсутствие решений, полностью совпадающих с заявленным объектом разработки, при наличии близких по назначению аналогов, что подтверждает актуальность и новизну выбранного направления.

Метрологическое обеспечение НИР основано на использовании стандартных метрик качества вопросно-ответных систем (Exact Match, Recall@k, Precision@k, F1@k, LLM-as-a-Judge и др.) и на экспериментальных исследованиях на открытых датасетах HotpotQA и RuBQ, что позволяет количественно оценивать достигнутый научно-технический уровень и сопоставлять его с современными аналогами.

Разработка графа знаний как памяти для LLM-агента тесно связана с проектом «Цифровой наставник», направленным на анализ коммуникаций менеджеров с клиентами. Ожидается, что внедрение такой графовой памяти в интеллектуальный тренажер позволит значительно упростить работу менеджеров, снизить отток клиентов и повысить их лояльность, тем самым увеличивая прибыль компании. Предлагаемые методы и архитектура имеют потенциал применения и в других областях: корпоративных диалоговых и аналитических системах, вопросно-ответном поиске по слабоструктурированным данным, инструментах обучения и контроля качества коммуникаций.

1 Обзор наиболее релевантных (SOTA) методов построения графа знаний для LLM-агента и методов обхода графа знаний для извлечения информации

1.1 Описание предметной области

Разработка и обучение таких больших языковых моделей как GPT4 [3], Qwen2 [4] и LLaMA [5] вызвало революцию в области технологий искусственного интеллекта и фундаментально изменило подходы по обработке естественного языка. Данные модели, построенные на Transformer-архитектуре и обученные на обширных текстовых датасетах, продемонстрировали беспрецедентные возможности в понимании, интерпретации и генерации человеческого языка. Появление таких инструментов в открытом/частичном доступе позволило повысить эффективность взаимодействия между машинами и людьми для решения практических задач в различных областях: медицине [6], финансовом деле [7], системе образования [8].

Несмотря на свои возможности, связанные с пониманием языка и генерацией текста, LLM-модели сталкиваются с трудностями при обработке вопросов, требующих знаний из специализированной области, релевантная информация по которым отсутствовала в тренировочной выборке. Такие "пробелы в знаниях" могут привести к явлению, известному как "галлюцинации" [9], когда модель генерирует неточную или ложную информацию. Чтобы снизить влияние данной проблемы, крайне важно предоставить LLM-модели дополнительные тексты из внешних источников. Подход Retrieval-Augmented Generation (RAG) является естественным решением описанной проблемы, который направлен на повышение фактологической точности и специфичности генерируемой информации путем интеграции компонент поиска в данный процесс. RAG-подход привлек широкое внимание исследователей и представителей бизнеса благодаря своей эффективности и широкому применению, став ключевым направлением в данной области. Несмотря на свои явные преимущества у стандартных RAG-методов есть ряд ограничений:

1. Пренебрежение взаимосвязями (Neglecting Relationships): на практике части текстовой информации не изолированы друг от друга, а взаимосвязаны. Стандартные RAG-техники не в состоянии извлечь необходимый набор связанной информации только с помощью оценки их семантической близости к user-вопросу.
2. Избыточная информация (Redundant Information): в RAG-методах извлеченные фрагменты текста часто объединяются в один и подаются на вход LLM-модели в качестве одной из частей промпта (prompt) для генерации ответа. Это делает финальный промпт чрезмерно длинным, что приводит к дилемме "Lost in the middle" [10].

3. Отсутствие глобальных знаний (Lacking Global Information): стандартные RAG-методы могут извлекать только подмножество изолированных фрагментов текста и не в состоянии охватить их глобальные свойства/категории для анализа, поэтому они имеют серьёзные ограничения при обработке запросов на резюмирование заданной области знаний (Query-Focused Summarization).

Graph Retrieval-Augmented Generation (GraphRAG) выступает в качестве одного из ответвлений RAG-подхода, направленный на решение вышеописанных проблем. В отличие от традиционных RAG-техник, GraphRAG-методы выполняют извлечение (в том или ином виде) набора элементов, которые содержат информацию о типе связи друг с другом, из предварительно-сформированной графовой структуры данных. В качестве таких атомарных элементов информации могут выступать узлы графа, триплеты (пары связанных вершин), пути (связанный набор триплетов) или подграфы, которые далее используются для генерации ответа на user-вопрос. GraphRAG-подход учитывает взаимосвязи между имеющимися наборами знаний, обеспечивая более точный поиск. Кроме того, хранение информации в виде графа предполагает приведение неструктурированных текстов на естественном языке к набору "сухих" коротких фактов, что позволяет сократить длину подаваемого на вход контекста для LLM-модели и смягчить проблему излишней информации. Из-за более концентрированного способа хранения можно на этапе извлечения охватить больше знаний и тем самым повысить качество решения QFS-задачи.

В данной главе выполнен обзор существующих GraphRAG-методов применительно к решению задач генерации фактологических и контекстно-зависимых (узкоспециализированных) текстов на естественном языке с помощью языковых моделей. В отдельности рассматриваются две важных составляющих: способ построения графа (Graph Generation) и способ поиска релевантной информации в нём (Graph Retrieval). Для достижения поставленной цели решены следующие задачи:

1. Выполнен поиск научных публикаций по GraphGeneration- и GraphRetrieval-методам в открытых научных архивах.
2. Сформирован набор актуальных научных работ.
3. Выполнен обзор отобранных статей.
4. Выполнен сравнительный анализ отобранных статей.
5. Сформулированы рекомендации по использованию/комбинации разобранных методов для повышения качества генерации предметно-ориентированных текстов на естественном языке с помощью языковых моделей.

1.2 Определение GraphRAG-подхода

GraphRAG – это подход, в рамках которого используются внешние графы знаний для повышения степени детализации и информативности генерируемых ответов на user-вопросы. Основной задачей GraphRAG-методов является извлечение наиболее ре-

levantного информационного фрагмента (knowledge) из базы знаний, который позволит улучшить качество ответа в рамках некоторой предметной области. Процесс генерации формально может быть записан как

$$a^* = \arg \max_{a \in A} p(a|q, \mathcal{G}), \quad (1.1)$$

где a^* это оптимальный ответ по вопросу q на основе графа $\mathcal{G}$ (Text-Attributed Graph), а A это множество возможных ответов. Распределение вероятностей ответов $p(a|q, \mathcal{G})$ может быть смоделировано через произведение $p_\phi(G|q, \mathcal{G})$ и $p_\theta(a|q, G)$ вероятностей, где ϕ и θ являются обучаемыми параметрами, и формально записано как

$$\begin{aligned} p(a|q, \mathcal{G}) &= \sum_{G \subseteq \mathcal{G}} p_\theta(a|q, G) p_\phi(G|q, \mathcal{G}) \\ &\approx p_\theta(a|q, G^*) p_\phi(G^*|q, \mathcal{G}), \end{aligned} \quad (1.2)$$

где G^* это оптимальный подграф с концентрированным набором релевантной информации. Так как число возможных подграфов растёт экспоненциально с увеличением размера исходного графа, использование эффективных алгоритмов аппроксимации поиска является важным требованием при интеграции подобных методов в соответствующие программные комплексы.

GraphRAG-подход может быть разбит на три основные стадии (смотрите на рисунок 1.1): построение/индексирование графа знаний (G-Indexing), поиск по графу (G-Retrieval) и генерация ответа на основе извлечённого подграфа (G-Generation).

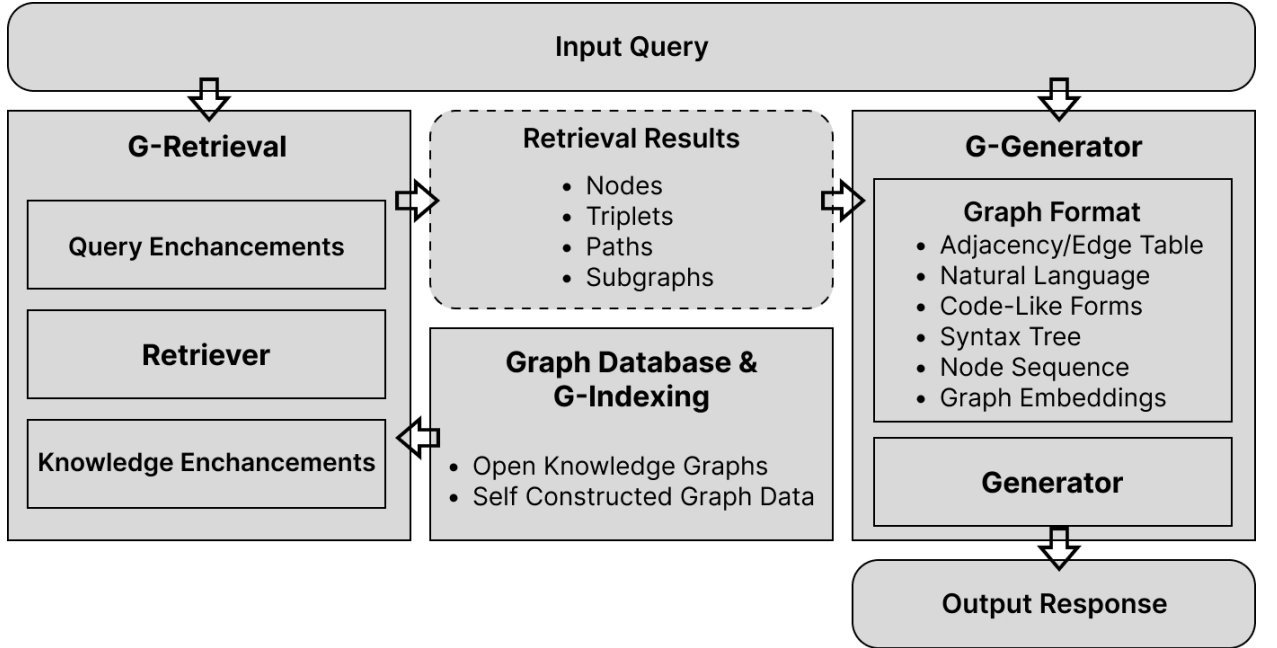


Рисунок 1.1 — Основные стадии GraphRAG-подхода

Graph-Based indexing (G-indexing). На первой стадии создается или выбирается графовая база данных G , в которой содержатся знания из заданной области, и для

неё выполняется процедура индексирования. Такая база данных может быть собрана из открытых графов знаний или построена на основе собственных материалов в виде текста. Процесс индексирования включает в себя: (1) создание взаимно-однозначных соответствий между вершинами/рёбрами графа и их свойствами/атрибутами и (2) упорядочивание информации для ускорения операций поиска/перебора. Гранулярность, заданная на этапе индексирования, существенно влияет на производительность последующей стадии извлечения информации и скорости обработки user-вопросов.

Graph-Guided Retrieval (G-Retrieval). После построения графа знаний на первой стадии далее можно запускать/применять алгоритмы поиска/извлечения информации для генерации ответов на входящие user-вопросы. В рамках второй стадии осуществляется извлечение наиболее релевантных элементов (например: сущностей, триплетов, путей, подграфов) из графовой базы данных в соответствии с запросом пользователя q , который представлен на естественном языке. Формально данный процесс может быть записан как

$$\begin{aligned} G^* &= G\text{-Retriever}(q, \mathcal{G}) \\ &= \arg \max_{G \in \mathcal{R}(\mathcal{G})} p_\theta(G|q, \mathcal{G}) \\ &= \arg \max_{G \in \mathcal{R}(\mathcal{G})} \text{Sim}(q, G), \end{aligned} \tag{1.3}$$

где G^* – оптимальный набор элементов из $\mathcal{G}$, а $\text{Sim}(\cdot, \cdot)$ – функция, которая измеряет семантическую близость между q и $G \in \mathcal{G}$. Функция $\mathcal{R}(\cdot)$ предназначена для сужения диапазона поиска с целью повышения производительности системы.

Graph-Enhanced Generation (G-Generation). Данная стадия отвечает за формирование ответа системы по подмножеству элементов из графа знаний. Ответ может быть представлен в виде: списка рекомендаций, резюме в виде нескольких абзацев текста, инструкций/планов решения user-проблемы. Генератор-модель принимает в качестве входных данных запрос q , извлеченные элементы G^* из графа и формирует финальный результат. Формально данный процесс может быть записан как

$$\begin{aligned} a^* &= G\text{-Generator}(q, \mathcal{G}^*) \\ &= \arg \max_{a \in A} p_\phi(a|q, G^*) \\ &= \arg \max_{a \in A} p_\phi(a|\mathcal{F}(q, \mathcal{G})^*), \end{aligned} \tag{1.4}$$

где $\mathcal{F}(\cdot, \cdot)$ это функция, которая переводит элементы графа в формат для обработки генератор-моделью.

1.3 Обзор GraphGenerator-методов

1.3.1 Методология поиска статей

Формирование релевантного множества научных публикаций выполнено в три этапа. На первом этапе были использованы специализированные поисковые системы: SciSpace, Scite, PaperDigest, Consensus, Elicit, Google Scholar. С помощью каждой системы были выбраны первые десять публикаций в поисковой выдаче по следующим четырём поисковым запросам: "LLM-based knowledge graph construction" , "LLM-based Temporal knowledge graph construction methods" , "Temporal QA over LLM-based Knowledge Graph methods" , "Temporal QA over LLM-based Text-attributed Knowledge Graph methods". При этом поисковая выдача была ограничена годом публикации: до 2019. На втором этапе была выполнена фильтрация отобранных работ на основе их кратких описаний (abstracts): удалены дубликаты; исключены работы, в которых авторы презентуют свой датасет в качестве тестового набора данных; исключены работы, где предлагаемый метод заточен на конкретную прикладную задачу. На третьем этапе отбирались работы, в которых детально описан предложенный метод. В результате было сформировано множество из 4 статей: в Таблице 1.1 отобранные работы представлены в хронологическом порядке.

Таблица 1.1 — Хронология статей, на основе которых выполнен обзор литературы по GraphGenerator-методам

N	Название метода	Дата публикации	Реализация метода
1	EDC [11]	01.11.24	https://github.com/clear-nus/edc
2	Zep [12]	20.06.25	https://github.com/getzep/graphiti
3	AutoSchemaKG [13]	01.08.25	https://github.com/HKUST-KnowComp/AutoSchemaKG
4	T-GRAG [14]	03.08.25	https://github.com/Arvin0313/T-GRAG

1.3.2 Описание существующих методов

1.3.2.1 EDC

В статье [11] авторы предложили EDC-метод для автоматического построения графа знаний без фиксированной онтологии с помощью LLM-модели по текстам на естественном языке. Можно выделить следующие ключевые/общие техники, которые были использованы в данном методе: "Prompt Engineering" , "Knowledge Graph Construction" , "Triplets Canonicalization" , "Schema Retriever".

Как показано на рисунке 1.2, метод состоит из трёх основных стадий: Extract, Define, Canonicalize. В рамках первой стадии с помощью LLM-модели выполняется

извлечение/формирование триплетов из входного текста без привязки к онтологии текущего графа. В рамках второй стадии для каждого отношения/предиката у полученных триплетов выполняется генерация его расширенного описание (определения). На последней третьей стадии выполняется канонизация/унификация новых отношений. Если в текущем графе уже существует отношение/предикат, которое семантически подходит для текущего нового триплета, то выполняется замена: существующий в графе "text-attributed"-предикат заменяет новый. В результате полученный набор триплетов добавляется в граф.

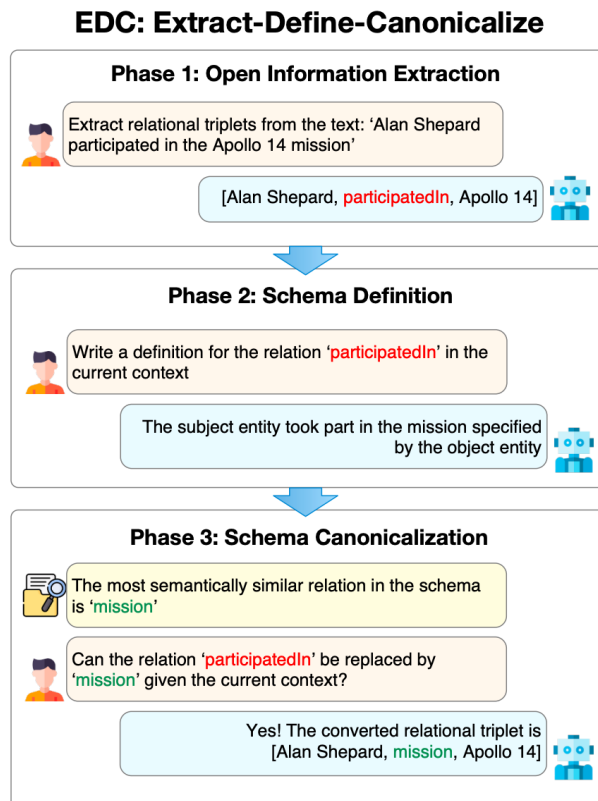


Рисунок 1.2 — Иллюстрация работы EDC-метода. Диаграмма взята из оригинальной статьи [11]

В статье описаны показатели эффективности предложенного метода при использовании следующих LLM-моделей в качестве основных text-генераторов: GPT3.5-turbo, GPT4, Mistral7B, E5-Mistral7B. Оценка эффективности метода была выполнена на следующих датасетах: WebNLG, REBEL, Wiki-NRE. В качестве метрик, по которым оценивалась эффективность метода были использованы следующие: Precision, Recall, F1, No.Relations (компактность) и Redundancy (семантическое различие типов). К недостаткам EDC-метода можно отнести следующие моменты:

1. Несмотря на наличие "Relation Resolution"-техники при форматировании триплетов, отсутствует механизм по унификации вершин (Nodes Resolution).
2. В предложенном методе нет механизма по явному учёту актуальности информации, сохраняемой в граф. Построенный таким образом граф знаний является time-insensitive.

3. Ключевые задачи в рамках каждой из трёх стадий метода выполняются за одну inference-операцию LLM-модели. В случае использования small/medium-size моделей будет заметна сильная деградация в качестве построенного графа. Требуется декомпозиция сложной LLM-задачи и проектирование/разработка более глубокого/проработанного мультиагентного пайплайна.
4. Отсутствует стратегия/алгоритм поиска устаревшей информации в графе.
5. Из-за отсутствия фиксированной онтологии, качество построенного графа сильно зависит от используемой LLM-модели (проблема стабильности).

1.3.2.2 Zep

В статье [12] авторы предложили трёхуровневую архитектуру/онтологию графа знаний, а также алгоритм автоматической генерации такого графа с помощью LLM-модели. Можно выделить следующие ключевые/общие техники, которые были использованы в данном методе: "Cross-encoder" , "Hybrid Search" , "LLM-driven Edge Invalidation" , "Entity Resolution".

Предложенная архитектура графа знаний состоит из трёх связанных подграфов: Episodic Subgraph, Semantic Entity Subgraph и Community subgraph. Episodic-подграф состоит из вершин, которые содержат исходные фрагменты текста. Между вершинами в Episodic-подграфе нет связей/отношений. Semantic-подграф в своих вершинах содержит именованные сущности и атомарную фактологическую информацию, которая была извлечена из соответствующих episodic-вершин. Формирование данного графа осуществляется с помощью LLM-модели: выполняется генерация триплетов с заданными типами вершин. Между Episodic-вершиной и соответствующих ей entity- и fact-вершинами создаётся гипер-ребро. Community-подграф в своих вершинах содержит короткие общие описания (резюме) семантически-похожих entity-вершин из Semantic-подграфа. Между summary-вершиной и entity-вершинами, которые данная summary-вершина обобщает, создаётся гипер-ребро.

Так же в рамках предложенной архитектуры присутствует способ учёта актуальности сохраняемой в граф информации; у всех триплетов в Semantic-подграфе есть следующие атрибуты: t_{valid} , $t_{invalid}$, $t_{created}$, $t_{expired}$. Добавление новых триплетов может привести к модификации time-атрибутов у существующих триплетов. Поиск устаревшей информации осуществляется за счёт оценки семантической близости (по плотным векторным представлениям) между новым триплетом и существующими, а также их последующей фильтрацией с помощью LLM-модели.

Предложенный метод позволил получить следующий прирост оценки качества по Accurasy-метрике в сравнении с baseline-методом (MemGPT [15]) на DMR (Deep Memory Retrieval) датасете: на 1.4 (94.8 - 93.4) процентов. В статье описаны показатели эффективности предложенного метода при использовании следующих LLM-моделей в качестве основных text-генераторов: BGE-m3, GPT4o-mini, GPT4-turbo. Оценка эффективности метода была выполнена на следующих датасетах: DMR, LongMemEval. В

качестве метрик, по которым оценивалась эффективность и производительность метода были использованы следующие: Accuracy и Latency. К недостаткам Zer-метода можно отнести следующие моменты:

1. Несмотря на наличие "Nodes Resolution"-техники при форматировании Entity-триплетов, отсутствует механизм по унификации связей/отношений (Relations Resolution).
2. Ключевые задачи по добавлению новой информации в граф выполняются за одну inference-операцию LLM-модели. В случае использования small/medium-size моделей будет заметна сильная деградация в качестве построенного графа. Требуется декомпозиция сложной LLM-задачи и проектирование/разработка более глубокого/проработанного мультиагентного пайплайна.

1.3.2.3 AutoSchemaKG

В статье [13] авторы предложили полностью автоматический способ построения графа знаний без заданной схемы. Можно выделить следующие ключевые/общие техники, которые были использованы в данном методе: "Schema Induction", "Prompt Engineering", "Conceptualization", "Knowledge Graph Construction", "Triple Extraction" (EE, EV, VV), "Contextual Enhancement for Entities".

Как показано на рисунке 1.3, из текста извлекаются триплеты 3-х типов: Entity-Entity, Entity-Event и Event-Event. Events-вершины фиксируют темпоральные и каузальные связи; они сохраняют больше информации чем обычные сущности. Следующий шаг - conceptualization: entity и events абстрагируют до концептов, и каждый узел привязывается к одному или нескольким концептам. Пример графа знаний, построенного с помощью предложенного метода, представлен на Рисунке 1.4.

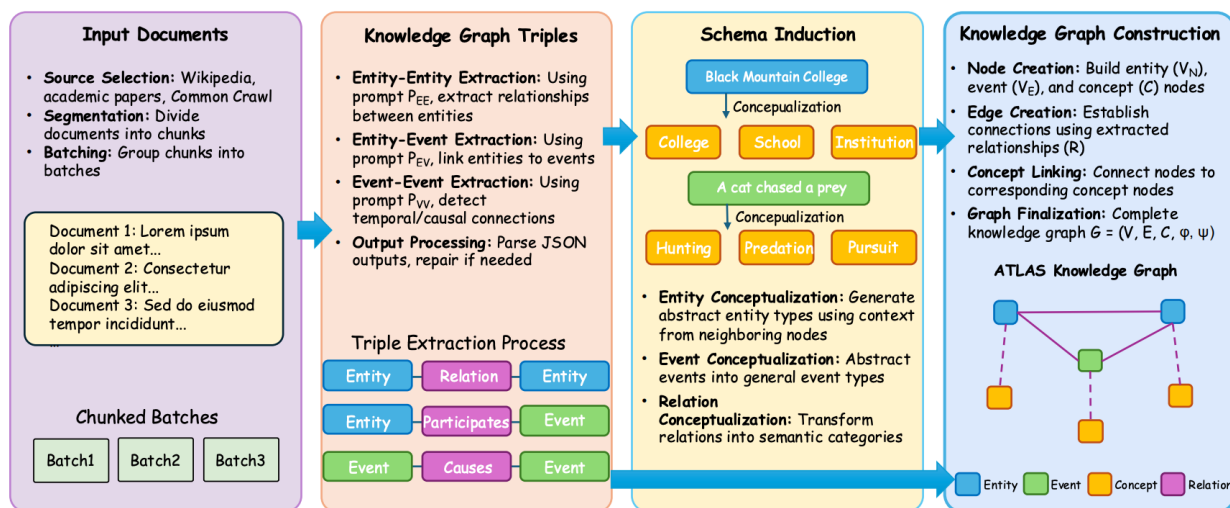


Рисунок 1.3 — Иллюстрация работы AutoSchemaKG-метода. Диаграмма взята из оригинальной статьи [13]

Предложенный метод позволил получить следующий прирост оценки качества по F1-метрике в сравнении с baseline-методом HippoRAG2 [2] на HotpotQA-датасете:

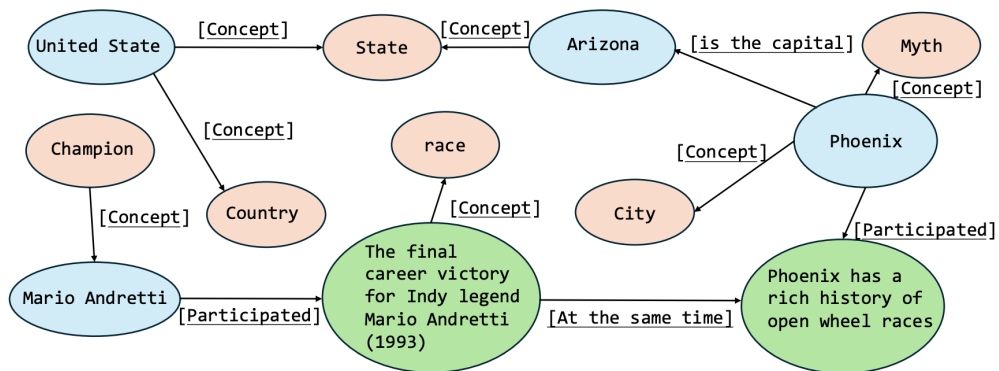


Рисунок 1.4 — Пример построенного графа знаний с помощью AutoSchemaKG-метода. Рисунок взят из оригинальной статьи [13]

на 2.8 (78.3 - 75.5) процентов. В статье описаны показатели эффективности предложенного метода при использовании следующих LLM-моделей в качестве основных text-генераторов: Llama3-8B-Instruct, Llama3.1-8B-Instruct, Llama3-13B, Llama3-70B. Оценка эффективности метода была выполнена на следующих датасетах: MuSiQue, HotpotQA, 2WikiMultihopQA (QA), FELM (Factual errors), MMLU. В качестве метрик, по которым оценивалась эффективность метода были использованы: EM (для QA), Precision, Recall, F1 (для качества извлеченных триплетов), Accuracy, F1 (FELM), Accuracy (MMLU). К недостаткам AutoSchemaKG-метода можно отнести следующие моменты:

1. Предложенная структура графа знаний имеет ограниченную применимость к специализированным доменам.

1.3.2.4 T-GRAG

Авторы предлагают подход к построению памяти и QA, решающий 2 проблемы: временная неоднозначность и избыточность нерелевантной информации на retrieval-стадии. Можно выделить следующие ключевые/общие техники, которые были использованы в данном методе: "Prompt Engineering" , "TKG Construction" , "Three-layer Retrieval" , "Temporal Query Decomposition" , "GraphRAG".

Как показано на рисунке 1.5, предложенный метод состоит из 5 модулей: TKG Generator - строит динамический граф с временными атрибутами у сущностей/связей/фактов. Temporal Query Decomposition - декомпозирует сложные временные вопросы на независимые под-вопросы. Three-layer Retriever - объединяет в себе три этапа: поиск релевантного подграфа по нужной временной метке, извлечение кандидатов-вершин по семантической близости и получение релевантных чанков знаний (фрагментов текста) из узлов. Valid Source Text Extractor - достает исходные текстовые фрагменты, релевантные под-вопросу. LLM augmented Generator - собирает ответ из 3-х источников: знания внутри узлов, топология подграфа и исходные текстовые фрагменты.

Предложенный метод позволил получить следующий прирост оценки качества по Accuracy-метрике в сравнении с baseline-методом (GraphRAG [1]) на Time-LongQA-

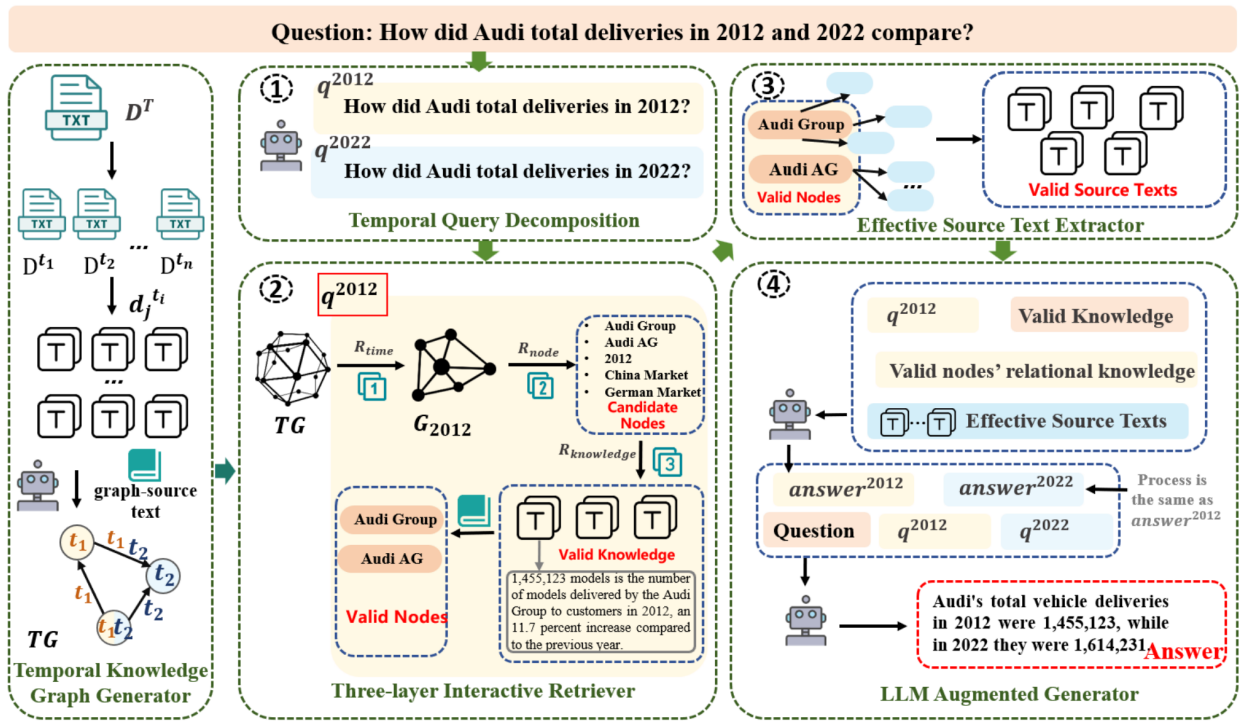


Рисунок 1.5 — Иллюстрация работы "T-GRAG"-метода. Диаграмма взята из оригинальной статьи [14]

датасете: на 22 процента. В статье описаны показатели эффективности предложенного метода при использовании следующих LLM-моделей в качестве основных text-генераторов: Llama3.1-70b, Qwen2.5-72b, Nemotron3.1-70b, Llama3.2-8B-Instruct, StellaEN1.5B-v5. Оценка эффективности метода была выполнена на Time-LongQA датасете. В качестве метрик, по которым оценивалась эффективность и производительность метода были использованы следующие: Accuracy и Latency. К недостаткам "T-GRAG"-метода можно отнести следующие моменты:

1. Для построения графа знаний с помощью данного метода требуются структурированные/отсортированные по времени текстовые данные.
2. В статье предложен механизм учёта только точечных отметок времени; интервалы времени и другие конструкции опускаются.
3. При семантическом поиске информации в построенном графе используются только плотные (dense) векторные представления. Требуется комбинация dense- и sparse-представлений.

1.3.3 Сравнительный анализ

В результате разбора отобранных GraphGenerator-методов составлена сравнительная Таблица 1.2 с их характеристиками.

Из Таблицы 1.2 видно, что каждый разобранный метод оценивался на уникальном датасете, в виду чего нет возможности их количественного сравнения по абсолют-

Таблица 1.2 — Сравнительная таблица характеристик разобранных GraphGenerator-методов. Характеристика Performance показывает эффективность метода по заданной метрике и на заданном датасете. Характеристика Train-free показывает: для работы метода не требуется дополнительного дообучения используемых LLM/transformer-моделей. Характеристика Time-sensitive KG показывает: при построении графа выполняется извлечение/сохранение отметок времени из исходного текста для учёта темпоральных аспектов структурируемых знаний. Характеристика Time interval awareness показывает: из исходного текста извлекаются временные интервалы валидности, хранящихся в нём (исходном тексте), знаний. Характеристика Nodes Resolution показывает: перед добавлением нового набора триплетов в граф выполняется унификация их вершин на основе существующего набора. Характеристика Relations Resolution показывает: перед добавлением нового набора триплетов в граф выполняется унификация их рёбер на основе существующего набора. Характеристика "Outdated knowledge search"-mechanism показывает: в методе предусмотрен/описан механизм поиска устаревшей информации в формируемом графе. Характеристика Constructed Knowledge Graphs Statistics показывает: в статье приведена статистика по графам знаний, построенных в рамках проводимых экспериментов. Характеристика Inference Time (per query) Statistics показывает: в статье приведены оценки скорости обработки одного user-вопроса на основе построенных графов. Характеристика Retrieval Embedders показывает типы embedder-моделей, которые используются для получения векторных представлений текста и при выполнении векторного поиска.

N	Характеристики			GraphGenerator-метод			
				EDC (GPT3.5-turbo)	Zep (GPT4-turbo)	AutoSchemaKG (Llama3.1 8B)	T-GRAG (QWEN2.5 70B / Llama3.2 8B)
1	Performance	F1	WebNLG	78,5	-	-	-
			HotpotQA	-	-	78,3	-
		LLM-as-a-Judge	DMR	-	94,8	-	-
			Time-LongQA	-	-	-	73,24 / 56,18
2	Train Free (+/-)			+	+	+	+
3	Time-sensitive KG (+/-)			-	+	-	+
4	Time interval awareness			-	+	-	-
5	Time Resolution			-	+	-	-
6	Nodes Resolution			-	+	-	-
7	Relations Resolution			+	-	-	-
8	"Outdated knowledge search"-mechanism			-	+	-	+
9	Constructed Knowledge Graphs Statistics			-	-	+	-
10	Inference Time (per query) Statistics			-	+	-	+
11	Retrieval Embedders			Dense			

ным значениям. Можно лишь обратить внимание, что Zep-метод по LLM-as-a-Judge-метрике сильно выделяется среди других методов. По поводу оценок производительности и характеристик/свойств построенных графов: только к половине методов авторы привели соответствующую статистику. В случае Zep- и T-GRAG-методов получены следующие оценки скорости обработки одного user-вопроса по Latency-метрике: 3.20 (GPT4o-mini) и 2.72 (Lama3.1 8B) секунд соответственно. Несмотря на наличие (в половине статей) информации о характеристиках построенных графов, отсутствует высокоуровневый анализ их качества: (1) количество единиц содержащейся полезной информации, (2) количество единиц потерянной информации при попытке её обработки

и сохранения в граф, (3) количество единиц актуальной и устаревшей информации, (4) процент ошибок разбора/парсинга ответов LLM-модели в рамках промежуточных шагов по сохранению информации в граф, (5) процент найденной устаревшей информации при обновлении/дополнении графа и т.п. Введение подобных метрик позволит выявить новые недостатки/ограничения у предлагаемых GraphConstruction-алгоритмов и ускорить процесс по их устранению. На данный момент качество построенного графа оценивается на основании полученных QA-оценок эффективности, что, очевидно, является недостаточным показателем.

В свою очередь, стоит обратить внимание на их temporal-свойства: только половина из разобранных методов дополнительно извлекает отметки времени из обрабатываемых текстов и использует $t_{created}$, $t_{expired}$ атрибуты у триплетов для поиска релевантной информации при решении QA-задач. При этом только у Zep-метод есть/предложена механика по учёту интервалов времени, в рамках которых, хранимая в графе, единицы информации считаются актуальными.

Также стоит обратить внимание на общий шаблон предлагаемых алгоритмов построения графов. Почти на каждом шаге используется LLM-модель, при этом за одну inference-операцию ей приходится решать достаточно сложную (составную) задачу. При использовании medium-size моделей (7B/8B) предложенные алгоритмы будут сильно деградировать по качеству в сравнении с теми же алгоритмами, но на основе large-size моделей (например, DeepSeek V3 или GPT4o-mini); из-за данного недостатка страдает стабильность, воспроизводимость и практическая применимость данных методов. Дополнительно, стоит отметить, что все рассмотренные методы используют только плотные (dense) векторные представления при оценке релевантности хранящихся знаний в графе (в виде текста на естественном языке) к входящим запросам. На практике было показано, что комбинация плотных (dense) и разреженных (sparse) векторных представлений повышает эффективность retrieval-операций и информационного поиска в целом.

1.3.4 Выводы

Основываясь на информации, структурированной в Таблице 1.2, можно выдвинуть следующую гипотезу: комбинация алгоритмов (1) унификации вершин/рёбер триплетов, (2) извлечения и унификации темпоральной информации, а также (3) поиска и фиксации устаревшей информации позволит повысить качество/стабильность построенного графа знаний и, как следствие, добавит эффективности при решении предметно-ориентированных (close domain) QA-задач.

1.4 Обзор GraphRetrieval-методов

1.4.1 Методология поиска статей

Формирование релевантного множества научных публикаций выполнено в три этапа. На первом этапе были использованы специализированные поисковые системы: SciSpace, Scite, PaperDigest, Consensus и Elicit. С помощью каждой системы были выбраны первые десять публикаций в поисковой выдаче по следующим четырём поисковым запросам: "RAG on Knowledge Graphs" , "Enhancing RAG—approach with Knowledge Graphs" , "Graph RAG" и "RAG with integration of Large Language Models (LLMs) and Knowledge Graphs (KGs)". При этом поисковая выдача была ограничена годом публикации: до 2019. На втором этапе была выполнена фильтрация отобранных работ на основе их кратких описаний (abstracts): удалены дубликаты; исключены работы, в которых авторы презентуют свой датасет в качестве тестового набора данных; исключены работы, где предлагаемый метод заточен на конкретную прикладную задачу. В результате было получено 19 статей. На третьем этапе отбирались работы, в которых: детально описан предложенный метод и используются общепризнанные тестовые наборы данных (benchmarks). В результате было сформировано множество из 10 статей: в Таблице 1.3 отобранные работы представлены в хронологическом порядке.

Таблица 1.3 — Хронология статей, на основе которых выполнен обзор литературы по GraphRetrieval-методам

N	Название метода	Дата публикации	Реализация метода
1	ToG [16]	15.07.23	https://github.com/IDEA-FinAI/ToG
2	RoG [17]	02.10.23	https://github.com/RManLuo/reasoning-on-graphs
3	PMKGE [18]	15.02.24	отсутствует
4	KG-RAG [19]	20.05.24	https://github.com/dsanmart/KG-RAG
5	GRAG [20]	26.05.24	https://github.com/HuieL/GRAG
6	GNN-RAG [21]	30.05.24	https://github.com/cmavro/GNN-RAG
7	ToG 2.0 [22]	15.07.24	https://github.com/GasolSun36/ToG
8	DoG [23]	05.09.24	https://github.com/rem1-group/DoG
9	GCR [24]	16.10.24	https://github.com/RManLuo/graph-constrained-reasoning
10	PDA [25]	16.10.24	отсутствует

1.4.2 Описание существующих методов

1.4.2.1 ToG

В статье [16] авторы предложили ToG-метод (LLMs "Think" along the reasoning paths "on" knowledge "graph" step-by-step), который использует языковую модель для интерактивного исследования вершин/связей графа знаний и выполнения рассуждений о следующих действиях на основе извлечённой информации. Можно выделить следующие ключевые/общие техники, которые были использованы в данном методе: "Prompt Engineering" , "LLM-base filtering" , "Dense Embeddings-based Nodes Linking".

Способ обхода графа основывается на идее BeamSearch-алгоритма: см. Рисунок 1.6. В качестве шага инициализации из user-вопроса с помощью LLM-модели извлекаются тематические сущности (topic entites) и сопоставляются с вершинами графа знаний. Далее выполняется итеративный алгоритм обхода графа для построения путей рассуждений (reasoning paths), в которых должна содержаться информация для ответа на user-вопрос. Алгоритм обхода графа состоит из двух шагов: (1) поиск релевантных связей (Relation Exploration) и (2) поиск релевантных сущностей (Entity Exploration). На каждом из шагов формируется набор связей/вершин, которые напрямую связаны с последними (tail) вершинами/связями у построенных путей на предыдущих итерациях. После получения набора возможных кандидатов вершин/связей выполняется их фильтрация с помощью LLM-модели на основе user-вопроса. В результате фильтрации оставшиеся N вершин/связей добавляются в соответствующие пути рассуждений. В завершении итерации выполняется проверка (с помощью LLM-модели) построенных путей: если содержащейся в них информации достаточно, чтобы ответить на user-вопрос, то осуществляется переход на финальный этап работы алгоритма для исполнения соответствующей inference-задачи (QA) с помощью LLM-модели, иначе выполняется переход на новую итерацию обхода графа.

Предложенный метод позволил получить следующий прирост оценки качества по EM-метрике в сравнении с baseline-методом (KD-CoT [26]) на датасетах CWQ и WebQSP: на 11.9 (67.6 - 55.7) и 14 (82.6 - 68.6) процентов соответственно. В статье описаны показатели эффективности предложенного метода при использовании следующих LLM-моделей в качестве основных text-генераторов: GPT3.5-turbo, GPT4, Llama2:70B-Chat. Оценка эффективности метода была выполнена на следующих датасетах: CWQ, WebQSP, GrailQA, QALD10-en, Simple Questions, Web Questions, T-REx, Zero-Shot RE, Creak. В качестве метрик, по которым оценивалась эффективность метода была использована только Hits@1 (Exact Match). К недостаткам ToG-метода можно отнести следующие моменты:

1. Для фильтрации вершин/связей графа используется LLM-модель, что существенно замедляет reasoning-процесс.
2. Извлечение информации осуществляется на основе исходного user-вопроса без дополнительных этапов модификации/корректировки (preprocessing). В случае, ес-

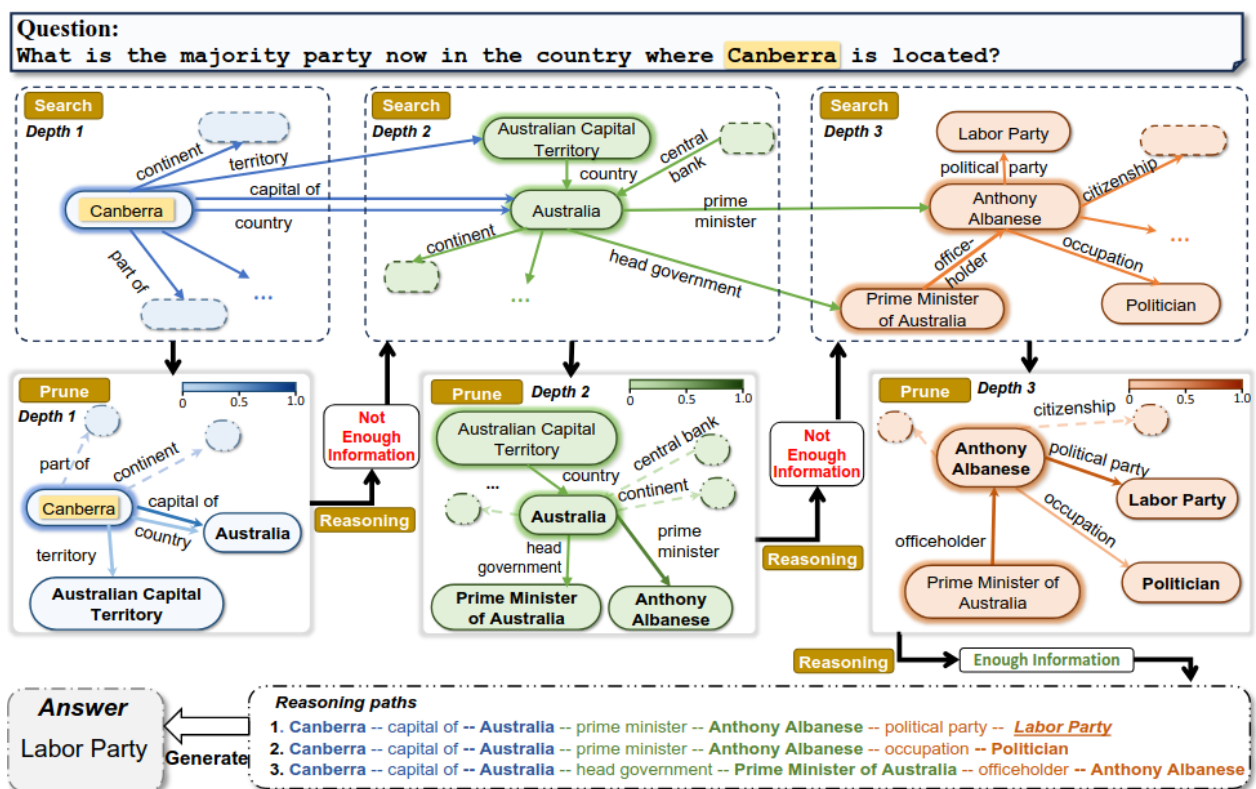


Рисунок 1.6 — Иллюстрация работы ToG-метода на примере. Диаграмма взята из оригинальной статьи [16]

ли вопрос содержит шум или является неполным, то это значительно повлияет на качество генерируемого ответа.

3. Выполняется однозначное сопоставление сущностей из вопроса с вершинами графа. Не рассматривается ситуация, когда для сущности из вопроса нет релевантной вершины из графа знаний.
4. Все построенные пути сразу подаются на вход финальной LLM-модели для генерации ответа без дополнительной стадии сжатия/резюмирования полученной информации: "Lost in the middle"-дилемма.

1.4.2.2 RoG

В статье [17] авторы предложили функцию ошибки для дообучения одной LLM-модели на две задачи: (1) генерация релевантных цепочек связей (Relation Paths) по user-вопросу для сопоставления с триплетом в графе знаний и (2) генерация финального ответа на user-вопрос по цепочкам извлечённых триплетов (Reasoning Paths). При дообучении на (1)-задачу выполняется обуславливание на связи/вершины в имеющемся графе знаний. Можно выделить следующие ключевые/общие техники, которые были использованы в данном методе: "Prompt Engineering", "End-to-end Model Architecture", "LLM Fine-Tuning". Пример работы предложенного метода представлен на Рисунке 1.7.

Предложенный метод позволил получить следующий прирост оценки качества

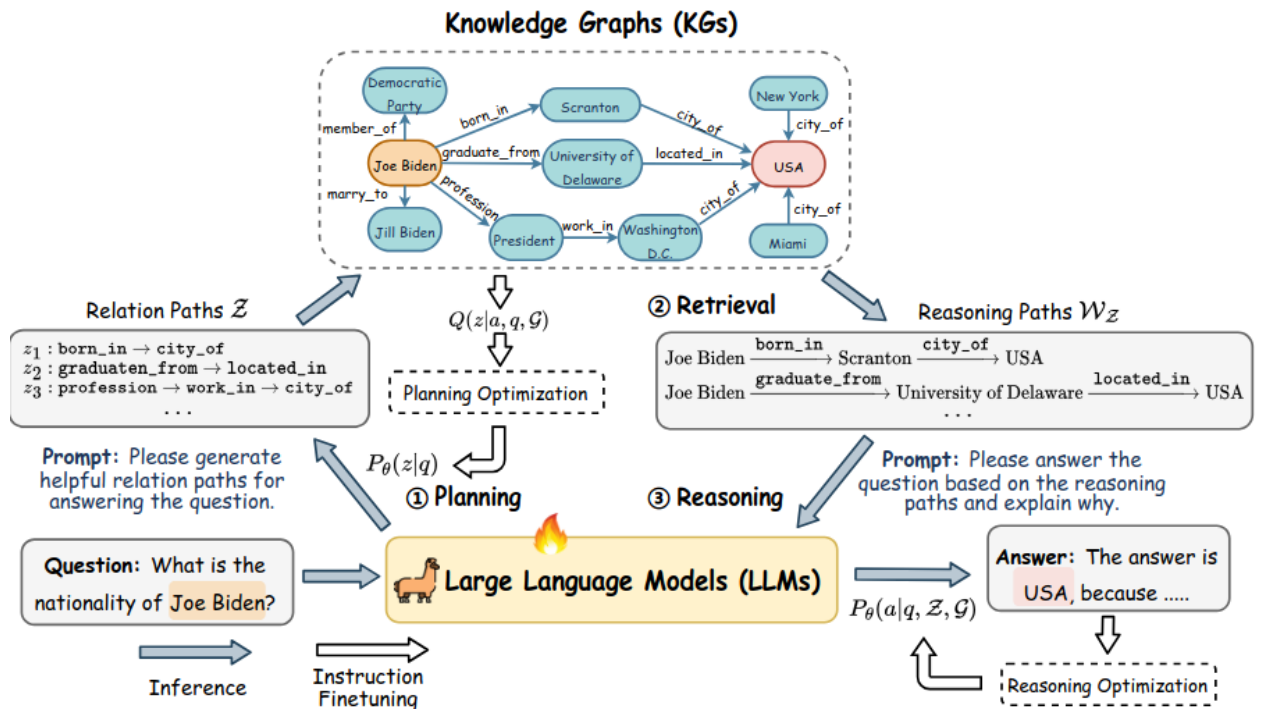


Рисунок 1.7 — Иллюстрация работы RoG-метода на примере. Диаграмма взята из оригинальной статьи [17]

по EM-метрике в сравнении с baseline-методом (KD-CoT [26]) на датасетах CWQ и WebQSP: на 6.9 (62.6 - 55.7) и 17.1 (85.7 - 68.6) процентов соответственно. В статье описаны показатели эффективности предложенного метода при использовании Llama2:7B-Chat LLM-модели в качестве основного text-генератора. Оценка эффективности метода была выполнена на следующих датасетах: CWQ, WebQSP. В качестве метрик, по которым оценивалась эффективность метода были использованы следующие: Hits@1 (Exact Match) и F1. К недостаткам RoG-метода можно отнести следующие моменты:

1. При добавлении новой или удалении старой информации из графа знаний потребуется заново выполнить дообучение используемой LLM-модели.
2. За один inference-шаг LLM-модели сразу генерируются все цепочки связей, по которым должна храниться релевантная информация для получения ответа на user-вопрос. Из-за сложности решаемой задачи и "Lost in the middle"-дилеммы сгенерированный набор цепочек будет содержать большое количество галлюцинаций. В свою очередь инкапсуляция задачи в виде одного промпта для LLM-модели затрудняет процедуру интерпретации/отладки полученного результата.
3. Для извлечения цепочек рассуждений из графа выполняется однозначное сопоставление сгенерированных путей со связями/вершинами в графе: сгенерированные пути, которые содержат связи-синонимы не рассматриваются.
4. Отсутствует конкретный механизм предобработки/чистки исходного user-вопроса. Если вопрос содержит шум или требует комплексного (составного) анализа/резюмирования некоторой предметной области, то система не сможет сгенерировать релевантный ответ: он будет неполным, содержать галлюцинации.

1.4.2.3 PMKGE

В статье [18] авторы предложили PMKGE-метод, который позволяет скорректировать и верифицировать внутренние знания LLM-модели для генерации ответа на user-вопрос. Метод состоит из трёх основных стадий (рисунок 1.8): (1) генерация псевдо-графа (Pseudo-Graph Generation), (2) верификация сгенерированного графа с помощью имеющегося графа знаний (Atomic Knowledge Verification) и (3) генерация финального ответа на user-вопрос на основе верифицированного графа. Можно выделить следующие ключевые/общие техники, которые были использованы в данном методе: "Prompt Engineering", "Knowledge Graph Generation", "KG Pruning/Enhancing".

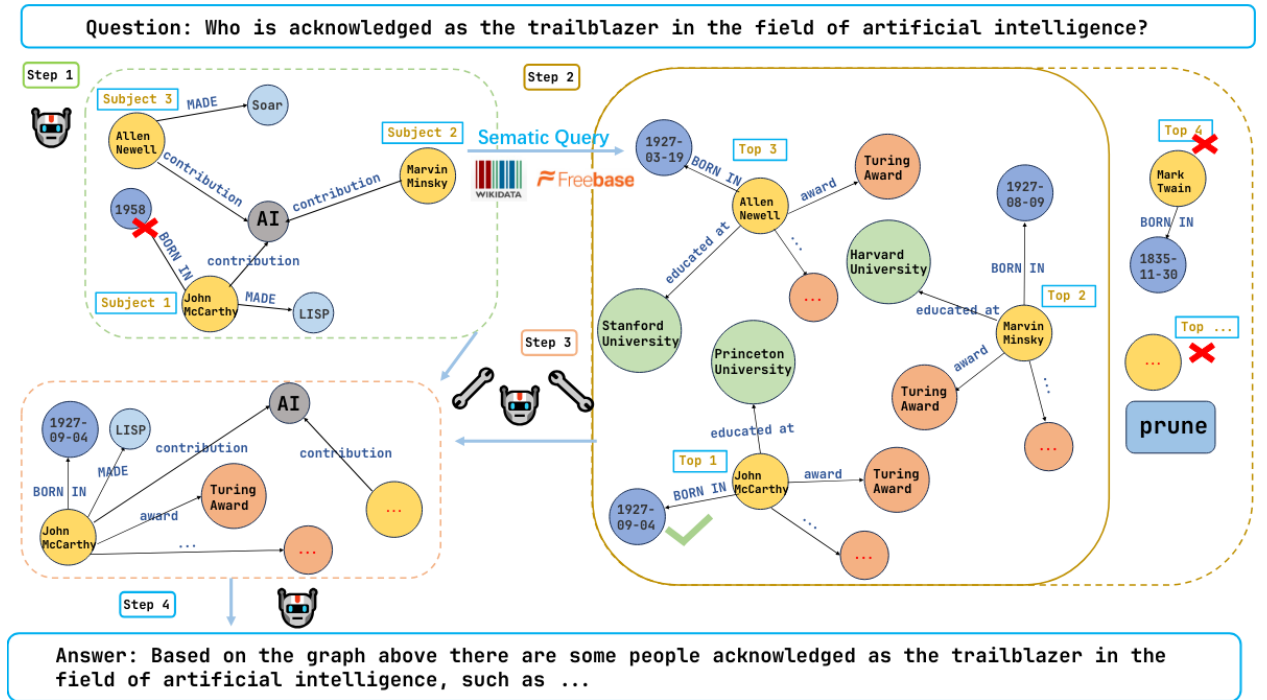


Рисунок 1.8 — Иллюстрация работы PMKGE-метода на примере. Диаграмма взята из оригинальной статьи [18]

На первой стадии с помощью LLM-модели и подготовленного LLM-промпта генерируется псевдо-граф G_p , на основе которого можно получить ответ на user-вопрос. Граф генерируется в формате Cypher-языка запросов для базы данных Neo4j. Полученный граф может быть неполным или содержать галлюцинации, поэтому его необходимо сравнить/дополнить с имеющимся графом знаний G_{base} . Вторая стадия метода состоит из трёх шагов:

1. Для каждого триплета из G_p выбирается N ближайших триплетов из G_{base} по метрике косинусной близости их векторных представлений. За счёт этого формируется промежуточный граф G_t . Однако, если G_p содержал большое количество триплетов, то размер G_t может превысить длину контекстного окна используемой LLM-модели для генерации ответа, поэтому на следующем шаге применяются две pruning-техники.
2. Выполняется фильтрация вершин из G_t по значению их степени: количеству рё-

бер, инцидентных данной вершине. В графе оставляется k -вершин, где k соответствует количеству вершин в G_p . Далее вершины фильтруются по "entity confidence score"-метрике, которая считается как среднее значений косинусной близости векторных представлений инцидентных триплетов для данной вершины к user-вопросу. Если вершина имеет значение метрики ниже 0.7, то она удаляется из G_t вместе с инцидентными рёбрами.

3. В результате двух предыдущих шагов получен граф G_g с концентрированным набором информации для ответа на user-вопрос. Выполняется корректировка/верификация G_p на основе G_g с помощью LLM-модели и подготовленного LLM-промпта.

В результате будет получен верифицированный граф G_f . На последней третьей стадии метода выполняется генерация ответа на user-вопрос на основе G_f с помощью LLM-модели.

Предложенный метод позволил получить следующий прирост оценки качества по EM-метрике в сравнении с baseline-методом IO [27] на датасетах SimpleQuestions и QALD-10: на 10.1 (40.0 - 29.9) и 11.8 (56.5 - 44.7) процентов соответственно. В статье описаны показатели эффективности предложенного метода при использовании следующих LLM-моделей в качестве основных text-генераторов: GPT3.5, GPT4. Оценка эффективности метода была выполнена на следующих датасетах: SimpleQuestions, QALD-10 и Nature Questions. В качестве метрик, по которым оценивалась эффективность метода были использованы следующие: Hits@1 (Exact Match) и ROUGE L. К недостаткам PMKGE-метода можно отнести следующие моменты:

1. Верифицированный граф G_f строится на основе триплетов из сгенерированного графа G_p . В случае, если G_p будет содержать некорректные цепочки триплетов, то в верифицированном графе они также будут присутствовать, что негативным образом повлияет на качество генерируемого ответа.
2. Перед запуском алгоритма отсутствует стадия по модификации/корректировке исходного user-вопроса. В случае если user-вопрос будет сложным (составным) или будет содержать шум, то это негативным образом скажется на качестве сгенерированного и верифицированного графов соответственно и значительно снизит релевантность финального ответа системы.
3. При корректировке/дополнении G_p на основе G_g , несмотря на применение pruning-техник, может быть получен достаточно длинный LLM-промпт, что приведёт к потере части информации при генерации G_f : "Lost in the middle"-дилемма.
4. Для каждого сгенерированного триплета из G_p обязательно извлекается фиксированное количество триплетов из G_{base} : в методе не предусмотрены обработчики для ситуаций, когда в G_{base} нет релевантных триплетов.
5. Для составных "multi-hop"-вопросов, которые требуют нескольких шагов рассуждений на основе последовательно-поступающей информации, будет получено низкое качество ответов, так как PMKGE-метод предполагает поиск информации и

генерацию ответа в рамках одной итерации.

6. Предложенный метод плохо подходит для генерации ответов в рамках специализированной области, где требуется анализ и резюмирование имеющейся информации. Ввиду того, что G_p генерируется только на основе внутренних знаний LLM-модели на стадии верификации часть важной информации из G_{base} может быть пропущена, так как она не упоминается (в том или ином виде) в G_p -триплетах.

1.4.2.4 KG-RAG

В статье [19] авторы предложили "Chain of Exploration"-алгоритм (CoE), который использует возможности LLM-моделей для последовательного поиска и анализа вершин/связей в графе знаний для генерации ответа на user-вопрос. Пример работы предложенного метода представлен на рисунке 1.9.

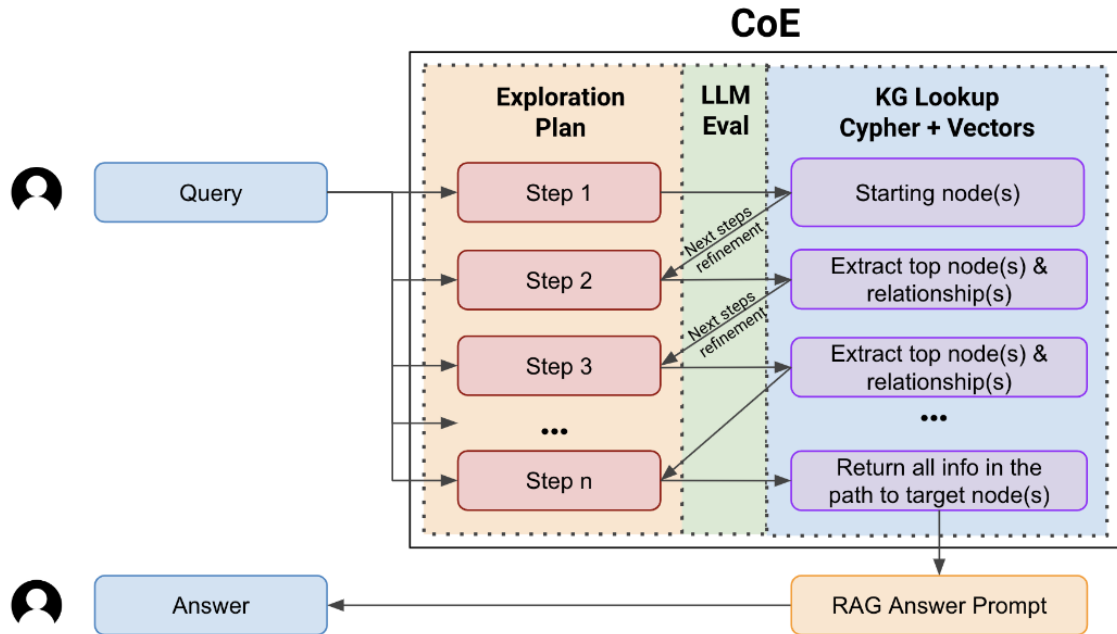


Рисунок 1.9 — Иллюстрация работы KG-RAG-метода на примере. Диаграмма взята из оригинальной статьи [19]

Алгоритм CoE состоит из трех основных стадий: планирования, поиска и оценки. Стадия планирования включает генерацию последовательности шагов, которые направляют процесс поиска в имеющемся графе знаний. Поиск релевантных вершин и связей в графе реализован в виде комбинации семантического поиска (по векторным представлениям триплетов) и фильтрации найденных триплетов-кандидатов с помощью LLM-модели. На стадии оценки выполняется проверка текущего набора извлечённой информации с помощью user-вопроса. На данной стадии принимается решение: продолжить исследование графа по текущему плану, регенерировать план и начать поиск заново или сгенерировать финальный ответ по user-вопросу на основе извлечённой информации.

Предложенный метод позволил получить значение EM-метрики на CWQ-

датасете, сравнимое с полученным значением для baseline-метода (Embedding-RAG): 19.0 и 28.0 соответственно. Авторы утверждают, что такой результат работы их алгоритма был связан с низким качеством текстов в CWQ-датасете. К недостаткам KR-RAG-метода можно отнести следующие моменты:

1. Несмотря на наличие шага планирования, в рамках которого потенциально-сложный (составной) user-вопрос разбивается на подзапросы для поиска конкретной информации, отсутствуют обработчики ситуаций, когда user-вопрос состоит из нескольких независимых подвопросов. Обработка таких типов вопросов в последовательном/смешанном режиме в среднем потребует больше времени и потенциально может повлиять (негативно) на качество финального ответа.
2. Для фильтрации вершин/связей графа используется LLM-модель, что существенно замедляет reasoning-процесс CoE-алгоритма.

1.4.2.5 GRAG

В статье [20] авторы предложили GRAG-метод, который позволяет извлекать множество релевантных подграфов из имеющегося графа знаний и совместно интегрировать их как текстовую информацию, так и топологическую структуру в LLM-модель для повышения качества генерации ответа на user-вопрос (рисунок 1.10).

На этапе подготовки необходимых структур данных исходный граф знаний разбивается на ego-графы: такие графы, которые состоят из центральной вершины и всех вершин, напрямую (или через K -рёбер) связанных с центральной. Для полученного множества подграфов вычисляются их соответствующие векторные представления, которые в результате сохраняются в векторное хранилище. На этапе поиска релевантной информации выполняется извлечение N -подграфов на основе исходного user-вопроса по значению косинусной близости их векторных представлений. Далее для удаления нерелевантных триплетов из извлечённых подграфов применяется grouping-техника. В результате отфильтрованные подграфы объединяются в один граф. Перед началом генерации финального ответа для графа с предыдущего шага получается его векторное представление с помощью GNN-сети. Данное векторное представление, содержащее информацию о топологии вершин и рёбер извлечённого графа, конкатенируется к каждому скрытому векторному представлению токенов LLM-промпта, состоящего из user-вопроса и текстового представления извлечённого графа, перед их подачей на вход каждому decoder-слою LLM-модели при генерации финального ответа.

Предложенный метод позволил получить следующий прирост оценки качества по F1-метрике в сравнении с baseline-методом G-Retriever [28] на датасете WebQSP: на 4.67 (72.75 - 68.08) процентов. К недостаткам GRAG-метода можно отнести следующие моменты:

1. При изменении информации в графе знаний потребуется пересчитать векторные представления для части проиндексированных ego-графов.
2. Отсутствуют шаги по предобработке исходного user-вопроса для конкретизации

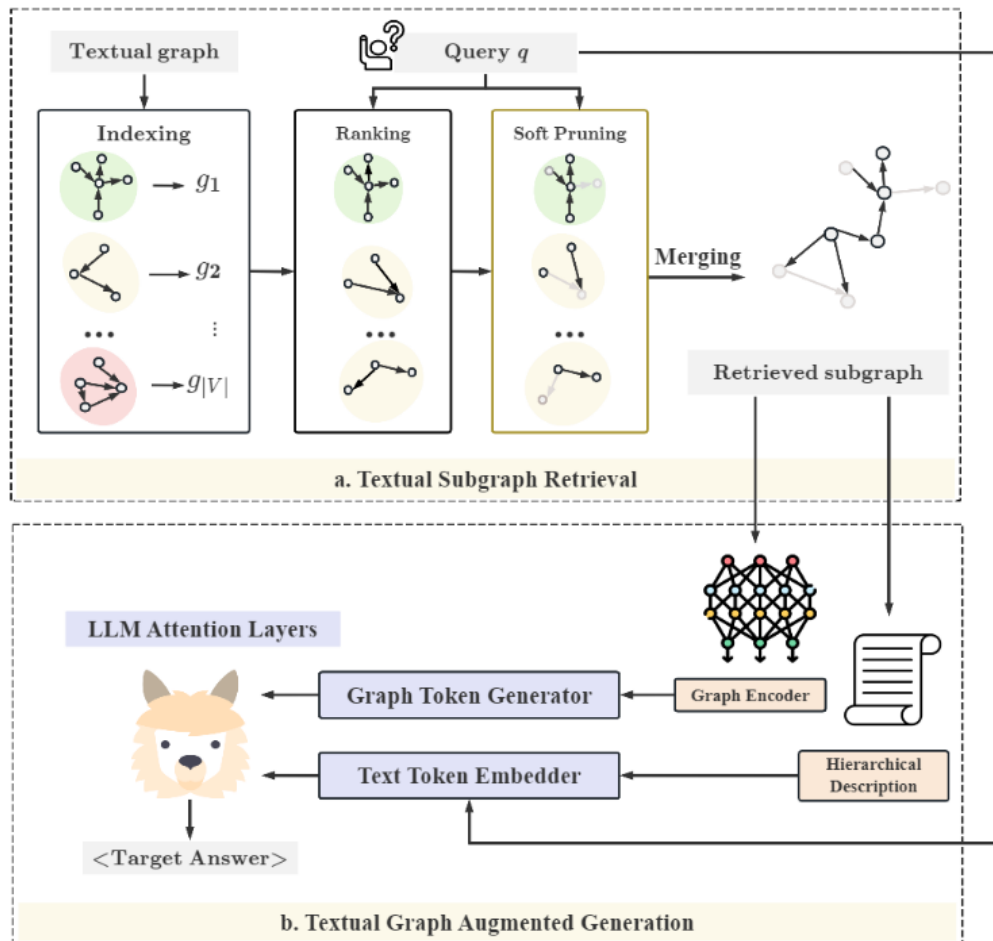


Рисунок 1.10 — Иллюстрация работы GRAG-метода на примере. Диаграмма взята из оригинальной статьи [20]

запрашиваемой информации и повышения качества поиска.

3. Ширина извлекаемых подграфов, как и их количество, фиксировано.
4. В методе не предусмотрены обработчики ситуаций, когда в графе знаний нет релевантной информации для генерации правильного ответа на user-вопрос. В результате LLM-модель будет обуславливаться на свои внутренние знания о мире, что может привести к генерации ответа с неполной, устаревшей или ложной информацией.

1.4.2.6 GNN-RAG

В статье [21] авторы предложили "GNN-RAG"-метод, который комбинирует свойства, связанные с пониманием естественного языка, LLM-моделей и информацию об отношениях/связях объектов графа, содержащуюся в GNN-сетях, для решения KGQA-задачи.

Предложенный алгоритм поиска/генерации ответа на user-вопрос можно разбить на четыре основных стадии: представлено на рисунке 1.11. На первой стадии с помощью GNN-сети на основе user-вопроса выполняется поиск вершин, которые могут быть потенциальными ответами: решается задача бинарной классификации. На вто-

рой стадии с помощью LLM-модели извлекаются ключевые сущности из user-вопроса. Далее выполняется однозначное сопоставление сущностей из вопроса с вершинами графа. На третьей стадии между всеми парами вершин-кандидатов и вершин, сопоставленных сущностям из вопроса, строятся кратчайшие пути. На финальной четвёртой стадии из построенных путей извлекается текстовая информация и подаётся вместе с user-вопросом на вход LLM-модели для генерации ответа.

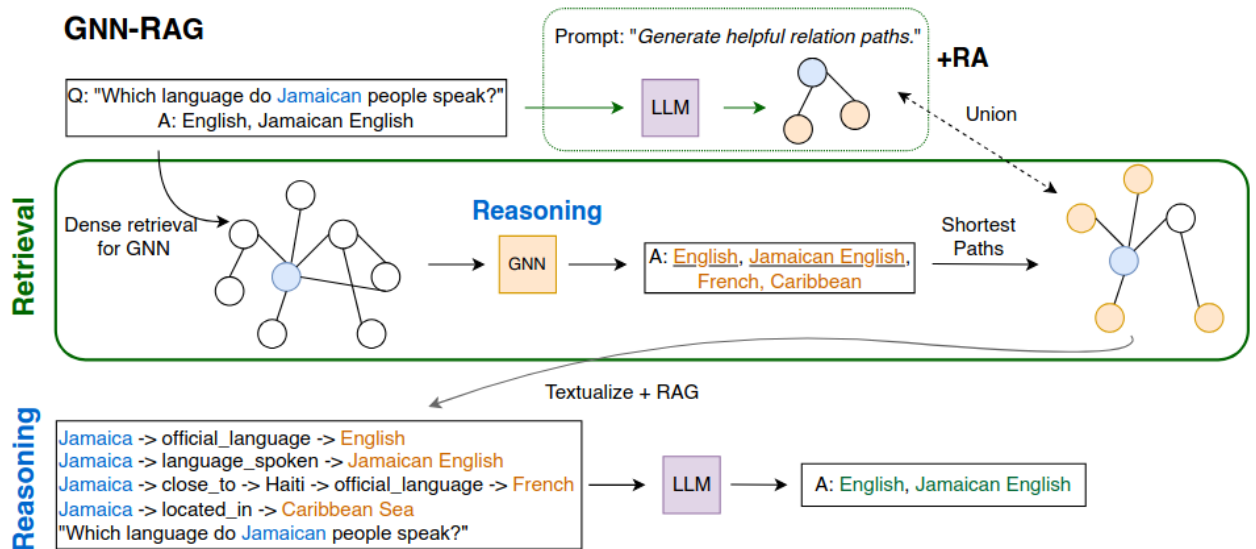


Рисунок 1.11 — Иллюстрация работы GNN-RAG-метода на примере. Диаграмма взята из оригинальной статьи [21]

Предложенный метод позволил получить следующий прирост оценки качества по EM-метрике в сравнении с baseline-методом KD-CoT [26] на датасетах WebQSP и CWQ: на 17.1 (85.7 - 68.6) и 11.1 (66.8 - 55.7) процентов соответственно. К недостаткам "GNN-RAG"-метода можно отнести следующие моменты:

1. Отсутствует стадия предобработки исходного user-вопроса для декомпозиции процесса поиска на независимые части, который позволит ускорить работу системы и повысить качество финального ответа.
2. Каждой сущности из user-вопроса ставится в соответствие одна вершина из граф знаний. Не рассматриваются ситуации, когда сущность является обобщённым объектом и ей соответствует несколько вершин из графа, а также когда для сущности нет релевантной вершины.
3. Метод использует необоснованную эвристику о том, что на кратчайших путях между отобранными вершинами содержится необходимая информация для ответа на user-вопрос. Качество работы данной эвристики сильно зависит от способа построения графа знаний, в котором будет осуществляться поиск.
4. После построения/извлечения путей из графа нет дополнительной pruning-стадии для фильтрации нерелевантных триплетов, которые могут внести дополнительный шум в LLM-промпт и снизить качество генерируемого ответа.
5. Для составных "multi-hop"-вопросов, которые требуют нескольких шагов рассуждений на основе последовательно-поступающей информации, будет получено низ-

кое качество ответов, так как "GNN-RAG"-метод предполагает поиск релевантной информации и генерацию ответа в рамках одной итерации.

1.4.2.7 ToG2.0

В статье [22] авторы предложили модификацию ToG-метода. Можно выделить три ключевых изменения (рисунок 1.12): (1) двухэтапное извлечение ключевых сущностей из user-вопроса, (2) генерация дополнительного вопроса (clue queries) для каждой tail-вершины на текущей итерации алгоритма и (3) фильтрация вершин на основе семантической близости связанных с ними полносвязных текстов (контекстов) к соответствующим доп. вопросам и исходному user-вопросу. Можно выделить следующие ключевые/общие техники, которые были использованы в данном методе: NER, "Prompt engineering", "Dense Embeddings-based Nodes Linking", "LLM-base Relations Filtering".

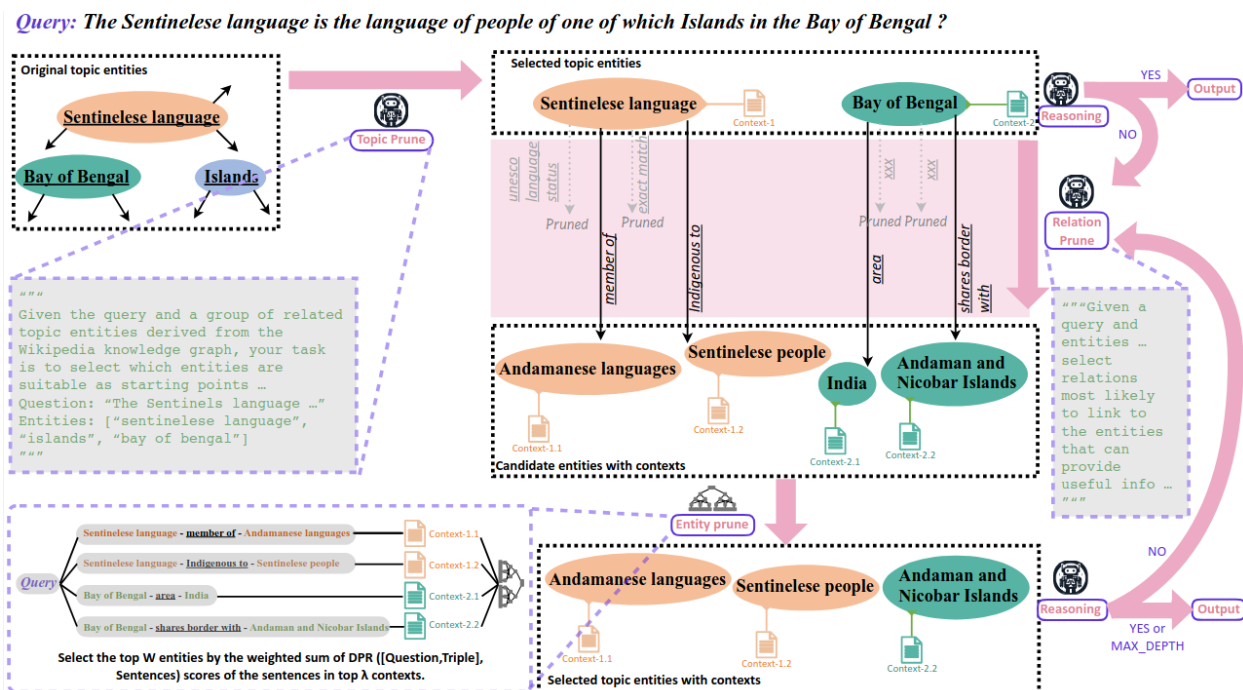


Рисунок 1.12 — Иллюстрация работы ToG-метода на примере. Диаграмма взята из оригинальной статьи [22]

В качестве способа извлечения ключевых сущностей из user-вопроса сначала применяется NER-модель, а потом применяется LLM-модель для их фильтрации и выбора самых релевантных. В начале каждой итерации обхода графа для каждой tail-вершины у построенных путей с помощью LLM-модели генерируется дополнительный вопрос, который корректирует стратегию отбора инцидентных связей и смежных вершин.

Предложенный метод позволил получить следующий прирост оценки качества по EM-метрике в сравнении с baseline-методом ToG [16] на датасете WebQSP: на 4.93 (81.13 - 76.20) процентов. В статье описаны показатели эффективности предложенного метода при использовании GPT3.5-turbo LLM-модели в качестве основного text-

генератора. Оценка эффективности метода была выполнена на следующих датасетах: WebQSP, HotpotQA, QALD-10-en, FEVER, Creak, Zero-Shot RE. В качестве метрик, по которым оценивалась эффективность метода были использованы следующие: Hits@1 (Exact Match) и Accuracy. К недостаткам ToG2.0-метода можно отнести следующие моменты:

1. Для извлечения сущностей требуется обученная NER-модель.
2. Для фильтрации связей графа используется LLM-модель, что существенно замедляет reasoning-процесс.
3. Выполняется однозначное сопоставление сущностей из вопроса с вершинами графа. Не рассматриваются ситуации, когда для сущности из вопроса нет релевантной вершины в графе знаний.
4. На финальной стадии все построенные пути сразу подаются на вход LLM-модели для генерации ответа без дополнительной стадии сжатия/резюмирования полученной информации: "Lost in the middle"-дилемма.

1.4.2.8 DoG

В статье [23] авторы предложили DoG-метод (Dabate on Graph), который позволяет в интерактивном режиме взаимодействовать LLM-модели с имеющимся графом знаний для итеративного поиска необходимой информации и генерации ответа на user-вопрос. Можно выделить следующие ключевые/общие техники, которые были использованы в данном методе: "Prompt Engineering" , "Dense Embeddings-based Nodes Linking" , "MultiAgency" , "Sub-questions Generation".

Как показано на рисунке 1.13, имея K hor-вопрос q и начальную тематическую сущность e_i , извлечённую из q , выполняется сопоставление e_i с вершиной графа знаний и получение набора её инцидентных связей R . Затем с помощью LLM-модели выбирается одна релевантная связь r из R с соответствующей инцидентной tail-вершиной e_{i+1} . Далее на основе полученного триплета и текущего K hor-вопроса с помощью LLM-модели принимается решение о последующем действии: перейти на этап генерации ответа на исходный user-вопрос или сгенерировать новый подвопрос и выполнить по нему поиск в рамках новой итерации. В случае реализации последнего сценария, группа из трёх LLM-агентов, играющих роли эксперта, критика и лингвиста, на основе текущего триплета выполняют преобразование K hor-вопроса в $K - 1$ hor-вопрос (упрощённый) за счёт его последовательного перефразирования/уточнения, при этом tail-сущность e_{i+1} является последующей тематической сущностью для начала обхода графа.

Предложенный метод позволил получить следующий прирост оценки качества по EM-метрике в сравнении с baseline-методом KD-CoT [26] на датасетах WebQSP и WebQuestions: на 14.8 (91.0 - 76.2) и 25.5 (80.0 - 54.5) процентов соответственно. В статье описаны показатели эффективности предложенного метода при использовании следующих LLM-моделей в качестве основных text-генераторов: GPT3.5-turbo, Qwen2:14B,

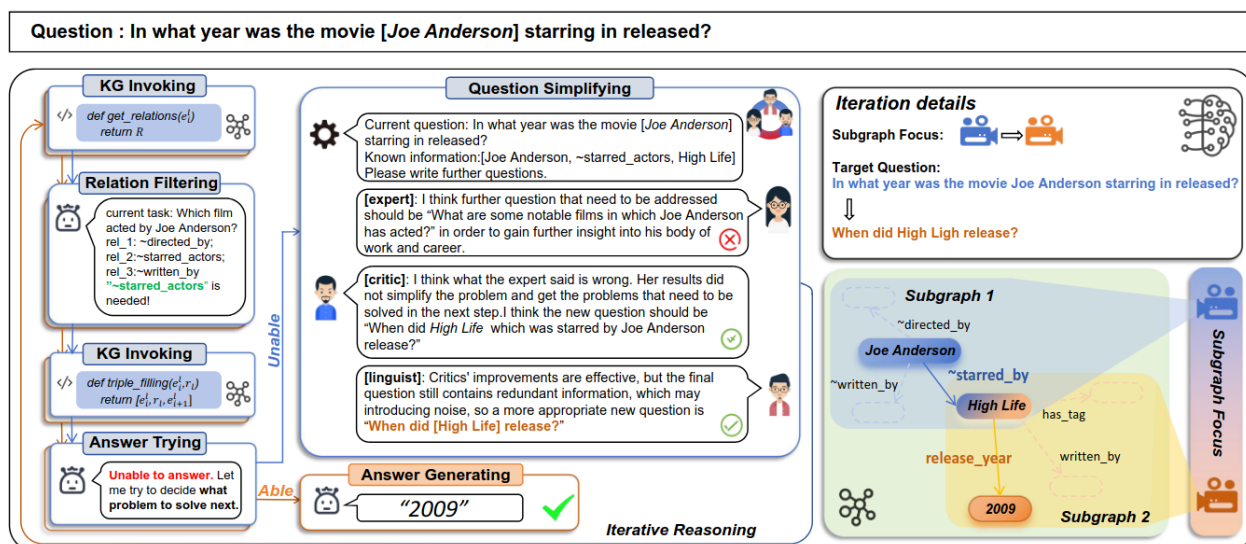


Рисунок 1.13 — Иллюстрация работы DoG-метода на примере. Диаграмма взята из оригинальной статьи [23]

Llama3:8B, GPT4. Оценка эффективности метода была выполнена на следующих датасетах: MetaQA, WebQSP, CWQ, Web Questions, GrailQA. В качестве метрик, по которым оценивалась эффективность метода были использованы следующие: Hits@1 (Exact Match) и F1. К недостаткам DoG-метода можно отнести следующие моменты:

1. Обход графа знаний осуществляется строго по одному пути. В случае, если вопрос предполагает проведение анализа/резюмирования некоторой информации по нескольким объектам, то информации, извлечённой из графа по одному пройденной пути, с большой вероятностью, будет недостаточно для генерации полного ответа.
2. Несмотря на наличие стадии, в рамках которой потенциально-сложный (составной) user-вопрос разбивается на более простые вопросы для поиска конкретной информации, отсутствуют обработчики ситуаций, когда user-вопрос состоит из нескольких независимых под-вопросов. Обработка таких типов вопросов в последовательном/смешанном режиме в среднем потребует больше времени и потенциально может повлиять (негативно) на качество финального ответа.

1.4.2.9 GCR

В статье [24] авторы предложили алгоритм, который позволяет LLM-модели сразу генерировать релевантный набор цепочек рассуждений за счёт обуславливания используемой стратегии (decoding strategy) для предсказания следующего токена в последовательности на имеющийся граф знаний (рисунок 1.14.) Для генерации верифицированных цепочек рассуждений используется специализированная LLM-модель (KG-specialized LLM), которая была дообучена на данную задачу. Для интеграции имеющегося графа знаний в decoding-процесс LLM-модели он переводится в специальную структуру данных: префиксное дерево (KG-Trie). После получения релевантных цепочек

чек рассуждений они вместе с исходным user-вопросом подаются на вход общей LLM-модели, которая генерирует финальный ответ.

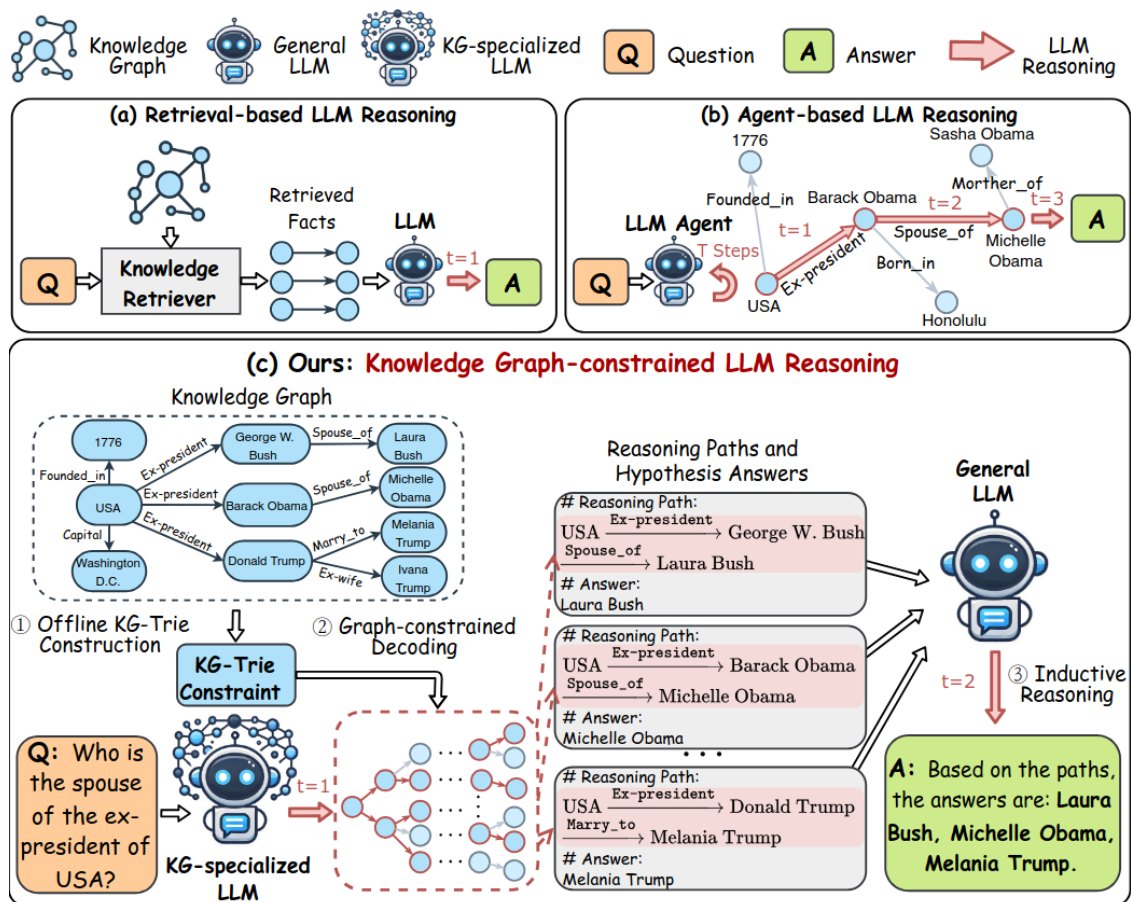


Рисунок 1.14 — Иллюстрация работы GCR-метода на примере. Диаграмма взята из оригинальной статьи [24]

Предложенный GCR-метод (Graph-constrained Reasoning) позволил получить следующий прирост оценки качества по EM-метрике в сравнении с baseline-методом (GNN-RAG [21]) на датасетах WebQSP и CWQ: на 6.9 (92.6 - 85.7) и 5.9 (72.7 - 66.8) процентов соответственно. К недостаткам GCR-метода можно отнести следующие моменты:

1. Потребуется обучить "KG-specialized LLM"-модель на имеющемся графе знаний.
2. При изменении информации в графе знаний также потребуется перестроить префиксное дерево (KG-Trie).
3. Извлечение информации осуществляется на основе исходного user-вопроса без предобработки. В случае, если вопрос содержит шум или является неполным, то это значительно повлияет на качество генерируемого ответа.
4. Для составных "multi-hop"-вопросов, которые требуют нескольких шагов рассуждений на основе последовательно-поступающей информации, будет получено низкое качество ответов, так как GCR-метод предполагает поиск релевантной информации и генерацию ответа в рамках одной итерации.
5. Количество путей рассуждений, которые извлекает/генерирует "KG-specialized

LLM"-модель фиксируется заданными значениями гиперпараметров beamsearch-стратегии генерации.

1.4.2.10 PDA

В статье [25] авторы предложили PDA-метод (Pyramid-Driven Alignment), использующий 5W1H-методику, которая применяется при описании и анализе проблемы путём получения ответов на связанные вопросы, для генерации плана поиска и извлечения информации из графа знаний с целью получения ответа на исходный user-вопрос. Метод может быть разбит на две основные стадии (рисунок 1.15): генерация плана поиска (Pyramid Alignment) и итеративный обход графа знаний с целью сбора релевантной информации и ответа на user-вопрос (KG Reasoning). Можно выделить следующие ключевые/общие техники, которые были использованы в данном методе: "Prompt Engineering" , "Dense Embeddings-based Nodes Linking" , "Search Planning".

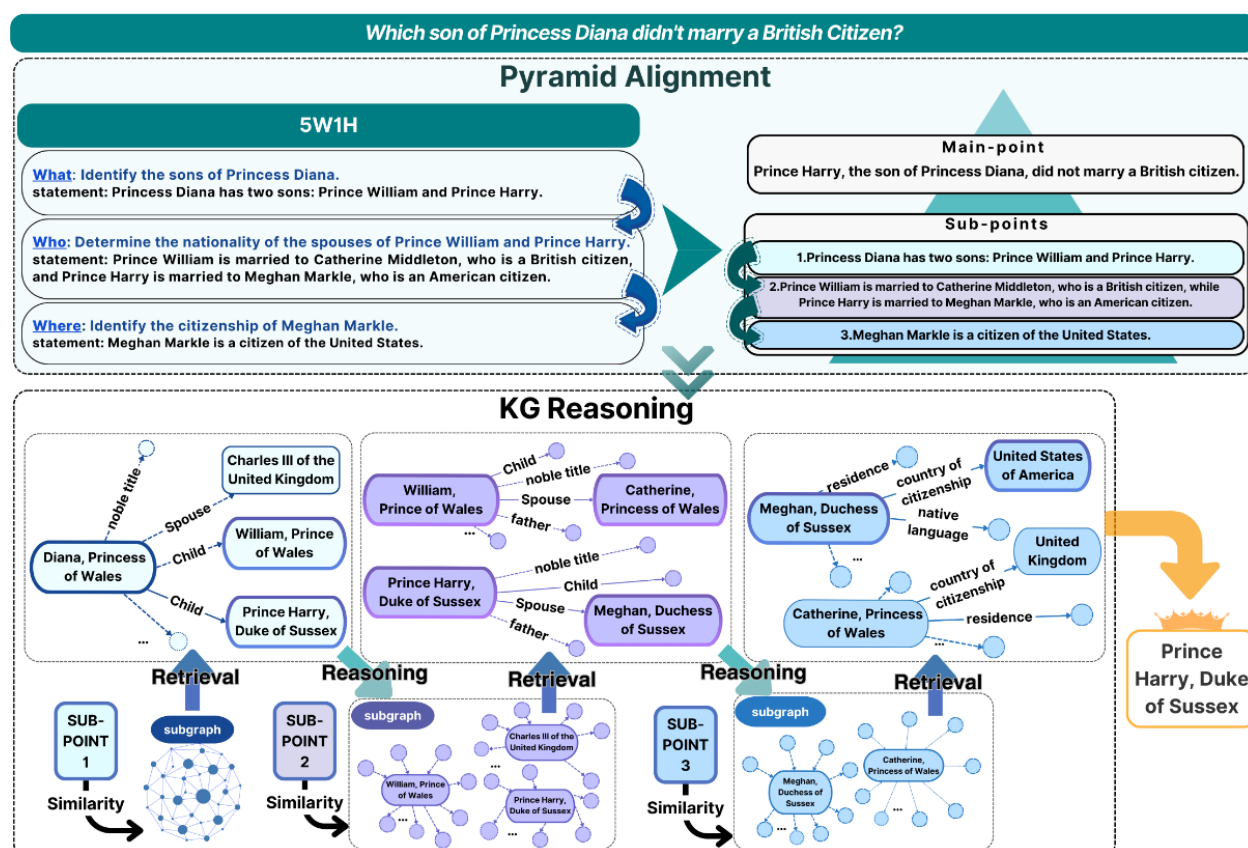


Рисунок 1.15 — Иллюстрация работы PDA-метода на примере. Диаграмма взята из оригинальной статьи [25]

На первой стадии с помощью LLM-модели в рамках одного inference-шага по 5W1H-методике генерируется законченный план поиска/уточнения информации по исходному user-вопросу. При этом в плане уже содержатся ответы (statements) на уточняющие 5W1H-вопросы. Полученные statement-ответы будут использоваться на следующей стадии для осуществления поиска по графу и извлечения верифицированных знаний. На второй стадии, имея список из statement-ответов, выполняется итеративный

обход графа. В качестве шага инициализации из первого statement-ответа с помощью LLM-модели извлекаются ключевые сущности и однозначно сопоставляются с вершинами в графе знаний. Далее выполняются Retrieval- и Reasoning-шаги. На Retrieval-шаге из графа отбираются все триплеты, которые содержат tail-вершины из путей в рамках текущей i -ой итерации. На основе данных триплетов формируется G_i подграф. Строковые представления триплетов в данном подграфе сортируются по значению косинусной близости их векторных представлений с i -ым statement-ответом; из них отбираются N релевантных триплетов T_i . На Reasoning-шаге с помощью полученных триплетов T_i выполняется корректировка поиска. Триплеты, которые инцидентны tail-вершинам из T_i формируют подграф G_{i+1} для выполнения Retrieval-шага в рамках следующей итерации. В результате такого последовательного обхода будет получен набор триплетов $T = \sum_{i=0}^k T_i$, на основе которого с помощью LLM-модели будет сгенерирован ответ на исходный user-вопрос.

Предложенный метод позволил получить следующий прирост оценки качества по EM-метрике в сравнении с baseline-методом KD-COT [26] на датасете WebQSP: на 4.65 (73.25 - 68.60) процентов. В статье описаны показатели эффективности предложенного метода при использовании следующих LLM-моделей в качестве основных text-генераторов: GPT3.5-turbo, Qwen2:14B, Llama3:8B, GPT4. Оценка эффективности метода была выполнена на следующих датасетах: WebQSP, Mintaka, 2WikiMultiHopQA. В качестве метрик, по которым оценивалась эффективность метода была использована только Hits@1 (Exact Match). К недостаткам PDA-метода можно отнести следующие моменты:

1. План поиска генерируется один раз: никак не учитывается информация, которая извлекается на промежуточных этапах обхода графа знаний.
2. На Retrieval-шаге извлечение триплетов T_i осуществляется на основе текущего statement-ответа, который был сгенерирован LLM-моделью без дополнительного обуславливания на верифицированную информацию. В случае если внутренние знания LLM-модели являются устаревшими, неверными или неполными, то полученный statement-ответ будет содержать большое количество галлюцинаций, что негативным образом повлияет на релевантность извлекаемой информации и качество финального ответа системы.
3. Выполняется однозначное сопоставление сущностей из вопроса с вершинами графа на шаге инициализации поиска. Не рассматриваются ситуации, когда для сущности из вопроса нет релевантной вершины в графе знаний или когда сущность является обобщённым объектом и ей должно соответствовать несколько вершин.
4. На Retrieval-шаге извлекается фиксированное количество триплетов.
5. В методе не предусмотрены обработчики ситуации, когда в графе знаний нет релевантной информации для генерации правильного ответа на user-вопрос.
6. Несмотря на наличие стадии планирования отсутствует стадия корректировки/модификации исходного user-вопроса для декомпозиции процесса поиска на

независимые части, которая позволит ускорить работу системы и повысить качество финального ответа.

1.4.3 Сравнительный анализ

В результате разбора отобранных GraphRetrieval-методов составлена сравнительная Таблица 1.4 с их характеристиками.

Из Таблицы 1.4 видно, что лидерами по качеству являются методы, которые используют технику итеративного сбора информации: ToG, ToG2.0, DoG. Метод GCR также находится в этом числе лидеров, но нужно иметь в виду, что для него было выполнено предварительное дообучение на заданные датасеты. Можно заметить, что использование LLM-модели в качестве способа фильтрации вершин/связей во время обхода графа даёт прирост в качестве извлекаемой информации и как следствие усиливает релевантность финального ответа. Также можно обратить внимание, что для лидирующих методов извлечённая из графа знаний информация подаётся на вход финальной LLM-модели в виде списка путей обхода графа. Такой формат позволяет учитывать связь между триплетами знаний при генерации ответа на user-вопрос.

При этом у всех рассмотренных методов отсутствует стадия по обработке/чистке исходного user-вопроса перед инициализацией поиска релевантной информации. Вторым общим недостатком методов является статическое сопоставление ключевых сущностей из user-вопроса с вершинами графа в качестве шага инициализации поиска. Не рассматриваются ситуации, когда сущность является обобщённым объектов и ей соответствует несколько вершин, а также когда для сущности нет релевантной вершины. Также стоит обратить внимание на то, что только у трёх из десяти методов в том или ином виде реализован механизм обработки ситуации, когда в графе знаний нет релевантной информации для генерации релевантного ответа на user-вопрос.

1.4.4 Выводы

Основываясь на информации, структурированной в Таблице 1.4, можно выдвинуть следующую гипотезу: комбинация алгоритма итеративного поиска на основе обновляемого плана с алгоритмом динамического сопоставления сущностей из вопроса с вершинами графа знаний и техниками модификации/корректировки исходного user-вопроса, резюмирования/сжатия исходной информации позволит получить прирост метрик эффективности на задачах предметно-ориентированной генерации текста.

Таблица 1.4 — Сравнительная таблица характеристик разобранных GraphRetrieval-методов. Характеристика Performance показывает качество метода по EM-метрике (Hits@1) на датасетах WebQCP, CWP, SimpleQuestions, WebQuestions, QALD-10, GrailQA. Характеристика Train-free показывает: для работы метода не требуется дополнительного дообучения используемых моделей под используемый граф знаний. Характеристика Iterative search показывает, что во время поиска выполняется итеративный сбор информации. Характеристика Search seed показывает объект, на основе которого оценивается релевантность извлекаемых триплетов во время обхода граф знаний. Характеристика Plan enhancement показывает: во время обхода граф на промежуточных шагах выполняет улучшение плана поиска на основе извлечённой информации. Характеристика "No answer"-mechanism показывает: в методе предусмотрен механизм, который позволяет сообщить, что в графе знаний нет информации для генерации правильного ответа на вопрос. Характеристика Entities2Nodes-matching показывает количество вершин, которые сопоставляются ключевым сущностям из user-вопроса для инициализации обхода графа. Характеристика Nodes-filtering показывает способ фильтрации вершин во время обхода графа. Характеристика Relations-filtering показывает способ фильтрации связей, во время обхода графа. Характеристика LLM-knowlege approximation показывает: для инициализации поиска используется информация, которая была сгенерирована только на основе внутренних знаний о мире LLM-модели. Характеристика Graph CRUD-free показывает, что изменение информации в графе требует константного/линейного времени. Характеристика Base question preprocessing показывает: перед началом поиска информации исходный user-вопрос проходит стадию предобработки для удаления лишнего шума и его декомпозиции на независимые части. Характеристика Retrieved triplets postprocessing показывает способ подачи извлечённых триплетов на вход LLM-модели для генерации финального ответа на user-вопрос.

N	Характеристики		GraphRetrieval-метод									
			ToG (ChatGPT / GPT4)	RoG (Llama2:7B)	PMKGE (GPT3.5 / GPT4)	KG-RAG (GPT4)	GRAG (Llama2:7B)	GNN-RAG (Llama2:7B)	ToG2.0 (GPT3.5)	DoG (GPT4 / GPT3.5)	GCR (Llama3.1:8B + ChatGPT)	PDA (GPT3.5)
1	Accuracy	WebQCP	76,2 / 82,6	85,7	-	-	72,75	85,7	81,13	91,0 / 88,6	92,6	73,25
		CWQ	57,1 / 67,6	62,6	-	19	-	66,8	-	56,0 / 58,2	72,7	-
		SimpleQuestions	53,6 / 66,7	-	34,3 / 40,0	-	-	-	-	-	-	-
		WebQuestions	54,5 / 57,9	-	-	-	-	-	-	80,0 / 78,2	-	-
		QALD-10	50,2 / 53,6	-	48,6 / 56,5	-	-	-	54,05	-	-	-
		GrailQA	68,7 / 81,4	-	-	-	-	-	-	80,0 / 77,8	-	-
		HotpotQA	- / 41,0	43,0	- / 42,6	-	36,1	43,0	40,9	45,3	45,9	36,5
2	Train-free (+/-)		+	-	+	-	+	-	-	+	-	+
3	Iterative search (+/-)		+	-	-	+	-	-	+	+	-	+
4	Search seed (base question, subquestion, plan step)		base question	plan step	base question	plan step	base question	base question	sub-question	sub-question	base question	plan step
5	Plan enhancement (+/-)		-	-	-	+	-	-	-	-	-	-
6	"No answers"-mechanism (+/-)		+	-	-	+	-	-	-	+	-	-
7	Entities2Nodes-matching (single, fixed)		single	single	-	fixed	-	single	single	single	-	single
8	Nodes-filtering (LLM, vector sim, stats, mixed)		LLM	-	stats	mixed	vector sim	stats	vector sim	LLM	LLM	stats
9	Relations-filtering (LLM, vector sim, stats, mixed)		LLM	-	LLM	mixed	vector sim	stats	LLM	LLM	LLM	vector sim
10	LLM-knowlege approximation (+/-)		-	+	+	-	-	-	-	-	+	-
11	Graph CRUD-free (+/-)		+	-	+	+	-	-	+	+	-	+
12	Base question preprocessing (+/-)		-	-	-	-	-	-	-	-	-	-
13	Retrieved triplets postprocessing (triplets, paths)		paths	paths	triplets	triplets	triplets (+ graph embds)	triplets	paths	paths	paths	triplets

1.5 Выводы

Можно сформулировать следующие вектора развития GraphRAG-методов:

1. Большинство GraphRAG-методов используют статические базы данных, однако, с течением времени неизбежно появляется новая информация (в виде сущностей и связей), которую необходимо учитывать и интегрировать в систему. Быстрое внесение этих изменений часто является сложной задачей: требуется обновление большей части структуры информационного хранилища. Разработка эффективных алгоритмов динамических обновлений знаний в графе значительно повысит производительность и актуальность GraphRAG-систем.
2. В разобранных статьях авторы предлагают хранить в графах знаний только текстовую информацию, не включая другие модальности, такие как изображения, аудио и видео, которые потенциально могут значительно повысить показатели насыщенности информации и, как следствие, увеличить точность генерируемых ответов.
3. В GraphRAG-методах извлечённая из графа информация (в виде набора триплетов или путей) преобразуется в одну текстовую последовательность для обработки LLM-моделью, что приводит к длинному/зашумлённому LLM-промпту. LLM-модели пока не могут качественно обрабатывать длинные последовательности и обуславливаться на всю информацию при генерации ответа. Для решения этой проблемы решающую роль играют алгоритмы сжатия/резюмирования без потерь: они выполняют фильтрацию избыточной информации и приводят предложения из длинных в более короткие и содержательные. Такие алгоритмы помогут LLM-модели захватить больше полезной информации из набора текстов и ускорить генерацию ответа.

2 Разработка LLM агента, использующего разработанную архитектуру графа знаний и разработанный в алгоритм извлечения информации из графа знаний

2.1 Разработка архитектуры хранения структурированных и слабо-структурированных данных (графа знаний) и алгоритма, реализующего это решение

Memorize-пайплайн отвечает за извлечение знаний из различных источников и их сохранение в виде графа знаний. Архитектура пайплайна модульная и последовательная: входные данные проходят через несколько этапов, каждый из которых реализован отдельным модулем. На рисунке 2.1.1 показана обобщенная схема пайплайна.

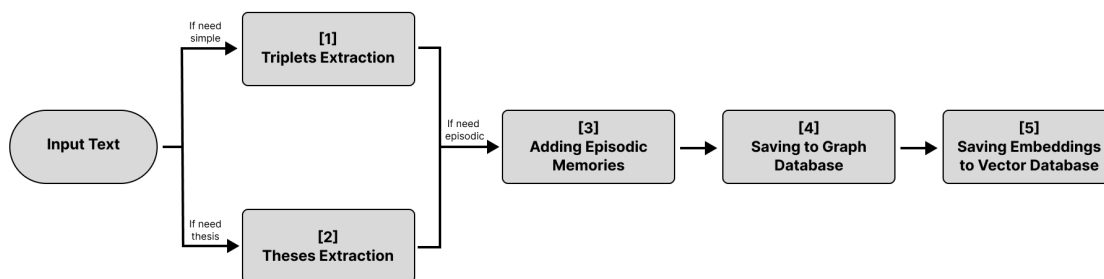


Рисунок 2.1.1 — Высокоуровневая архитектура Memorize-пайплайна

Базовый вариант пайплайна не учитывает время (все знания считаются статическими), модифицированная версия memorize конвейера, принимающая во внимание темпоральную составляющую, описана в соответствующем разделе. Основные этапы обработки следующие:

- Извлечение информации из неструктурированных данных. Первым шагом текстовая информация (например, диалог, документ) преобразуется в структурированные элементы знаний. Для этого используется модуль на базе LLM, который из сырого текста извлекает факты в форме триплетов (субъект, предикат, объект), а также более сложные тезисы и определяет эпизодические воспоминания. На этом этапе данные превращаются в набор структур (триплеты, тезисы, эпизоды), которые впоследствии интегрируются в граф знаний.
- Обработка структурированных данных. Если входные сведения уже представлены в структурированном формате (например, списки фактов или таблицы), они напрямую преобразуются в триплеты по заданным правилам. Такой источник минует этап обработки с LLM и сразу формирует связи для графа.
- Актуализация и сохранение знаний. На финальном этапе извлеченные знания интегрируются в общий граф знаний. Модуль обновления графа проверяет новые

сведения на дубликаты. При необходимости он может удалить устаревшие факты перед добавлением новых (в базовом варианте эта опция отключена). Затем новые узлы и связи добавляются в графовую базу данных, а текстовые представления знаний сохраняются также в векторной базе данных в виде эмбедингов для последующего семантического поиска. Далее подробно рассматриваются принципы работы каждого этапа.

2.1.1 Обработка слабоструктурированных данных

Знания из слабоструктурированных источников извлекаются с помощью LLM, которая выступает тут в роли парсера, превращающего фрагменты текста в формализованные знания. Обработка осуществляется в несколько подэтапов. Сначала происходит извлечение фактов в форме триплетов. LLM получает на вход сырой текст и конкретную инструкцию по выделению триплетов. Пример промпта: "Основная цель - тщательно собрать информацию из входных данных [Текст] и организовать эти данные в четкий, структурированный граф знаний. Граф знаний состоит из набора триплетов... [Извлеченные триплеты] следует возвращать в следующем формате: `subject_1 | relation_1 | object_1; ...`". На выходе LLM генерирует список триплетов, соответствующих фактам, упомянутым в тексте. Например, из отрывка "Корабельщики в ответ: «Мы объехали весь свет; За морем житье не худо; В свете ж вот какое чудо: Остров на море лежит" могут быть получены триплеты: [корабельщики, объехали весь, свет], [житье, не худо за, море], [чудо, есть в, свет]. После извлечения список триплетов подвергается разбору и фильтрации: автоматически проверяется соответствие формату (три элемента в каждом, текст без лишних частей), устраняются возможные дубликаты. В результате остается набор атомарных фактов. Критерием атомарности является неделимость информации – каждый триплет выражает одно отношение и не должен объединять несколько независимых фактов. На следующем шаге LLM выделяет из текста более сложные утверждения - тезисы, представляющие собой полные законченные мысли, которые могут охватывать более двух сущностей. Промпт для модели формулируется так, чтобы получить отдельные самодостаточные утверждения из текста. Модель возвращает список тезисов и перечень упоминаемых в каждом тезисе сущностей. Например, из того же текста LLM может извлечь тезисы: "За морем житье не худо", "В свете чудо – остров на море лежит". Каждый из этих тезисов является отдельным утверждением, в котором участвующие сущности уже были вычленены. Извлеченные тезисы в структуре памяти будут играть роль гиперсвязей – то есть связывать между собой сразу все указанные в них сущности (желтые связи на рисунке 2.1.2). Перед добавлением в граф тезисы фильтруются: удаляются возможные повторные утверждения. Помимо отдельных фактов и тезисов в граф знаний может сохраняться и эпизодическое воспоминание (исходный фрагмент текста). Это позволяет системе при необходимости обращаться к первоначальной формулировке данных. Эпизодическое воспоминание связывается со

всеми сущностями, упомянутыми в данном фрагменте (синие связи на рисунке 2.1.2). Таким образом, эпизодическая память фиксирует контекст появления фактов. На этапе сохранения эпизоды также проверяются на дублирование.

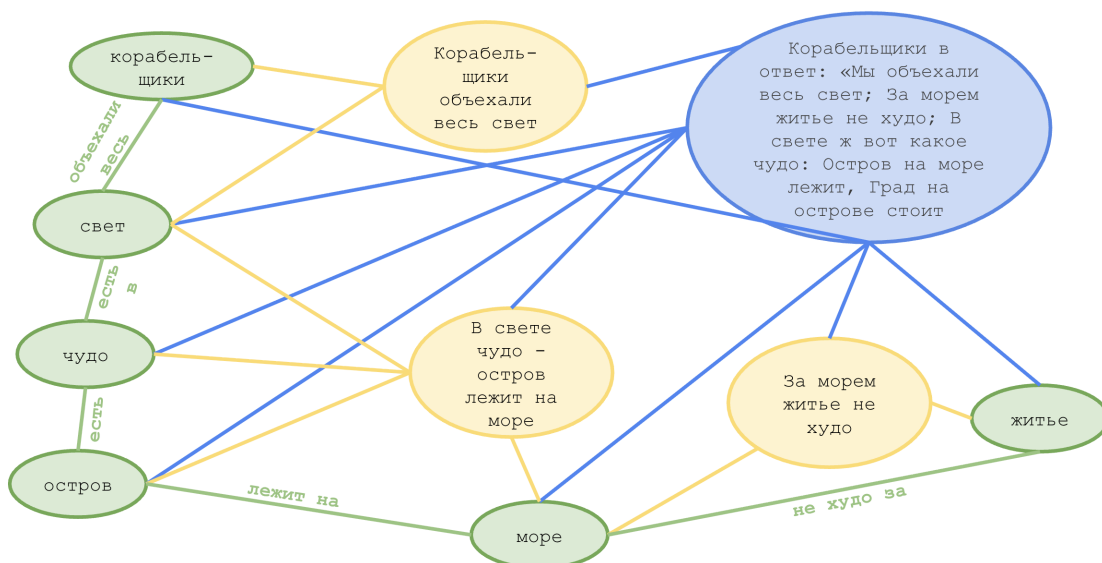


Рисунок 2.1.2 — Пример фрагмента графа знаний

Благодаря использованию LLM, пайплайн способен извлекать знания из неструктурированных данных универсального формата, а последующая фильтрация обеспечивает надежность и единообразие выделенных фактов.

2.1.2 Интеграция структурированных данных в граф

Для структурированных источников разработан алгоритм прямого внесения фактов в граф знаний. Ожидается, что данные поступают в виде списка заданного формата (триплеты или аналогичные им записи). Каждый такой факт сохраняется как простой триплет: для указанных сущностей (субъекта и объекта) создаются или переиспользуются соответствующие объектные узлы, а между ними устанавливается связь с заданным типом отношения. Аналогично случаю с неструктурированным вводом предусмотрены проверки на дубли. Это предотвращает разрастание графа за счет одинаковых знаний из разных источников.

Таким образом, для структурированных данных знания встраиваются напрямую. Это повышает эффективность: пропуская стадию извлечения, пайплайн может быстро интегрировать большие объемы уже готовых фактов.

2.1.3 Представление знаний в графе

После этапов извлечения или преобразования все знания представляются и хранятся в виде графа знаний с несколькими типами вершин и ребер. Объектные узлы (зе-

ленные вершины на рисунке 2.1.2) – это узлы графа, соответствующие атомарным сущностям, упомянутым в данных. Каждый уникальный объект создается как одна вершина; повторные упоминания той же сущности сопоставляются с уже существующим узлом (например, все факты о Москве будут привязаны к одному узлу "Москва"). Простые связи (зеленые ребра) – ребра графа, соединяющие две объектные вершины и отражающие прямое отношение между ними. Каждая такая связь соответствует извлеченному триплету: субъект -> объект. Тезисные узлы (желтые вершины) – узлы-гиперсвязи, представляющие тезисы, то есть сложные утверждения, объединяющие несколько сущностей. Наконец, эпизодические узлы (синие вершины) – узлы-гиперсвязи, представляющие эпизодические воспоминания, то есть исходные фрагменты текста, сохраненные целиком. По аналогии с тезисом, эпизодический узел подключается ребрами ко всем объектным узлам, чьи сущности присутствуют в данном фрагменте.

2.1.4 Актуализация знаний

Поиск устаревшей информации в графе производится следующим образом. В начале множество сущностей, фигурирующих в только что извлеченных триплетах и тезисах сопоставляется с существующими вершинами графа для обнаружения совпадений. Если таковые присутствуют, то формируется список уникальных триплетов, инцидентных этим вершинам. Далее с помощью специального промпта LLM пытается заменить найденные знания только что извлеченными. Если какие-то триплеты или тезисы могут быть заменены, то они удаляются из графа. Благодаря этому правилу, каждое уникальное знание хранится в графе единожды.

2.1.5 Сохранение данных в графовую и векторную базы

После преобразования входных данных в графовые структуры проводится сохранение знаний в базах данных двух типов. Во-первых, в графовую базу данных (neo4j, milvus): программный модуль обращения к графовой БД получает на вход список узлов и связей, подлежащих добавлению, и выполняет серию операций записи. Во-вторых, в векторную базу данных: для каждого нового элемента знания (будь то простой факт или текст тезиса/эпизода) вычисляется эмбединг, который сохраняется во внешнем векторном хранилище. Описанные модули сохранения обеспечивают долговременную память агенту. Графовая БД гарантирует структурированный доступ к фактам и их связям, а векторная БД – смысловой поиск и сопоставление по контексту.

2.1.6 Оценка производительности

В рамках экспериментов по оценке производительности memorize-пайплайна была измерена скорость построения графа знаний при связке графовой БД Neo4j и век-

торной БД Milvus для нескольких датасетов (DiaASQ, HotpotQA, TriviaQA, RuBQ) и разных LLM (Llama3.1-8B, Qwen2.5-7B, DeepSeek V3-7B, GPT-4o mini, GigaChat Max). Для каждого набора данных фиксировалось: количество обрабатываемых контекстов, длина контекста, суммарное время построения графа и производительность в "контекстах в минуту".

При типичной длине фрагмента 550–760 символов средняя скорость добавления в память составила около 0.9–1.8 контекста в минуту, в зависимости от модели и датасета. Наиболее высокую производительность показала конфигурация с GPT-4o mini (в среднем по трем датасетам ~ 1.86 контекста/мин), тогда как DeepSeek V3 продемонстрировала минимальные значения (~ 0.9 контекста/мин). Для построения графа на основе RuBQ использовался только GigaChat Max, чья производительность составила 1.19 контекста в минуту. В целом для построения крупных графов по HotpotQA и RuBQ (3.5–3.9 тысяч контекстов) общее время варьировалось от 69.5 до 113.5 часов, при времени на один контекст порядка 1.2–1.7 минуты. Одновременно были собраны графы с десятками тысяч вершин (70–86 тысяч) и сотнями тысяч ребер (195–305 тысяч), что продемонстрировало масштабируемость memorize-пайплайна на связке Neo4j+Milvus, а также подтвердило, что основным ограничивающим фактором остается стоимость LLM инференса.

Дополнительно были проведены интеграционные тесты, измеряющие скорость коннекторов к графовым и векторным БД (время прохождения фиксированного набора интеграционных тестов при использовании каждой БД). По результатам для dense-векторных коннекторов (используемых в memorize-пайплайне для хранения эмбедингов) наиболее быстрым оказался InMemory вариант, тогда как Milvus показал самое длительное время выполнения: данная БД отработывала приблизительно в 30 раз медленнее чем InMemory и примерно в 2.5-3 раза медленнее чем Elasticsearch, OpenSearch и Weaviate. Для BM25-коннекторов картина аналогичная: InMemory-реализация была быстрее OpenSearch, Elasticsearch, Weaviate примерно в 15–16 раз. Это подчеркивает, что выбор векторного хранилища заметно влияет на накладные расходы всей системы: векторный поиск в рамках Milvus самый медленный, если использовать более быстрые векторные базы данных, то и на построение графа уйдет меньше времени.

В блоке графовых БД различия менее существенны: InMemory-граф отработал быстрее всего, а Neo4j и Kuzu показали время выполнения интеграционных тестов всего лишь примерно в 1.7–2 раза больше. В тестах с "graph model" разброс между InMemory и Neo4j составил порядка 10%, а между InMemory и Kuzu - порядка 60%, что заметно меньше, чем разброс по векторным БД.

Таким образом, среди инфраструктурных компонентов наибольшее влияние оказывает выбор векторной БД, причем Milvus в текущей конфигурации является самым узким местом по скорости. Переход на более быстрые dense/BM25-хранилища непосредственно снижает общее время построения графа. Выбор конкретной графовой БД влияет на производительность заметно меньше, чем выбор векторного хранилища.

2.1.7 Выводы

Memorize-пайплайн задает последовательный механизм преобразования разнородных источников в граф знаний с четкой структурой: знания из слабоструктурированных текстов извлекаются LLM в виде триплетов, тезисов и эпизодических воспоминаний, а данные из структурированных источников напрямую преобразуются в триплеты. Использование 3 типов вершин (объектных, тезисных и эпизодических) и механизмов дедупликации обеспечивает компактное и непротиворечивое хранение знаний, а связка графовой БД и векторной БД позволяет одновременно поддерживать топологический доступ к фактам и семантический поиск по их текстовым представлениям. Эксперименты с Neo4j и Milvus показали, что при типичных длинах контекстов пайплайн устойчиво обрабатывает крупные датасеты, формируя графы с десятками тысяч вершин и сотнями тысяч ребер при скорости порядка 0.9–1.8 контекста в минуту, причем основным ограничивающим фактором выступает стоимость LLM-инференса. Дополнительные интеграционные тесты продемонстрировали, что выбор векторного хранилища критичен для производительности: векторный поиск в рамках Milvus самый медленный, и при использовании более быстрых векторных баз данных суммарное время построения графа может быть существенно сокращено, тогда как влияние выбора конкретной графовой БД заметно слабее. В совокупности это подтверждает, что предложенная архитектура memorize-пайплайна является масштабируемой, гибкой к типу входных данных и пригодной для дальнейшей оптимизации за счет замены инфраструктурных компонентов без изменения логики построения графа.

2.2 Алгоритм извлечения информации из памяти LLM-агента на основе графа знаний для решения QA-задач

Схематично пайплайн без временных меток изображен на рисунке 2.2.1. Видно, что конвейер разбит на тринадцать стадий. На вход конвейеру поступает пользовательский вопрос в строковом формате, который передается в модуль "Question Decomposition". В данном модуле реализуется очистка вопроса от шума (удаление стоп-слов, исправление ошибок), обогащение вопроса (добавление более понятных языковых конструкций, раскрытие слабоопределенных фраз и терминов, перефразирование с соблюдением грамматики и синтаксиса) и декомпозиция сложных вопросов на независимые под-вопросы. Целью такой декомпозиции является не только облегчение поиска релевантной информации, но и сокращении временных затрат благодаря параллельной обработке полученных под-вопросов. Затем следует стадия "Initial Exploration Plan Generation" в которой происходит инициализация плана поиска (при первом запуске конвейера). После выполняется извлечение сущностей из поискового запроса (конкретного шага плана поиска) в модуле "Named Entities Extraction". Далее на стадии "Entities2NodesMatching" извлеченные сущности сопоставляются вершинам гра-

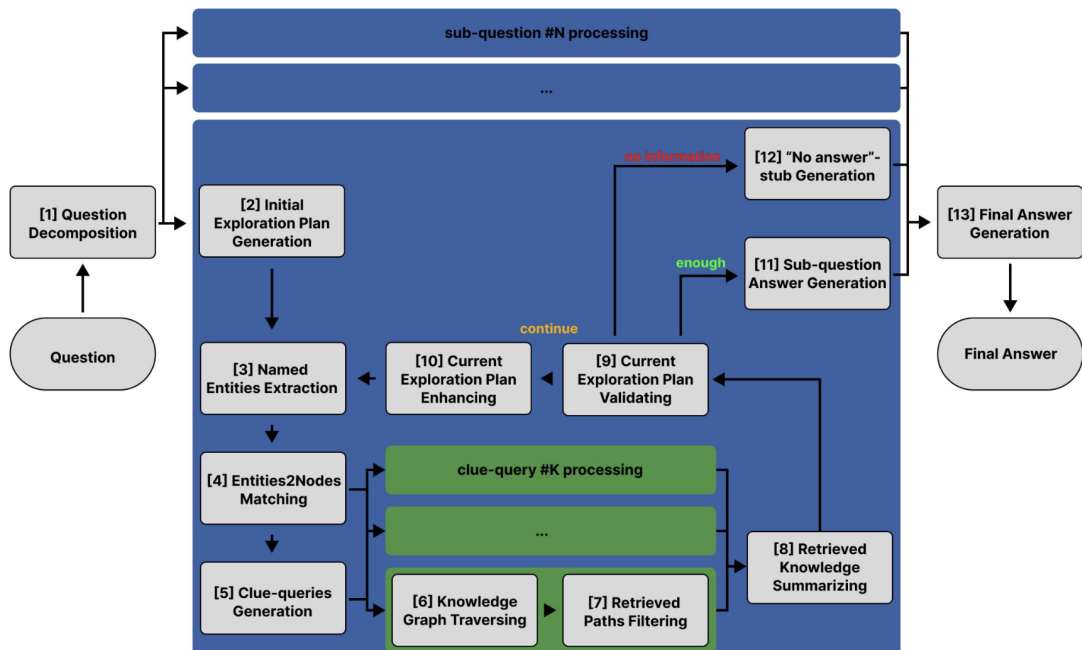


Рисунок 2.2.1 — Высокоуровневая архитектура QA-пайплайна

фа. С учетом полученного соответствия и поискового запроса в модуле "Clue-queries Generation" генерируются вспомогательные вопросы (clue-query). Извлеченной из вопроса сущности может сопоставляться несколько вершин графа, поэтому при генерации clue-queries мы генерируем вспомогательный вопрос для каждой комбинации из декартова произведения релевантных вершин графа (мы ограничиваем это число сверху 4, чтобы не потратить на этой стадии слишком много ресурсов). Затем с каждым вспомогательным под-вопросом мы обходим граф, собирая релевантную информацию. Обход реализуется следующими методами (или комбинацией любых 2-х указанных алгоритмов): А со звездочкой, поиск в ширину, лучевой поиск, наивный поиск триплетов и WaterCircles (использующий поиск в ширину, стратующий с seed вершин - сопоставленных сущностям вопроса узлов графа). Подробнее об этих методах будет рассказано в соответствующем разделе. После обхода графа в модуле "Knowledge Graph Traversing" следует стадия фильтрации "Retrieved Paths Filtering". Она заключается в фильтрации триплетов по типу, а также в их ранжировании на основе релевантности к вспомогательному под-вопросу. Далее в рамках 8 этапа "Retrieved Knowledge Summarizing" для каждого вспомогательного под-вопроса и найденной к нему релевантной информации генерируются clue-answers. Чтобы получить ответ на исходный запрос из шага плана поиска осуществляется стадия суммаризации: подаем в LLM все clue-queries, clue-answers и сам запрос и просим синтезировать ответ. Затем на стадии "Current Exploration Plan Validating" мы проверяем, нужно ли с учетом новой информации (полученного ответа на поисковый запрос) скорректировать план поиска. Если нет, то переходим к следующему шагу текущего плана (если такие еще остались). Если необходимость в изменении плана есть, то следующие шаги регенерируются в рамках "Current Exploration Plan Enhancing". Иными словами, эта стадия реализует динамическое улучшение стратегии

поиска на основе уже найденной информации по предыдущим шагам плана. В итоге для следующего шага (перегенерированного или нет) мы снова начинаем с 3 стадии. После каждого полученного ответа на шаг плана мы проверяем, достаточно ли найденной информации для ответа на независимый под-вопрос. Если нет и мы не исчерпали лимит по шагам - тогда идем двигаемся по плану дальше (возможно скорректированному). Если информации достаточно - то мы генерируем ответ. За это решение отвечает отдельный модуль AnswerClassifierAgent. Генерация ответа происходит на стадии "Sub-question Answer Generation". Таким образом мы получаем ответы на под-вопросы, на которые был декомпозирован исходный пользовательский вопрос. Чтобы добиться ответа на него, остается суммаризировать их. Это реализуется на последней стадии "Final Answer Generation". Мы подаем LLM исходный вопрос, под-вопросы с ответами на них и просим сгенерировать финальный ответ. Важным нюансом является тот факт, что мы предусматриваем сценарий NoAnswer события. Если LLM понимает, что предоставленной ей информации недостаточно, чтобы сформировать надежный ответ, то она генерирует `<|NotEnoughInfo|>` и не будет пытаться выдумывать.

Ключевые этапы описанного конвейера следующие:

- механизма декомпозиции пользовательского вопроса;
- механизм планирования
- механизм динамического сопоставления сущностей с вершинами графа

Они и формируют мотивацию QA-пайплайна. Современные методы как правило не включают стадию анализа исходного пользовательского вопроса, что лишает их возможности обрабатывать составные вопросы, требующие итеративной (multi-hop) обработки. Также зачастую имеющиеся подходы выполняют статическое сопоставление ключевых сущностей из пользовательского вопроса с вершинами графа в качестве шага инициализации поиска. Это сильно ограничивает дальнейший поиск, поскольку как минимум здесь не учитывается наличие "обобщенных" сущностей в вопросе.

Благодаря реализованной в нашем конвейере декомпозиции представляется возможным обрабатывать независимые части исходного вопроса параллельно, тем самым сокращая временные затраты на получение финального ответа. При помощи механизма планирования мы можем динамически корректировать стратегию поиска на основе имеющейся информации в графе знаний, тем самым повышая точность генерируемого ответа. Наконец, механизм динамического сопоставления сущностей с вершинами графа позволяет (на основе древовидного способа хранения вершин) корректно обрабатывать сущности различной гранулярности, за счет чего алгоритмы поиска смогут собрать/подготовить больше релевантной информации и, тем самым будет получен более полный финальный ответ.

Далее в подразделах представлено более подробное описание модулей, используемых в рамках данного конвейера.

2.2.1 Обработка пользовательского вопроса

2.2.1.1 Очистка вопроса

На данной стадии выполняется запрос к LLM для удаления стоп-слов и другой избыточной информации из вопроса. Далее следует запрос, требующий исправить грамматические ошибки в пользовательском вопросе.

2.2.1.2 Обогащение вопроса

Текущий этап подразумевает обогащение вопроса более конкретной и понятной информацией. В рамках этой стадии происходит три запроса к LLM. В первом обращении добавляются понятные языковые конструкции и важные детали. При втором обращении размытые и редко встречающиеся фразы заменяются на более понятные и формальные конструкции. Например, вопрос "как оформить ип на упрощенке?" преобразуется в "Как оформить статус индивидуального предпринимателя на упрощённой системе налогообложения (УСН)?". Наконец, третий запрос исправляет грамматические и синтаксические ошибки (если они возникли на предшествующих этапах).

2.2.1.3 Декомпозиция вопроса

Очевидно, что не каждый вопрос требует декомпозиции. Если попытаться разбить простой вопрос на независимые под-вопросы, то мы рискуем получить шум и галлюцинации. Поэтому перед декомпозицией осуществляется этап классификации на предмет того, требуется ли вопросу декомпозиция. Мы подаем LLM вопрос и просим вернуть только "True" (если вопрос можно разбить на простые независимые под-вопросы) "False" (если вопрос не требует декомпозиции). Далее если LLM дает положительный ответ, мы в отдельном обращении просим ее разбить вопрос на под-запросы. Например, мы ожидаем, что вопрос "Кто заработал больше: Microsoft или Google — в прошлом году?" превратится в серию под-вопросов: "Какую прибыль Microsoft получила в прошлом году?", "Какую прибыль Google получила в прошлом году?", "Чья прибыль больше: Microsoft или Google?".

2.2.2 План поиска

План поиска включает в себя несколько подэтапов. Разберем их подробнее.

2.2.2.1 Инициализация плана

По под-вопросу, полученному после декомпозиции, мы просим LLM составить план поиска информации. Подразумевается, что этот план должен состоять из списка поисковых запросов, которые потенциально могут привести к необходимым знаниям.

2.2.2.2 Корректировка плана

После инициализации плана мы начинаем работать с его первым шагом (первым поисковым запросом). После того, как мы находим информацию на этот поисковый запрос и генерируем на ее основе ответ, мы проверяем, достаточно ли этого, чтобы удовлетворить исходный под-вопрос (на основе которого и был сгенерирован план). Если информации достаточно - мы прекращаем итерироваться по плану и переходим к следующей стадии (генерация ответа на под-вопрос). Если информации недостаточно, то возможно прежде чем переходить к следующему шагу плана, сам план стоит скорректировать. Действительно, мы нашли новые важные данные, которые могут повлиять на выбранную стратегию поиска. Поэтому после того, как мы получили эти новые данные, мы обращаемся к LLM, снова решая задачу классификации (подобно тому, как это было при декомпозировании вопроса). Учитывая вопрос, связанный с ним план поиска и найденную на данный момент информацию, LLM опять нужно ответить "True" (требуется корректировки дальнейших шагов плана) "False" (корректировка не требуется). В случае положительного ответа осуществляется этап регенерации последующих пунктов плана поиска. На основе тех же данных, что мы подавали LLM в задаче классификации, мы просим ее скорректировать план. После этого мы переходим к следующему актуализированному шагу поиска. В итоге реализуется динамическая корректировка стратегии плана поиска.

2.2.3 Сопоставления сущностей из вопроса и вершин графа

Сначала сущности извлекаются из вопроса с помощью LLM. Мы подаем на вход запрос и просим извлечь сущности в формате <сущность#1> | <сущность#2> | ... | <сущность#N>. Так, например, из вопроса "Какое устройство дольше работает от аккумулятора: iPhone11 Pro Max или Xiaomi 11?" мы получаем: "время работы аккумулятора | iPhone11 Pro Max | Xiaomi 11". На следующем этапе мы сопоставляем этим сущностям вершины графа. Для реализации этой операции предусмотрено 2 опции. Первая заключается в использовании древовидной структуры. Вершины исходного графа преобразуются в дерево, где листья - это сами вершины, а узлы - их суммаризация. Используя указанную структуру, мы сопоставляем сущностям вершины графа. Вторая опция представляет собой векторный поиск по эмбедингам вершин графа. Максимальное число вершин, соответствующих сущности - настраиваемый гиперпараметр, который мы по умолчанию полагаем равным 3.

2.2.4 Генерация вспомогательных запросов

После предыдущей стадии мы имеем соответствие между сущностями вопроса и вершинами графа знаний. Используя эту информацию, мы можем регенерировать

под-вопрос так, чтобы он стал более конкретным и лучше подходил под наши данные. Например, пусть мы имеем вопрос "Какие отделения банка работают в городе в воскресенье?". Предположим, что для сущностей нашли такие сопоставления: банк - "ПАО Сбербанк", город - Москва, воскресенье - воскресенье. Тогда мы можем получить уточняющий вспомогательный запрос: "Работают ли отделения Сбербанка в Москве в воскресенье?". Поскольку, как было замечено ранее, каждой сущности может соответствовать сразу несколько вершин графа, мы берем декартово произведение всех сопоставленных вершин. Для каждой комбинации из этого произведения и исходного вопроса мы обращаемся к LLM, чтобы сгенерировать вспомогательный под-запрос. Максимальное число `clue-queries` - гиперпараметр, который по умолчанию равен 4 (то есть мы берем не более 4-х комбинаций из декартова произведения). Далее мы работаем с полученными вспомогательными запросами.

2.2.5 Обход графа

Для полученного вспомогательного запроса нам необходимо найти релевантные триплеты в графе знаний. Чтобы осуществить это используются разные алгоритмы обхода графа. В качестве модели для получения векторных представлений текста используется E5-small из HuggingFace-репозитория.

2.2.5.1 WaterCircles

Идея метода заключается в том, что мы стартуем "кольцами" от вершин графа, которые были сопоставлены сущностям запроса, и находим пересекающиеся пути. Далее преобразуем их в набор триплетов и отбираем самые часто встречающиеся цепочки. Данный алгоритм подходит для задач, в которых важны структурные связи между несколькими узлами графа, и нужно найти связи между фактами, опираясь на пересечения путей.

Ключевые гиперпараметры, предусмотренные в конфигурационном классе `WaterCirclesSearchConfig`:

1. `strict_filter` - если `True`, то отдаем предпочтение подтвержденным пересечениям цепочкам; уменьшаем шум, но может потерять редкие факты.
2. `hyper_num`, `episodic_num`, `chain_triplets_num`, `other_triplets_num` - ограничивают размер каждого "кольца" и конечного пула триплетов (сколько гипер, эпизодических и других триплетов брать). По сути отвечают за баланс между полнотой поиска и шумом/временем работы.
3. `do_text_pruning` - опциональная обрезка текстового шума при извлечении тезисов.
4. `accepted_node_types` - отвечает за то, какие типы вершин можно обходить. По умолчанию: `object / hyper / episodic / time`.

2.2.5.2 A со звездочкой

В качестве одного из алгоритмов обхода графа знаний для извлечения релевантных триплетов был реализован A*-алгоритм. В рамках алгоритма выполняется поиск кратчайших путей между всеми парами вершин, сопоставленных сущностям. В результате такого обхода все триплеты, попавшие в кратчайшие пути сохраняются в единый список с удалением дубликатов (по их содержанию). При использовании данного алгоритма выдвигается следующая гипотеза: информация, которая требуется для генерации правильного ответа содержится в триплетах из кратчайших путей между сущностями исходного вопроса. Во время обхода граф считается невзвешенным и неориентированным. В качестве d-метрики расстояния между смежными вершинами используется константное значение. В качестве эвристической h-метрики расстояния между текущей и конечной вершиной было реализовано три варианта:

1. "ip" - dot-product расстояние эмбедингов между текущей и конечной вершинами.
2. "weight with short path" - длина кратчайшего пути, полученного с помощью bfs-алгоритма, умноженная на "ip"-метрику.
3. "avg weighted with short path" - усредненное значение ip-расстояний эмбедингов смежных вершин в пути от начальной до текущей и ip-расстояния между текущей и конечной вершинами, умноженное на длину кратчайшего пути, полученного с помощью bfs-алгоритма, между текущей и конечной вершинами.

Интуиция за такими эвристическими метриками следующая. "ip" дает семантическую близость по эмбедингам (полезно когда семантика важнее топологии). "weight with short path" и "avg weighted with short path" напротив, отдают предпочтение топологической близости, что снижает риск концентрации только на семантически близких, но топологически удаленных ветках.

2.2.5.3 Поиск в ширину

Классический обход в ширину от seed вершин до заданной глубины и ширины с общими лимитами на количество пройденных узлов. Метод хорош как быстрый baseline и подходит для задач, когда релевантная информация лежит достаточно близко к вершинам графа, сопоставленным сущностям запроса.

Ключевые гиперпараметры, предусмотренные в конфигурационном классе NaiveBFSGraphSearchConfig:

1. max_depth - сколько шагов от seed вершины допустимо сделать.
2. max_width - какая ширина разрешена на каждом уровне, срезает ветвистость поиска.
3. max_passed_nodes - глобальный предохранитель на общее число посещенных вершин.
4. accepted_node_types - отвечает за то, какие типы вершин можно обходить. По умолчанию: object / hyper / episodic / time|

2.2.5.4 Лучевой поиск

Идея метода заключается в построении нескольких лучших путей ("лучей") от узлов графа, сопоставленных сущностям запроса. На каждом шаге остаются только топ-ветви по скору, затем финально отбирается набор путей по одной из стратегий. В компоненте `BeamSearchTripletsRetriever` в процессе итераций накапливаются два множества путей: `ended_paths` - завершенные пути и `continuous_paths` - пути, которые все еще продолжались на этапе остановки. Для каждого пути вычисляется итоговый скор, по которому эти пути сортируются на финальном этапе. Всего предусмотрено 3 режима сортировки (`ended_first`, `continuous_first`, `mixed`): при `ended_first` берется топ-N элементов из `ended_paths` (по убывания скору), а затем, если нужно, добавляются пути из `continuous_paths`; при `continuous_first` - наоборот; при `mixed` пути из двух множеств объединяются и сортируются одним списком. В итоге данный алгоритм дает баланс между полнотой и точностью. Он подходит для задач с широким графом знаний, в которых требуется балансировать извлечение разных маршрутов.

Ключевые гиперпараметры, предусмотренные в конфигурационном классе `GraphBeamSearchConfig`:

1. `max_depth` - сколько глубоко можем уйти seed вершины.
2. `max_paths` - максимальное количество построенных путей.
3. `mean_alpha` - отвечает за учет длины построенного пути при усреднении его ценности (релевантности).
4. `same_path_intersection_by_node`, `diff_paths_intersection_by_node`, `diff_paths_intersection_by_rel` - флаги пересечений, уменьшающие повторяющиеся пути.
5. `final_sorting_mode` - способ финальной фильтрации полученного набора путей (`ended_first`, `continuous_first`, `mixed`).
6. `accepted_node_types` - отвечает за то, какие типы вершин можно обходить. По умолчанию: `object` / `hyper` / `episodic`.

2.2.5.5 Наивный поиск

Данный алгоритм не требует обхода графа. Он заключается в поиске семантически близких триплетов к тексту запроса (`retrieve` + `rerank` операции). Метод подходит, когда важны текстовые формулировки/подсказки из запроса и необходимо за короткое время собрать релевантные факты без прохождения по структуре графа. Наивный поиск также полезно использовать как дополнение к структурным методам, описанным выше.

Ключевые гиперпараметры, предусмотренные в конфигурационном классе `NaiveGraphSearchConfig`:

1. `max_k` - максимальное количество триплетов, которое может быть извлечено из графа.

Кроме того, предусмотрен специальный класс `MixtureTripletsRetriever`, позволяющий использовать комбинацию любых 2-х методов, описанных выше. Благодаря ему при обходе графа представляется возможным опираться как на его структуру (топологию), так и учитывать семантическую близость фактов.

2.2.6 Фильтрация триплетов

После обхода графа мы получили множество релевантных вспомогательному запросу триплетов. Следующим этапом является фильтрация и ранжирование этих триплетов. Цель этой стадии - сократить множество кандидатов до топ-К по релевантности с учетом типа связи и повторов, прежде чем передать их в последующие стадии QA-пайплайна. Ключевые конфигурационные параметры:

1. `accepted_triplets_types` - перечень типов связей, которые допускаются к дальнейшей обработке (по умолчанию — `hyper, simple`). Используется для первичной фильтрации по типу.
2. `max_k` - верхняя граница числа возвращаемых триплетов после ранжирования (по умолчанию 50).

Данный этап можно разделить на следующие шаги:

1. Удаление дублей - убираем одинаковые связи по идентификатору отношения `relation.id`.
2. Типовая фильтрация - отбрасываем триплеты, чьи типы отношений не входят в `accepted_triplets_types`.
3. Переранжирование и усечение. Если после предыдущего этапа осталось менее `max_k` элементов, то возвращаем весь набор. Иначе берем `relation.id` оставшихся триплетов и ранжируем их. Результат - упорядоченный список релевантных `relation.id`, по которым восстанавливаются соответствующие триплеты.

2.2.7 Получение ответа на под-вопрос

2.2.7.1 Генерация ответов на вспомогательные запросы

Наш вопрос из исходного плана поиска преобразовался в несколько вспомогательных запросов. Для каждого вспомогательного вопроса мы нашли релевантную информацию в имеющемся графе знаний. Теперь требуется получить ответ, обусловленный на эту информацию. Для этого мы обращаемся к LLM, подавая вспомогательный запрос и собранную информацию, и просим, опираясь только на эти данные, сгенерировать ответ на вопрос.

2.2.7.2 Суммаризция ответов на вспомогательные запросы

Чтобы получить ответ на исходный запрос из шага плана поиска нам необходимо суммаризировать результаты для clue-queries. Для этого мы подаем в LLM все clue-queries, clue-answers, сам поисковой запрос и просим синтезировать ответ. Чтобы избежать галлюцинаций, мы также просим LLM опираться только на предоставленную ей информацию. Если же релевантной информации недостаточно для уверенного ответа, мы требуем вернуть `<|NotEnoughInfo|>`. Таким образом мы завершаем процесс получения ответа для конкретного шага плана. Как упоминалось в разделе 2.2.2.2 после каждого такого шага мы проверяем, достаточно ли теперь информации, чтобы дать ответ на под-вопрос, на который был декомпозирован пользовательский вопрос. Разберем эту стадию подробнее.

2.2.7.3 Генерация ответа на под-вопрос

В модуле AnswerClassifierAgent мы решаем задачу классификации с помощью LLM подобно тому, как это описывалось в разделах 2.2.1.3 и 2.2.2.2. По поисковым запросам из плана, найденной по ним информации, и самому под-вопросу мы просим LLM ответить либо только "True" (данных достаточно чтобы дать надежный ответ), либо только "False" (данных не хватает и нужно двигаться по плану дальше). В случае положительного ответа мы переходим к следующей стадии, которая и заключается в генерации ответа на под-вопрос. Мы подаем в LLM ту же информацию, что и при задаче классификации, и требуем синтезировать ответ.

В итоге для каждого под-вопроса мы получаем ответ. При этом в качестве ответа может также выступать метка `<|NotEnoughInfo|>` - если в нашем графе знаний недостаточно информации для уверенного ответа. В этом случае и на финальном этапе (при попытке ответить на исходный пользовательский вопрос) мы пойдем по NoAnswer сценарию.

2.2.8 Получение ответа на исходный пользовательский вопрос

Чтобы ответить на исходный пользовательский вопрос нам остается только суммаризировать ответы на декомпозированные под-вопросы. Для этого мы подаем LLM сам вопрос, под-вопросы с ответами на них и просим сгенерировать финальный ответ. Пусть, например, пользователь спросил "Каковы требования и стоимость годового обслуживания для пакета 'Премиум'?", и мы разложили этот вопрос на "Какова стоимость годового обслуживания пакета 'Премиум'?" и "Какие документы требуются для оформления пакета 'Премиум'?". Предположим, что для этих независимых под-вопросов мы получили соответствующие данные: "Стоимость годового обслуживания - 4 990" и "Требуются паспорт и ИНН. Дополнительные документы не запрашиваются". Тогда в качестве финального ответа мы ожидаем получить "Стоимость пакета 'Преми-

ум' в 2024 году - 4 990; для оформления нужны паспорт и ИНН".

2.2.9 Выводы

Разработанный QA-пайплайн задает полнофункциональный алгоритм извлечения информации из графа знаний и генерации ответа на пользовательский вопрос, который объединяет декомпозицию этого вопроса, поэтапное планирование поиска с возможностью корректировки стратегии и динамическое сопоставление сущностей с вершинами графа. Предварительная очистка и обогащение вопроса, классификация на предмет необходимости декомпозиции и построение плана поиска позволяют адаптировать процесс под сложные составные запросы и обрабатывать независимые под-вопросы параллельно, сокращая время получения ответа. Механизм динамического соответствия сущностей и узлов графа (через древовидное представление и векторный поиск) обеспечивает корректную работу с сущностями разной гранулярности и расширяет покрытие релевантных областей графа.

Использование нескольких специализированных алгоритмов обхода (WaterCircles, A*, поиск в ширину, лучевой и наивный поиск), а также их комбинаций, дает возможность гибко балансировать между учетом топологии графа и семантической близостью фактов. Последовательные этапы фильтрации и ранжирования триплетов снижают шум и формируют компактное и при этом информативное множество кандидатов для дальнейшей генерации ответов. Многоступенчатая схема ответов (clue-answers, суммаризация по шагам плана, классификация достаточности знаний и финальная агрегация по под-вопросам) позволяет адаптивно останавливаться при накоплении достаточного объема информации, а предусмотренный сценарий $\langle |NotEnoughInfo| \rangle$ уменьшает риск галлюцинаций. Таким образом, благодаря описанным методам мы получаем QA-пайплайн, способный давать корректные ответы на сложные multi-hop вопросы.

2.3 Количественные оценки эффективности реализованной Retrieve/Rerank-операции

2.3.1 Постановка исследования

В данном разделе описываются различные схемы и эксперименты поиска и переранжирования информации (retrieve/rerank) для задач вопросно-ответного поиска. Целью проведенных экспериментов было определить оптимальные комбинации методов извлечения (retrieval) и ранжирования (reranking) документов по вопросу, измеряя качество топ-результатов метриками Precision@k, Recall@k, F1@k и MAP@k. Рассматриваются три подхода:

- SingleStep (одностадийный) – использование одного алгоритма поиска (например, классический BM25 или нейросетевая модель (эмбеддер)).
- MultiStep (многостадийный) – каскад из двух или трёх этапов: сначала "быстрый" retriever извлекает кандидатов, затем более мощная модель (или несколько последовательно) переупорядочивает найденные документы.
- Ensemble (ансамбль) – объединение результатов нескольких независимых моделей с помощью взвешенного RRF.

Таким образом, формально исследуется задача максимизации релевантности топ- k документов при заданном вопросе и корпусе документов за счёт оптимального сочетания алгоритмов. Практическая важность состоит в повышении полноты и точности извлечения поддерживающих фактов для ответов на сложные вопросы, включая многошаговые (multi-hop) вопросы, при приемлемых затратах времени.

2.3.2 Алгоритмические формулировки задач

2.3.2.1 Одностадийный поиск (SingleStep)

Пусть дан вопрос q и коллекция документов $D = \{d_i\}$. Алгоритм поиска определяет некоторую функцию ранжирования $f(q, d)$, вычисляющую сходство или релевантность документа d вопросу q . В случае BM25 функция f основана на частотах и IDF терминов, в случае эмбединг-модели (например, E5-large или FRIDA) f может быть косинусным сходством между вектором вопроса и вектором документа. Результатом является упорядоченный список S_k из k документов с наибольшим значением f . Задача одношагового поиска – максимально повысить долю релевантных документов в S_k , т.е. оптимизировать Precision@ k , Recall@ k и др., используя ту или иную модель ранжирования.

2.3.2.2 Многостадийный поиск (MultiStep)

Здесь ранжирование происходит в несколько этапов: сначала более "быстрый" retriever получает большой корпус документов и возвращает широкий набор кандидатов, затем один или два этапа усложнённых моделей их переранжируют или отфильтровывают. Формально, первый retriever R_1 возвращает список кандидатов C_m размером m (обычно $m > k$, чтобы увеличить полноту). Далее второй алгоритм R_2 пересчитывает оценки только для документов из C_m . При необходимости третий этап R_3 может аналогично обработать еще меньший набор кандидатов, полученный на предыдущем шаге. Задача каскада – комбинировать преимущества моделей: первый этап обеспечивает высокий recall (охват релевантных), последующие повышают precision за счёт более точного понимания запроса. Например, в экспериментах реализованы схемы: BM25 $\rightarrow$ E5-large, BM25 $\rightarrow$ E5-large $\rightarrow$ FRIDA и др., где стрелка обозначает передачу топ- m результатов на вход следующей модели.

2.3.2.3 Ансамблевый поиск (Ensemble)

Ансамбль из m отдельных ранжировщиков R_i комбинирует их списки результатов без последовательной фильтрации. Каждый алгоритм R_i возвращает свой ранг-лист документов с рангом $r_i(d)$ для документа d . Слияние на основе Reciprocal Rank Fusion вычисляет совокупный балл документа как сумму обратных рангов с весовыми коэффициентами: $score(d) = \sum_{i=1}^m w_i \frac{1}{C+r_i(d)}$. Здесь C – константа сглаживания, а w_i – вес, отражающий доверие к i -му ранжировщику. Более высокий ранг (меньшее r_i) даёт больший вклад через обратную величину. Веса нормируются (например, $w_1 + w_2 = 1$ для двух систем) и позволяют задать приоритет более сильной модели. Например, если при объединении dense-поиска и BM25 присвоить вес 0.7 первому и 0.3 второму, итоговый ранговый балл wRRF документов вычисляется как 70% от их reciprocal rank в векторном поиске плюс 30% от reciprocal rank в sparse-поиске. В итоге ансамблевый подход возвращает топ- k документов с наибольшими $score(d)$. Задача ансамбля – улучшить покрытие и точность за счёт объединения разнородных источников: как правило, комбинация лексического и семантического поиска позволяет находить документы, пропущенные одним из методов, и сглаживать их индивидуальные ошибки.

2.3.3 Источники данных

Для экспериментов выбраны два бенчмарка вопросно-ответного поиска: HotpotQA и RuBQ. HotpotQA – широко известный датасет английских вопросов, требующих мультимодального (многошагового) поиска по Википедии. Он содержит около 113 тысяч пар вопрос-ответ. Каждый вопрос HotpotQA обычно требует найти два связанных факта на разных страницах, что проверяет способность методов извлекать связанные документы и объединять информацию. RuBQ – датасет, состоящий из примерно 1500 вопросов на русском языке различной сложности и ответов на них. RuBQ также используется в формате задачи поиска по текстовым документам: для каждого вопроса сформирован набор релевантных текстовых фрагментов, из которых минимум один содержит ответ. Данный датасет проверяет методы на русском языке, в том числе их умение работать с морфологией и специфическими именами собственными.

2.3.4 Конфигурации экспериментов

В экспериментах применяются следующие модели извлечения и ранжирования:

- BM25 – стандартный лексический поисковый алгоритм, основанный на частотах и IDF терминов. Реализован с помощью обратного индекса и отличается высокой скоростью работы.
- E5-large (из HuggingFace-репозитория) – трансформерная модель, обученная в контрастивной манере на больших массивах пар текстов для универсального семанти-

ческого поиска. Модель E5-large является мультязычной и показывает высокую точность семантического сопоставления на многих языках. В наших опытах E5-large используется для задачи Dense Retrieval: вопросы и документы отображаются в векторное пространство, где их близость измеряется косинусным сходством.

- FRIDA (из HuggingFace-репозитория) - трансформерная модель, основанная на энкодере T5 и обученная как мультязычный текстовый эмбеддер. В наших экспериментах FRIDA применяется аналогично E5-large – для семантического поиска по векторам.

Параметры экспериментальных схем. Для всех методов сравнение проводилось при фиксированном размере возвращаемого списка: $k = 3$ и $k = 5$. То есть оцениваются результаты топ-3 и топ-5 выдачи. Ансамблевые конфигурации объединяли результаты топ-20 от каждого участника с помощью RRF (константа $C = 60$). Веса w_i для RRF подбирались эвристически: так, для ансамбля E5-large+BM25 более высокая важность отдана E5-large (0.6 vs 0.4) в силу его большей точности, тогда как для E5-large+FRIDA веса были равными (0.5 vs 0.5), а для FRIDA+BM25 – наоборот, больший вес у FRIDA (0.6 vs 0.4). В тройном ансамбле E5-large+FRIDA+BM25 все три источника получили близкие веса (0.35, 0.35 и 0.3 соответственно для E5-large, FRIDA, BM25) – это отражает ожидание, что E5-large и FRIDA несут схожую семантическую информацию, а BM25 дополняет их лексическими совпадениями.

В каскадных схемах количество отбираемых кандидатов на каждом этапе контролировалось так, чтобы сохранить высокий recall. В двухступенчатых пайплайнах первый retriever извлекал $m = 25\text{--}50$ кандидатов, которые затем полностью переранжировывались второй моделью с выводом топ-k. Например, конфигурация BM25 -> E5-large означает: BM25 выдаёт 50 кандидатов, затем E5-large пересортировывает их и возвращает топ-k. В схеме BM25 -> E5-large -> FRIDA сначала BM25 выбирает 50 документов, E5-large сокращает до 25 лучших, и затем FRIDA ранжирует их, выдавая финальный топ-k.

Временные характеристики. В каждом подходе зафиксировано среднее время обработки одного вопроса. Одностадийный dense-поиск показал наименьшее время: так, E5-large обрабатывал запрос в среднем за 0.04 с (на HotpotQA), а FRIDA – 0.06 с. Для сравнения, поиск BM25 занял 0.09 с на запрос (несколько медленнее, вероятно из-за менее оптимизированного текстового индекса). Ансамбли подразумевают выполнения нескольких поисков: так комбинации E5-large+BM25 суммарно требует 0.15 с на запрос. Интересно, что комбинация E5-large+FRIDA оказалась даже быстрее (0.11 с). Каскадные пайплайны увеличивают время ещё сильнее: двухэтапный BM25 -> E5-large потребовал 0.30 с на запрос (в 2–3 раза дольше, чем каждый по отдельности), а трёхэтапный BM25 -> E5-large -> FRIDA – около 0.5 с (почти в 10 раз медленнее одиночной E5-large). Таким образом, между подходами имеется компромисс скорость-качество (по ключевым метрикам): более сложные схемы потенциально дают выигрыш в метриках, но за счёт возросшего времени обработки.

2.3.5 Результаты экспериментов

Мы получили подробные метрики для каждого подхода на двух датасетах. Ниже приведено сравнение основных конфигураций по метрикам Precision@k, Recall@k, F1@k и MAP@k (усреднённых по вопросам) при $k = 3$ и $k = 5$.

Таблица 2.1 — Результаты на HotpotQA (P@3, P@5, R@3, R@5, F1@3, F1@5, MAP@3, MAP@5)

Подход	P@3	P@5	R@3	R@5	F1@3	F1@5	MAP@3	MAP@5
Ensemble								
E5-large + BM25	<u>0.6110</u>	<u>0.3762</u>	<u>0.9165</u>	<u>0.9405</u>	<u>0.7332</u>	<u>0.5374</u>	<u>0.90296</u>	<u>0.91415</u>
E5-large + FRIDA	0.5173	0.3432	0.7760	0.8580	0.6208	0.4903	0.73067	0.76509
E5-large + FRIDA + BM25	0.5682	0.3666	0.8523	0.9165	0.6818	0.5237	0.81458	0.84342
MultiStep								
BM25 → E5-large	0.6103	0.3738	0.9155	0.9345	0.7324	0.5340	0.90092	0.90984
BM25 → E5-large → FRIDA	0.2415	0.1513	0.3623	0.3783	0.2898	0.2161	0.34396	0.35023
BM25 → FRIDA	0.2395	0.1499	0.3593	0.3748	0.2874	0.2141	0.34133	0.34730
BM25 → FRIDA → E5-large	0.2637	0.1595	0.3955	0.3988	0.3164	0.2279	0.39267	0.39412
E5-large → FRIDA	0.2420	0.1509	0.3630	0.3773	0.2904	0.2156	0.34412	0.34981
FRIDA → E5-large	0.2498	0.1504	0.3748	0.3760	0.2998	0.2149	0.37238	0.37290
SingleStep								
BM25	0.5758	0.3584	0.8638	0.8960	0.6910	0.5120	0.84738	0.86188
E5-large	0.6050	0.3703	0.9075	0.9258	0.7260	0.5290	0.89283	0.90131
FRIDA	0.2283	0.1436	0.3425	0.3590	0.2740	0.2051	0.32479	0.33119

Как видно из таблицы, на HotpotQA наивысших показателей достигают комбинации, включающие модель E5-large. Лучшие значения Precision@3 = 0.611 (61.1%) и Recall@5 = 0.9405 (94.1%) получены у ансамбля E5-large+BM25. Одностадийный поиск E5-large также показывает высокие результаты (P@3 = 60.5%, R@3 = 90.75%), лишь незначительно уступая ансамблю. Для многошаговых каскадов наиболее успешна конфигурация BM25 -> E5-large, достигшая P@3 = 61.03% и R@3 = 91.55% – практически на уровне лучшего ансамбля. Примечательно, что добавление третьего шага (BM25 -> E5-large -> FRIDA) существенно ухудшает метрики (precision падает до 24%). В целом, лучшим подходом для HotpotQA можно считать либо ансамбль E5-large+BM25, либо двухступенчатый каскад BM25 -> E5-large – они обеспечивают 73% F1@3 и 91%+ MAP@5, что заметно превосходит остальные. Модель FRIDA в одиночку продемонстрировала значительно более низкий результат (P@3 всего 22.8%), что говорит о том, что для английского multi-hop домена её эмбединги недостаточно эффективны. В то же время, BM25 как одиночный метод показывает неожиданно конкурентоспособную полноту (Recall@5 = 89.6%, лишь немного уступая E5-large). Это обусловлено природой HotpotQA: ключевые сущности вопроса зачастую явно присутствуют в тексте релевантных абзацев. Однако BM25 хуже ранжирует и часто помещает отвлекающие документы выше нужных, что выражается в более низкой точности (Precision@3 57.6% против

60.5% у E5-large). Модель E5-large лучше схватывает семантическую связь между разными формулировками вопроса и текста, поэтому выдаёт более релевантные результаты в топе. Добавление BM25 к E5-large в ансамбле дало небольшой прирост (Precision вырос на 0.6 процентных пункта, Recall на 0.9), подтверждая, что гибридный подход может улучшить некоторые случаи, где ключевые слова важны.

Таблица 2.2 — Результаты на RuBQ (P@3, P@5, R@3, R@5, F1@3, F1@5, MAP@3, MAP@5)

Подход	P@3	P@5	R@3	R@5	F1@3	F1@5	MAP@3	MAP@5
Ensemble								
E5-large + BM25	0.3889	0.2717	0.7791	0.8877	0.4956	0.3997	0.68916	0.72967
E5-large + FRIDA	0.4390	0.2844	0.8789	0.9295	0.5592	0.4185	0.83972	0.86277
E5-large + FRIDA + BM25	0.4141	0.2799	0.8350	0.9167	0.5291	0.4121	0.75996	0.79351
FRIDA + BM25	0.3770	0.2518	0.7648	0.8340	0.4825	0.3720	0.67983	0.70629
MultiStep								
BM25 → E5-large	0.3590	0.2216	0.7045	0.7154	0.4539	0.3251	0.68600	0.69284
BM25 → E5 → FRIDA	0.1958	0.1179	0.3974	0.3986	0.2512	0.1756	0.38819	0.38865
BM25 → FRIDA	0.1958	0.1179	0.3974	0.3986	0.2512	0.1756	0.38819	0.38865
BM25 → FRIDA → E5	0.1961	0.1181	0.3977	0.3989	0.2514	0.1758	0.39508	0.39554
E5-large → FRIDA	0.2407	0.1474	0.4912	0.4982	0.3096	0.2195	0.47862	0.48167
FRIDA → E5-large	0.2442	0.1479	0.4961	0.4990	0.3135	0.2201	0.49200	0.49334
SingleStep								
BM25	0.2410	0.1624	0.4782	0.5322	0.3061	0.2394	0.42090	0.43795
E5-large	<u>0.4635</u>	<u>0.2941</u>	<u>0.9187</u>	<u>0.9541</u>	<u>0.5882</u>	<u>0.4318</u>	<u>0.88537</u>	<u>0.90338</u>
FRIDA	0.2405	0.1473	0.4910	0.4980	0.3094	0.2194	0.47800	0.48106

В случае RuBQ картина несколько иная. Здесь безусловным лидером по всем метрикам выступает E5-large в одностадийном варианте: она достигает Precision@3 = 46.35% и Recall@3 = 91.87%, что значительно превосходит другие методы. Фактически, E5-large сумела для 91.9% вопросов поместить хотя бы один правильный фрагмент в тройку лучших результатов – отличный показатель. Для сравнения, у BM25 и FRIDA recall@3 лишь 48%–49%, то есть почти вдвое хуже. Такая разница обусловлена тем, что многие русскоязычные вопросы сформулированы иначе, чем тексты ответов (например, синонимы, морфологические вариации), и только семантическая модель E5-large справляется с этим соответствием. Кроме того, E5-large оказалась способна работать мультиязычно: несмотря на первоначальное обучение на английском, её мультиязычная версия успешно перенесла знания на русский домен.

Ансамблевые методы на RuBQ не дали прироста сверх E5-large. Хотя комбинация E5-large + FRIDA показала вторую по величине точность (P@3 = 43.9%) и высокую полноту (R@3 = 87.89%), она всё же уступает E5-large. Добавление BM25 в ансамбль лишь снижает показатели. Это указывает на то, что для русских вопросов модель E5-large уже охватывает почти все релевантные случаи, а выдача BM25 скорее приносит шум.

Многоступенчатые пайплайны на RuBQ показали себя плохо. Конфигурация

BM25 -> E5-large дала $P@3 = 35.9\%$ (против 46.3% у одной E5-large) и $Recall@3$ лишь 70.4% . Здесь проявляется типичный недостаток каскадов: если первый этап (BM25) пропускает релевантный документ, второй его уже не сможет вернуть. Аналогичная ситуация с пайплайнами, стартующими с FRIDA: FRIDA -> E5-large дает $recall@3$ всего 49.6% . Таким образом, каскады не оправдали себя для RuBQ – лишние этапы лишь теряют информацию.

Интересно отметить, что для RuBQ разница между $Precision@3$ и $Recall@3$ у лучшей модели E5-large довольно большая (0.463 против 0.919). Это объяснимо: почти каждый вопрос имеет один правильный фрагмент, и E5-large почти всегда его находит, но при этом топ-3 содержит ещё 2 неверных ответа, снижающих $precision$ примерно до 46% .

В целом, результаты показывают, что для HotpotQA умеренно сложные гибридные подходы обеспечивают наилучшее качество, тогда как для RuBQ достаточно одной мощной семантической модели. Далее рассмотрим конкретные примеры, иллюстрирующие успехи и неудачи этих подходов.

Посмотрим на конкретном примере, почему ансамбль E5-large и BM25 оказывается лучше одной модели BM25. Чтобы ответить на сложный вопрос "What government position was held by the woman who portrayed Corliss Archer in the film Kiss and Tell?" необходимо, во-первых, найти информацию о том, кто играл конкретного персонажа в "Kiss and Tell" (это один релевантный документ), а, во-вторых, определить какую позицию занимал этот человек в правительстве (второй релевантный документ). Комбинация E5-large и BM25 справилась с этой задачей. В топ-5 оказались оба релевантных документа со следующим содержанием: "Kiss and Tell is a 1945 American comedy film starring then 17-year-old Shirley Temple as Corliss Archer ..." и "Shirley Temple Black ... served as Chief of Protocol of the United States.". Здесь BM25 быстро нашёл страницы по именам, а E5-large повысил рейтинг именно необходимых текстов, потому что в вопросе фигурирует концепция "position" и модель понимает, что искомое – занимаемая должность актрисы. Если же посмотреть на результаты BM25, то там в топе оказывается только один релевантный документ - тот в котором говорится о фильме - так как он содержит больше лексических конструкций, встречающихся в вопросе. Другой документ, необходимый для ответа на вопрос, не входит даже в топ-10 выдачи по версии этой модели. На втором месте располагается следующий текст "A Woman Called Moses ... was portrayed by Cicely Tyson.", поскольку он содержит слова "woman" и "was portrayed", которые также фигурируют и в вопросе.

Также на примере вопроса "Кто придумал Винни-Пуха и Пятачка?" можно убедиться в том, что E5-large оказывается лучше более простой модели BM25. Эмбе́ддер без труда находит нужный документ и помещает его в топ-1: "Всего Аланом Милном было написано две прозаические книги с участием мишки: «Винни-Пух» (1926) ...". BM25 здесь менее эффективна - на первом месте тут оказывается абсолютно нерелевантный документ "Ридлиграммы ... раскладка фильма, выполненная Ридли Скоттом.

Делая её, он придумал ... видимо из-за слова "придумал".

В целом, приведённые примеры подтверждают статистические выводы: семантическая модель E5-large наиболее надёжно извлекает информацию даже при перефразировании и отсутствии прямого лексического совпадения.

2.3.6 Выводы

Лучшие результаты на HotpotQA достигнуты гибридными методами (ансамбль или каскад, включающие E5-large и BM25). Для RuBQ наилучший подход – единичный эмбеddер E5-large. Таким образом, модель E5-large можно считать оптимальной в общем случае, а использование BM25 полезно, когда требуется повысить метрики на многошаговых вопросах. Модель FRIDA в текущем виде не рекомендуется в качестве основного ретривера из-за заметно более низких показателей, хотя потенциально её можно применять совместно с другими.

С точки зрения производительности, одностадийный поиск E5-large не только наиболее результативен, но и оказался самым быстрым (около 40–50 мс на запрос в наших условиях). Это объясняется тем, что весь расчет – кодирование вопроса и быстрое вычисление похожести в векторном индексе. В то же время, каскад BM25 -> E5-large требует порядка 300 мс, а сложные схемы с FRIDA – до 300-500 мс на запрос, что существенно медленнее. Таким образом, при развертывании системы вопрос-ответ следует учитывать ограничение по времени: E5-large обеспечивает наилучшее соотношение точность/скорость на изученных наборах. Гибридные решения стоит применять, только если даже небольшое улучшение метрик оправдывает увеличение времени ответа.

2.4 Получение количественной оценки качества работы LLM-агента, созданного в ходе реализации алгоритма с использованием открытых публичных данных

2.4.1 Постановка исследования

Объект исследования – построение и использование графа знаний в качестве внешней памяти для LLM-агента. Цель исследования – получение эффективных конфигураций алгоритмов поиска релевантной информации в пространстве памяти LLM-агента для решения QA-задач.

2.4.2 Источники данных

Для оценки эффективности предложенного QA-пайплайна были выбраны HotpotQA- и RuBQ-датасеты. В результате предобработки исходных тестовых да-

тасетов, для построения графов знаний и оценки качества решения QA—задачи используются их подвыборки (детальная статистика приведена в Таблице 2.3):

- HotpotQA:
 - количество qa-пар – 2000;
 - количество релевантных контекстов – 3933.
- RuBQ:
 - количество qa-пар – 1350;
 - количество релевантных контекстов – 3503.

Таблица 2.3 — Информация по длине релевантных контекстов в рамках подготовленных подвыборок HotpotQA– и RuBQ-датасетов

Метрика / Датасет	Contexts Amount	Context Length				
		Mean	Std	Median	Min	Max
HotpotQA	3933	413.72	201.05	384	66	1019
RuBQ	3503	406.15	232.65	360	66	1020

2.4.3 Конфигурации экспериментов

Для оценки эффективности предложенного QA—пайплайна выполнена серия запусков (Ablation) при различных конфигурациях значений его гиперпараметров. В качестве LLM для перебора/сравнения были выбраны как 7B/8B-size, так и large-size («14B») представителей: Llama3.1 8B, GigaChat Max и DeepSeek V3. В качестве алгоритмов обхода графа тестируются NaiveRetriever (NR), WaterCircles (WC), BeamSearch (BS), а также комбинация BS и WC ("BS + WC"). Значения базовых алгоритмов в рамках ablation-экспериментов не перебирались и были фиксированы:

- NaiveRetriever: "max_k" – 50.
- WaterCircles: "strict_filter" – True; "hyper_num" – 15; "episodic_num" – 15; "chain_triplets_num" – 25; "other_triplets_num" – 6; "do_text_pruning" – False.
- BeamSearch: "max_depth" – 5; "max_paths" – 10; "same_path_intersection_by_node" – False; "diff_paths_intersection_by_node" – False; "diff_paths_intersection_by_rel" – False; "mean_alpha" – 0.75; "final_sorting_mode" – mixed.

Помимо этого перебирались значения гиперпараметра, отвечающего за вариант ограничений, который накладывается на граф знаний при его обходе одним из доступных алгоритмов. Ограничения связаны с типами вершин, которые запрещается посещать: "E"— запрещается посещать episodic-вершины; "T"— запрещается посещать thesis-вершины; "O"— запрещается посещать object-вершины; "all"— никаких ограничений на вершины графа не накладывается. Каждая QA—конфигурация оценивалась на 100 парах вопрос/ответ из HotpotQA— и RuBQ—подвыборок. При этом LLM, заданная в рамках конкретной QA—конфигурации и LLM, с помощью которой был построен соответствующий граф знаний для запуска QA—конвейера, совпадают. Таким

образом, суммарно в рамках ablation—экспериментов выполнена оценка качества 268 QA—конфигураций.

2.4.4 Результаты экспериментов

2.4.4.1 Характеристики построенных графов знаний

Для запуска QA—конвейера в рамках ablation—экспериментов выполнено построение четырёх графов знаний на заданном датасете и с помощью заданной LLM—модели. Характеристики построенных графов представлены в Таблице 2.4.

Таблица 2.4 — Характеристики графов знаний, построенных в рамках ablation—экспериментов

LLM-модель	Количество контекстов для сохранения в граф	Количество вершин			Количество связей			Среднее количество смежных вершин (по типу)			
		episodic	thesis	object	episodic	hyper	simple	object-соседей (y episodic-вершин)	thesis-соседей (y episodic-вершин)	object-соседей (y thesis-вершин)	object-соседей (y object-вершин)
		HotpotQA									
GigaChat Max	3933	3931	25211	49455	103186	82614	42249	19.83	6.42	3.27	1.45
DeepSeek V3		3933	30583	51333	111318	96259	55220	20.52	7.87	3.14	1.54
Llama3.1 8B		3933	26472	40930	90127	75510	29977	16.18	6.82	2.85	1.47
		RuBQ									
GigaChat Max	3503	3450	21174	51022	99056	71111	31202	22.56	6.16	3.35	1.29

2.4.4.2 Анализ эффективности QA—пайплайна в различных конфигурациях

В результате проведённых экспериментов подготовлена сравнительная таблица с лучшими QA—конфигурациями, а также её расширенная версия: см. Таблицу 2.5 и Таблицу 2.6 соответственно.

Таблица 2.5 — Лучшие QA—конфигурации по LLM-as-a-Judge метрике (Qwen2.5 7B), выявленные в рамках ablation—экспериментов

Модель			
/	GigaChat Max	DeepSeek V3	Llama3.1 8B
Датасет			
HotpotQA	0.79 / NR / all	0.64 / BS + WC / O	0.59 / BS + WC / O
RuBQ	0.87 / NR / all	-	
Mean:	0.83	0.64	0.59

Из таблицы 2.5 видно, что на HotpotQA—датасете лучшая оценка по LLM-as-a-Judge метрике получена при использовании GigaChat Max LLM—модели. При этом в качества алгоритма поиска/извлечения информации задан NaiveRetriever без ограничений на тип допустимых триплетов. В случае DeepSeek V3 и Llama3.1 8B моделей лучшей оказалась QA—конфигурация с комбинированным алгоритмом обхода графа и ограничением на обход object-вершин.

Таблица 2.6 — Лучшие показатели эффективности по LLM-as-a-Judge метрике (Qwen2.5 7B) QA-конфигураций в рамках заданной версии QA-пайплайна и алгоритма обхода графа

LLM-модель	Алгоритм обхода графа	LLM-as-a-Judge оценки			
		HotpotQA		RuBQ	
		weak QA-pipeline	medium QA-pipeline	weak QA-pipeline	medium QA-pipeline
GigaChat Max	BS + WC	0.66 / E	0.75 / O	-	
	BS	0.57 / O	-	0.8 / E	-
	NR	0.74 / all	0.79 / all	0.83 / E	0.87 / all
DeepSeek V3	BS + WC	0.52 / O	0.64 / O	-	
	BS	0.41 / E	-		
	NR	0.35 / E	0.54 / all		
Llama3.1 8B	BS + WC	0.59 / O	0.59 / O		
	BS	0.32 / E	-		
	NR	0.28 / E	0.36 / E		

По значениям из Таблицы 2.6 видно, что средний прирост эффективности QA-пайплайна по LLM-as-a-Judge метрике при использовании medium-версии (с механизмом планирования) по сравнению с weak-версией (заданных алгоритм обхода графа запускается один раз) составляет 6.4%.

2.4.5 Выводы

В результате проведённого исследования были выявлены лучшие конфигурации предложенного QA-пайплайна. В случае HotpotQA-датасета конфигурация из GigaChat Max LLM-модели, NaiaveRetriever алгоритма извлечения информации из графа (без наложения ограничений на тип доступной информации) позволила достичь следующих показателей эффективности: 0.61 по ExactMatch метрике (получен прирост по сравнению с существующими GraphRAG-методами, которые были описаны в Разделе 1.4) и 0.79 по LLM-as-a-Judge метрике. В случае RuBQ-датасета конфигурация из GigaChat Max LLM-модели, NaiaveRetriever алгоритма извлечения информации из графа знаний (без наложения ограничений на тип доступной информации) позволила достичь следующих показателей эффективности: 0.83 по LLM-as-a-Judge метрике.

2.5 Сравнение результатов по методам ToG, RoG, PMKGE, ToG2.0, DoG, PDA на HotpotQA-датасете по метрике Exact Match

В случае HotpotQA-датасета разработанное решение (конфигурация из GigaChat Max модели, NaiaveRetriever алгоритма извлечения информации из графа (без наложения ограничений на тип доступной информации)) позволила достичь 0.61 по ExactMatch метрике. Это превышает существующие GraphRAG-методы (см. таблицу 2.7), которые были описаны в Разделе 1.4.

Таблица 2.7 — Сравнение характеристик моделей на наборе HotpotQA (ЕМ, %)

Метрика	Набор	ToG (ChatGPT/ GPT-4)	RoG (Llama2- 7B)	PMKGE (GPT-3.5/ GPT-4)	ToG2.0 (GPT-3.5)	DoG (GPT-4/ GPT-3.5)	PDA (GPT-3.5)	Текущий подход (GigaChat Max)
ЕМ	HotpotQA	—/41,0	43,0	—/42,6	40,9	45,3	36,5	<u>61,0</u>

3 Разработка модели памяти LLM-агента на основе темпорального графа знаний и алгоритма поиска информации в ней

3.1 Модель памяти LLM-агента на основе темпорального графа знаний

К недостаткам предложенной/реализованной модели памяти (см. Раздел 2.1) можно отнести следующие моменты:

1. Онтология генерируемого графа знаний достаточно простая. Предложенного количества критериев, по котором можно осуществлять поиск и фильтрацию единиц информации недостаточно, что, в свою очередь, сказывается на эффективности и производительности QA-операций.
2. Из-за используемой онтологии графа, в соответствии с которой информация приводится в достаточно общие концепты, процедура формирования пространства памяти ведёт себя достаточно нестабильно и сильно зависит от качества используем LLM-модели, что может привести к потере части полезной информации и ограничить область применения такого решения.
3. В текущей реализации Memorize-пайплайна разные вершины графа могут содержать одну и ту же единицу информации, при этом представленную в разных формулировках. Наличие неявных дубликатов знаний может привести к потере целостности памяти после запуска процедуры по поиску устаревшей информации, а также к повышению вероятности возникновения противоречий между извлекаемыми фрагментами информации в рамках решения QA-задач.
4. В текущей реализации Memorize-пайплайна отсутствует стадия по определению/извлечению интервалов времени, в рамках которых каждая из сохраняемых единиц информации может считаться действительной/актуальной. Без возможности отслеживания валидности хранимой информации реализуемые поисковые QA-алгоритмы с большей вероятностью будут возвращать устаревшие, несоответствующие требованиям и вводящие в заблуждения ответы на user-вопросы.

В качестве способа решения описанных проблем предлагается новая версия модели памяти на основе темпорального графа знаний. Модифицированная версия Memorize-конвейера по формированию такого пространства памяти из текстов на естественном языке представлена на рисунке 3.1.1. Идея предлагаемой модели памяти взята из cookbook-поста от OpenAI "Temporal Agents with Knowledge Graphs".

Из рисунка 3.1.1 видно, что модифицированный вариант Memorize-конвейера состоит из 6 модулей. На вход конвейеру поступает неструктурированный фрагмент текста на естественной языке, информацию из которого необходимо сохранить в память LLM-агента. Данный текстовый фрагмент передаётся в "Statements Extraction"-модуль для выполнения его разбиения (с помощью LLM-модели) на независимые пол-

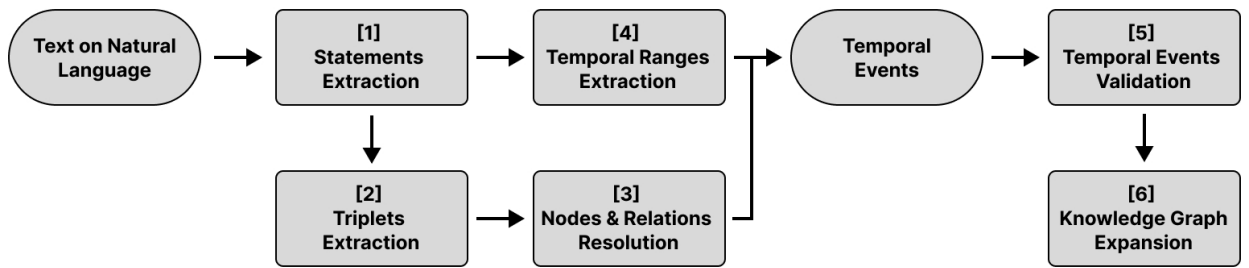


Рисунок 3.1.1 — Высокоуровневая архитектура конвейера по формированию/наполнению пространства памяти LLM-агента на основе темпорального графа знаний

ные единицы информации, которые будут представлять statement-вершины в графе; исходный текстовый фрагмент будет представлять отдельную episodic-вершину. Также выполняется категоризация извлечённых statement-вершин по episode- и temporal-типам: "FACT" , "OPINION" или "PREDICTION" и "STATIC" , "DYNAMIC" или "ATEMPORAL" соответственно. Далее полученный набор statement-вершин подаётся на вход двум модулям, которые могут выполнить их обработку в параллельном режим: "Temporal Ranges Extraction" и "Triplets Extraction". В рамках "Temporal Ranges Extraction"-модуля для каждой statement-вершины определяется/фиксируется интервал времени, в рамках которого содержащаяся в ней (в текущей statement-вершине) единица информации является актуальной/валидной. Заполняются следующие temporal-поля: $t_{created}$, t_{valid} , $t_{invalid}$, $t_{expired}$. В случае, если в statement-вершине присутствует относительная дата (например, "прошлый вторник"), то выполняется её приведение к абсолютному значению на основании заданных "внутренних часов" модуля. В рамках "Triplets Extraction"-модуля выполняется (с помощью LLM-модели) приведение единиц информации из каждой statement-вершины к формату "text-attributed"-триплетов: subject, predicate, object. Элементы "subject" и "object" соответствуют entity-вершинам в графе. Помимо самого названия (name) сущности, в entity-вершине хранится её тип (type) и краткое описание (description) для сохранения её контекста. Отношения между entity-вершинами имеют simple-тип и, помимо названий (name) отношений между конкретной парой атомарных элементов, хранят их определения (definition) в виде текста на естественном языке. Стоит обратить внимание, что simple-отношения формулируются в нейтральной форме ко времени (time-neutral), а entity-вершины в name-поле не могут содержать даты и отметки времени. Данные дополнительные поля у entity-вершин и simple-отношений нужны для их последующей унификации в рамках "Nodes & Relation Resolution"-модуля. Унификация выполняется на основе семантической близости к уже существующим вершинам и отношениям в графе. В результате обработки исходного фрагмента текста в рамках модулей 1–4 будет получен набор temporal-событий (Temporal Events), каждый из которых содержит: statement-вершину, временной интервал валидности хранящейся единицы информации (приведённый к абсолютным значениям) и соответствующий набор simple-триплетов.

Перед сохранением нового набора temporal-событий в граф, в рамках "Temporal Events Validation"-модуля, выполняется их проверка на актуальность/валидность при условии тех единиц информации и "time-validiy"-интервалов, которые уже в нём (данном графе) хранятся. Поиск устаревшей информации выполняется в две стороны:

1. Incoming vs. Existing. Выполняется поиск temporal-событий из числа новых, которые можно признать устаревшими за счёт уже существующих temporal-событий в графе.
2. Existing vs. Incoming. Выполняется поиск temporal-событий в существующем графе, которые можно признать устаревшими за счёт temporal-событий из числа новых/входящих.

В случае, если было найдено устаревшее temporal-событие, то у него редактируется *t_{invalid}*-поле, а также, в его "invalidated_by"-поле сохраняется идентификатор актуального temporal-события. В результате выполнения вышеописанных действий, информация из сходного текста в виде temporal-событий может быть добавлена в граф.

Традиционные графы знаний хранят информацию/знания о мире в виде статичных элементов, однако реальная информация постоянно меняется. То, что было верным полгода назад, сегодня может оказаться устаревшим, что создает риск ошибок/неверных ответов системы, если память LLM-агента на основе графа не учитывает изменения во времени. Темпоральные графы знаний позволяют (1) выполнять более точный поиск и предоставлять корректную информацию для генерации ответов на вопросы вроде "Что было верным на определенную дату?" и (2) анализировать, как менялись факты и взаимосвязи между ними, гарантируя, что ответ всегда обуславливается на наиболее актуальную информацию: см. Рисунок 3.1.2.

3.1.1 Представление знаний в графе

Предлагаемый вариант темпорального графа состоит из вершин трёх типов. Episodic – содержит в себе исходный фрагмент текста, который требуется сохранить в граф. Statement – содержит независимую полную единицу информации, извлеченную из соответствующей episodic-вершины. Данный тип вершин содержит следующие темпоральные поля: *t_{created}*, *t_{valid}*, *t_{invalid}*, *t_{expired}*, *invalidated_by*. Entity – содержит атомарную неделимую сущность, извлечённую из соответствующей statement-вершины. Данный тип вершин, помимо name-поля содержит следующие контекстные поля: *type*, *description*.

Episodic- и statement-вершины могут быть соединены гипер-ребром с episodic-типом (без текстового атрибута). Statement- и entity-вершины могут быть соединены гипер-ребром с statement-типом (без текстового атрибута). Entity-вершины могут быть соединены ребром с simple-типом; данный тип отношений содержит name- и definition-поля для описания характера связи. Между episodic- и entity-вершинами прямого отношения (связи) быть не может.

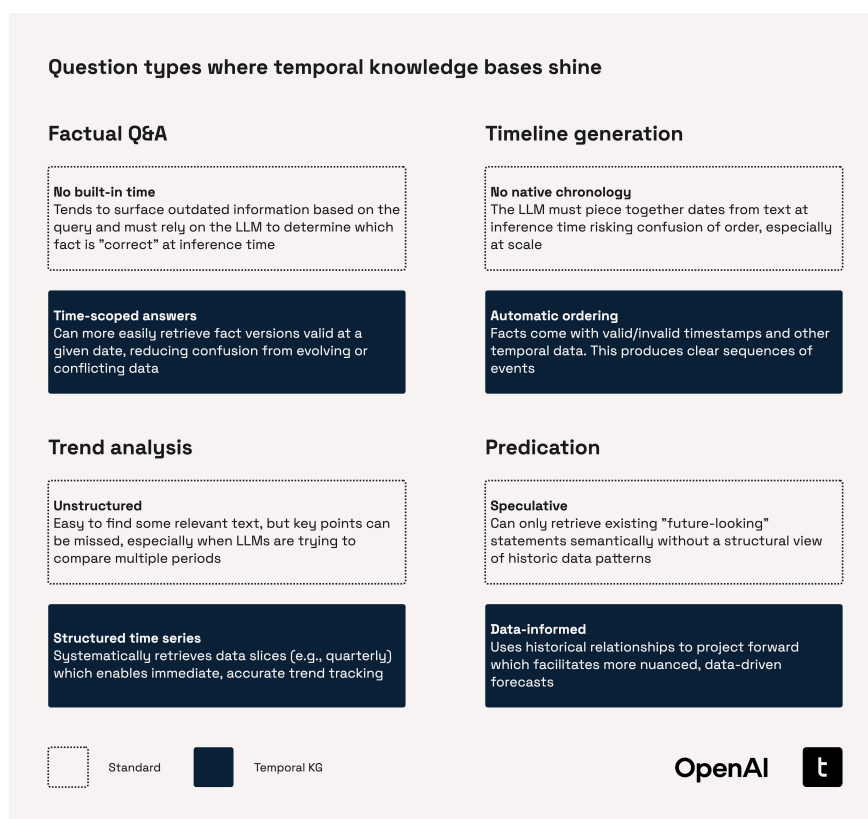


Рисунок 3.1.2 — Типы вопросов, для которых модель памяти на основе темпорального графа знаний является более подходящей по сравнению со стандартными (time-insensitive) онтологиями графов. Рисунок взят из cookbook-поста от OpenAI "Temporal Agents with Knowledge Graphs"

Также, statement-вершинам выполняется присвоение меток двух категорий: episode- и temporal-категорий. В случае episode-категории доступны следующие три метки:

- FACT – "Statements that are objective and can be independently verified or falsified through evidence."
- PREDICTION – "Statements that contain personal opinions, feelings, values, or judgments that are not independently verifiable. It also includes hypothetical and speculative statements."
- OPINION – "Uncertain statements about the future on something that might happen, a hypothetical outcome, unverified claims. It includes interpretations and suggestions. If the tense of the statement changed, the statement would then become a fact."

В случае temporal-категории доступны следующие три метки:

- ATEMPORAL – "Statements that will always hold true regardless of time therefore have no temporal bounds."
- STATIC – "Often past tense, think -ed verbs, describing single points-in-time. These statements are valid from the day they occurred and are never invalid. Refer to single points in time at which an event occurred, the fact X occurred on that date will always hold true."

- DYNAMIC – "Often present tense, think -ing verbs, describing a period of time. These statements are valid for a specific period of time and are usually invalidated by a Static fact marking the end of the event or start of a contradictory new one. The statement could already be referring to a discrete time period (invalid) or may be an ongoing relationship (not yet invalid)".

3.1.2 Алгоритм поиска устаревшей информации в графе

Предлагаемый алгоритм поиска устаревшей информации в темпоральном графе может быть разбит на шесть основных/общих шагов: см. Рисунок 3.1.3.

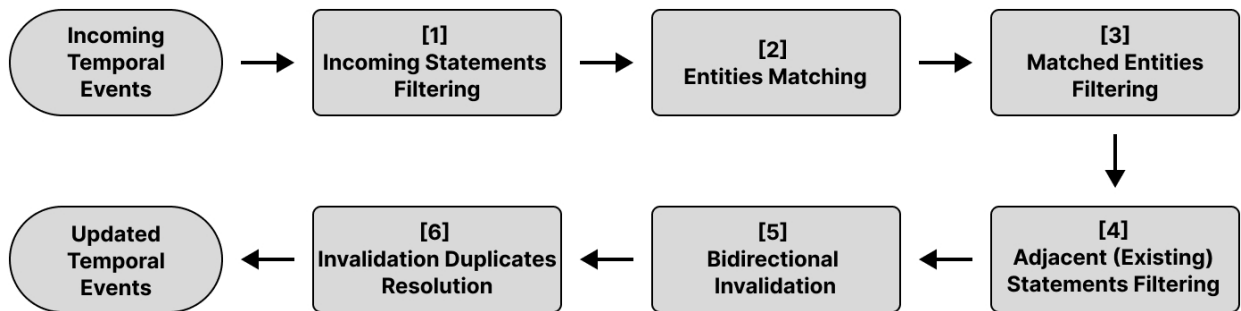


Рисунок 3.1.3 — Высокоуровневая архитектура пайплайна по поиску устаревшей информации в темпоральном графе знаний

Как показано на рисунке 3.1.3, на вход алгоритму поступает набор ранее сформированных Temporal-событий. В рамках первого "Incoming Statements Filtering"-шага выполняется фильтрация statement-вершин из входящих/новых temporal-событий за счёт следующих условий: (1) в качестве episode-метки должно быть указано значение "FACT" и (2) в качестве temporal-метки не должно быть указано значение "ATEMPORAL". Отфильтрованные таким образом statement-вершины (с мета-информацией из соответствующих temporal-событий) переходят на следующий шаг обработки. На втором "Entities Matching"-шаге для каждой входящей/новой statement-вершины выполняется сопоставление её entity-вершин с существующими entity-вершинами в темпоральном графе. В рамках третьего "Matched Entities Filtering"-шага выполняется фильтрация сопоставленных существующих entity-вершин за счёт следующих условий: каноническая entity-вершина для новой/входящей и существующей/сопоставленной entity-вершин должны совпадать. Отфильтрованные таким образом entity-вершины (с мета-информацией из соответствующих temporal-событий) переходят на следующий шаг обработки. В рамках четвёртого "Adjacent (Existing) Statements Filtering"-шага выполняется отбор/фильтрация существующих statement-вершин из графа, которые смежны с сопоставленными entity-вершинами, за счёт следующих условий: (1) в качестве episode-метки должно быть указано значение "FACT", (2) в качестве temporal-метки не должно быть указано значение "ATEMPORAL" (3) значение предиката (на уровне триплетов) у существующей и новой (которой соответ-

ствуется текущая entity-вершина) statement-вершин должны совпадать, (4) интервалы валидности для текущей пары "incoming" и "existed" statement-вершин должны пересекаться, (5) должен быть превышен заданный порог оценки семантической близости между "incoming" и "existed" statement-вершинами в текущей паре. Также можно задать максимальное количество "existed" statement-вершин, которое может приходиться на одну "incoming" statement-вершину для invalidation-процедуры. В результате данного шага каждой "incoming" statement-вершине будет сопоставлен набор "existing" statement-вершин для их перекрёстной проверки на актуальность в рамках следующего шага. На пятом "Bidirectional Invalidation"-шаге для каждой пары ("incoming" statement-вершина, набор релевантных "existed" statement-вершин) запускается процедура поиска устаревшей информации. С помощью LLM-модели для каждой пары "incoming" и "existed" statement-вершин выполняется перекрёстная проверка: (1) можно ли признать информацию в "existed" statement-вершине устаревшей на основании информации из "incoming" statement-вершины и (2) можно ли признать информацию в "incoming" statement-вершине устаревшей на основании информации из "existed" statement-вершины. Если statement-вершина признаётся устаревшей, то у неё изменяются значения в следующих temporal-полях: $t_{invalid}$, $t_{expired}$, $invalidated_by$. В рамках последнего шестого "Invalidation Duplicates Resolution"-шага разрешаются конфликтные ситуации, когда одна и та же statement-вершина признавалась устаревшей несколько раз за счёт разных других statement-вершин. В таком случае в соответствующие temporal-поля у устаревшей statement-вершины указываются самые ранние, с точки зрения invalid-отметки времени, значения. В результате работы данного алгоритма будут скорректированы/актуализированы темпоральные свойства у новых и существующих в графе statement-вершин; пример работы данного алгоритма представлен на Рисунке 3.1.4.

3.1.3 Выводы

За счёт усиления онтологии генерируемого графа, добавления механизма по явному выделению/учёту темпоральных свойств и увеличения количества критериев для фильтрации хранимой информации, ожидается: (1) повышение стабильности работы алгоритмов по построению и актуализации графа (снижение влияния LLM-модели на качество) и (2) повышение эффективности и производительности решения QA-задач на основе предложенного темпорального графа знаний.

3.2 Разработка алгоритма извлечения информации из графа знаний

Немалая доля вопросов пользователя может быть связана с конкретными временными метками. Тем более, если домен графа знаний относится, к примеру, к банковской сфере. Пользователя могут интересовать условия вклада на конкретную дату или

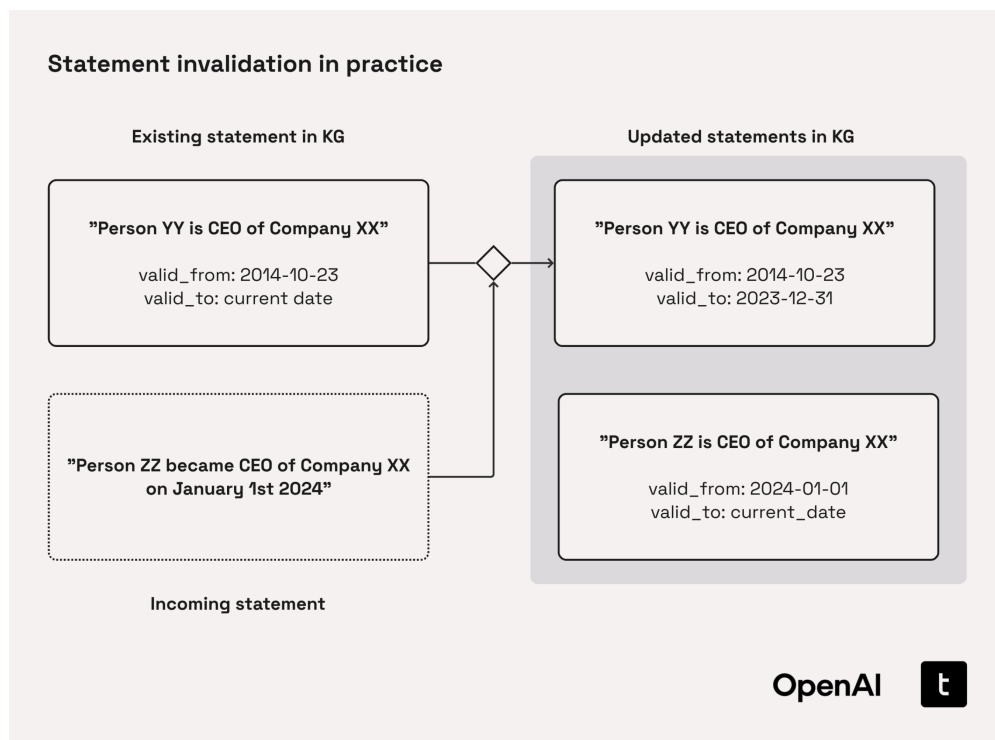


Рисунок 3.1.4 — Пример по корректировке полей валидности/актуальности у statement-вершин. Рисунок взят из cookbook-поста от OpenAI "Temporal Agents with Knowledge Graphs"

время работы отделений банка. Если дополнительно не прорабатывать логику работы с временными метками, мы рискуем либо не ответить на подобные вопросы, либо дать некорректный ответ. Действительно, одной семантической близости может оказаться недостаточно. Предположим, что пользователь спрашивает "Кто сейчас возглавляет компанию X?" и после обхода графа мы нашли следующую релевантную информацию "В 2023 году компанию X возглавляет Y" и "Директором предприятия X с 2025 года является Z". LLM может не распознать какой факт соответствует временной метке "сейчас" и дать некорректный ответ, особенно учитывая, что семантически первый ответ ближе к вопросу, чем второй. Поэтому естественным образом возникает необходимость адаптировать пайплайн для работы с временными метками.

Глобально, если в вопросе пользователя содержится временная метка, то она может быть двух типов: конкретная точка во времени или интервал. Если пользователь спрашивает "Что случилось 10 сентября 2023 года?", то этот вопрос относится к первому типу ($T = 10-09-2023$). Если же вопрос, например, "Как менялась ставка в 2024 году?", то мы имеем интервал (от $T_1 = 01-01-2024$ до $T_2 = 31-12-2024$ включительно), причем интервал может быть, как в этом случае "нечетким" и тогда мы сами определяем для него максимальный диапазон, а может быть конкретно задан пользователем ("Какие были условия кредитования в период с 7 октября 2022 по 1 февраля 2023?"). Кроме того, сама метка может быть задана как абсолютным значением, так и относительным ("Какая ключевая ставка была 2 недели назад?"). В первом случае ничего дополнительно делать не требуется, а во втором необходимо привести дату к абсолют-

ному значению. Если в вопросе пользователя встречается такая метка, мы с помощью LLM будем ее извлекать и преобразовывать, отталкиваясь от текущей даты, при которой был запущен пайплайн. Также по умолчанию для временных меток мы фиксируем следующий формат "%Y-%m-%d %H:%M:%S", и в случае если дата, указанная пользователем, не соответствует требуемой гранулярности, будем приводить ее к указанному формату: если в вопросе было "12 дня 18 января 2005", то мы преобразуем к "2025-01-18 12:00:00".

Сами вопросы, связанные со временем, могут сильно отличаться и, как следствие, требовать разных усилий для нахождения ответа. Пользователь может задать тривиальный вопрос, для поиска релевантной информации на который достаточно 2-3 шагов несложного метода обхода графа, а может, напротив, спросить что-то сложное и многокомпонентное, для чего потребуются комбинация продвинутых алгоритмов извлечения информации из графа. В силу данного факта логично адаптировать стратегию поиска под тип пользовательского вопроса. Это позволит не только разумно расходовать ресурсы и балансировать нагрузку, но и давать более точные и надежные ответы. Поэтому наш QA-пайплайн, учитывающий временные метки, также предполагает адаптивный выбор стратегии поиска.

Важно отметить, что верхнеуровнево текущий алгоритм повторяет классический QA-пайплайн без временных меток. Мы все также имеем 13 стадий, изображенных на схеме 2.2.1. Только теперь к ним, во-первых, добавляются новые стадии, а, во-вторых, меняется сама реализация некоторых стадий. В частности генерация вспомогательных запросов, обход графа и фильтрация. Для упрощения восприятия мы иллюстрируем некоторые ключевые стадии конвейера на рисунке 3.2.1 и подробно опишем их для каждой стратегии поиска.

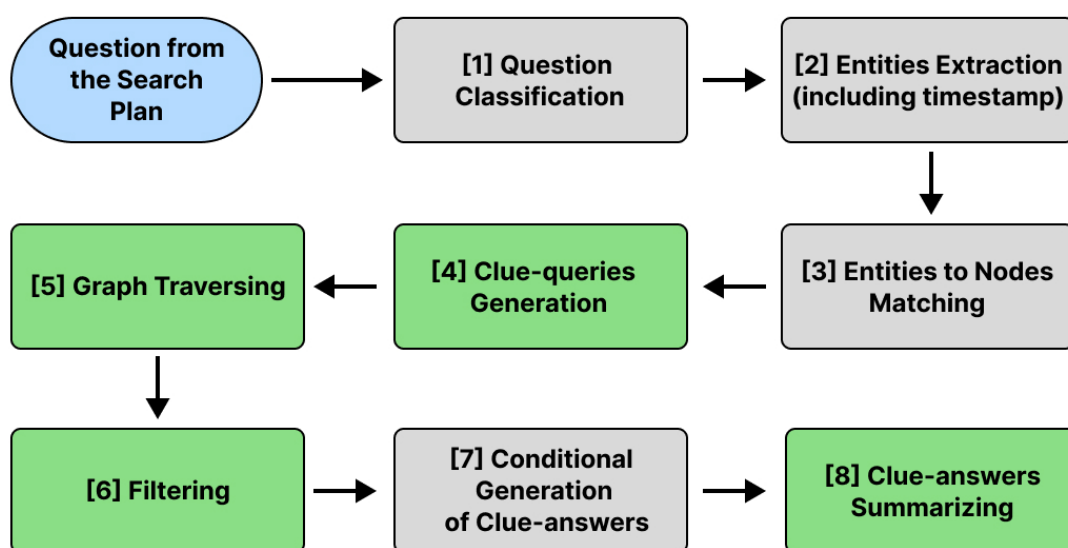


Рисунок 3.2.1 — Основные стадии QA конвейера для поиска релевантной информации в темпоральном графе. Ключевые этапы (которые меняются в зависимости от стратегии поиска) выделены зеленым цветом

Выбранный алгоритм извлечения информации зависит от типа пользовательского вопроса. То есть для входящего запроса решается задача классификации, по результатам которой запускается та или иная стратегия поиска. В рамках нашего QA-пайплайна логично классифицировать именно шаг плана поиска, так как, во-первых, от выбранной категории зависит в том числе и дальнейшая работа с clue-queries, а, во-вторых, это избавит нас от сложных multi-hop вопросов (так как на этой стадии исходный пользовательский вопрос уже декомпозирован).

Итак, когда алгоритм начинает работать с запросом, соответствующим шагу плана поиска, осуществляется стадия классификации. Всего предусмотрено 4 категории вопроса: Snapshot QA, Timeline/Range, Trend/Aggregation и Universal QA. Остановимся на каждой категории подробнее и опишем подходящие для них стратегии поиска.

3.2.1 Snapshot QA

К данной категории относятся конкретные вопросы с точной временной меткой T , например: "какая ставка на 25.10.2025?", "что произошло 18 декабря 2022?". После классификации происходит извлечение сущностей из вопроса. Только теперь к этой стадии добавляется отдельное обращение к LLM для того, чтобы получить временную метку. В prompt-подсказке описаны подробные инструкции, позволяющие модели извлекать даты разных типов. В рамках текущей стратегии поиска принципы работы с clue-queries остаются классическими (по извлеченным сущностям и сопоставленным им вершинам графа генерируем вспомогательные запросы). Затем реализуется стадия обхода графа - в качестве стартовой вершины берется извлеченная из вопроса временная метка. Поскольку данные вопросы достаточно простые, для них не требуются глубокие многошаговые маршруты. В силу этого используются следующие наивные алгоритмы обхода: поиск в ширину (с гиперпараметрами $\text{max_depth}=5$, $\text{max_width}=30$) и наивный поиск триплетов по семантической близости. Далее осуществляется стадия фильтрации:

1. Берем только те триплеты, чьи окна валидности покрывают T .
2. Исключаем инвалидированные триплеты: если запись покрывает T , но у нее есть `invalidated_by` с `valid_from` $\leq T$, ее необходимо отбросить: она уже была отменена на момент T .
3. Опционально: если нет триплетов, покрывающих T , можно взять последний известный до T .
4. Переранжирование по семантической близости (как и в классической фильтрации).

Может быть такое, что среди найденных релевантных триплетов есть противоречия. Это может помешать на стадии генерации ответа на вспомогательный запрос. Поэтому необходимо разрешать подобные конфликты. Для этого мы предоставляем LLM конкретные инструкции, которые актуальны и для других категорий вопросов. Идея в

том, чтобы разрешать противоречия, отталкиваясь от следующих приоритетов (именно в таком формате): временная релевантность, актуальность, полнота информации.

3.2.2 Timeline/Range

К данной категории относятся вопросы, связанные с историей изменений (что и когда изменилось), а также вопросы с временными промежутками (даже если явно не указано желание проследить изменения), например: "какая в 2024 году была ключевая ставка?", "что изменилось в условиях кредитования между январем и мартом?". Мы также извлекаем из вопроса временную метку (здесь уже подразумевается, что метка - интервал). Далее clue-queries генерируются как обычно. Затем происходит стадия обхода графа - теперь нам важно получить полную историю факта, поэтому мы хотим извлечь больше осмысленных цепочек. Для этого используются алгоритмы WaterCircles (`hyper_num=30-35`, `chain_triplets_num=25-35`, `strict_filter=False`) и лучевой поиск (`max_depth=5-10`, `max_paths=30-50`). Стадия фильтрации реализуется по следующему алгоритму:

1. Берем только те триплеты, чьи окна валидности пересекаются с временным интервалом вопроса $[T_1, T_2]$.
2. Ранжируем по семантической близости.
3. Сортируем триплеты по возрастанию `valid_from`.
4. Склеиваем одинаковые значения, идущие подряд (если у двух последовательных интервалов одинаковое нормализованное значение, то объединяем их в один (с `valid_from` первого и `valid_to` второго)).
5. Обрезаем временные метки по границам: у крайних интервалов сдвигаем `start = max(valid_from, T_1)`, `end = min(valid_to, T_2)`.

Кроме разрешения конфликтов важно обратить внимание LLM на то, что в найденных данных могут быть разрывы во времени. В этом случае важно, чтобы модель оставляла их, как есть, и не пыталась интерполировать.

3.2.3 Trend/Aggregation

Данная категория охватывает самый сложный тип вопросов. К ней относятся вопросы, в которых требуется сделать вывод о тренде (рост, падение и так далее). Иными словами, необходима не просто история изменений, как в предыдущей категории, а разбиение на временные промежутки и вывод. Примеры вопросов: "Как менялась ставка по месяцам в 2024 году?", "Какой тренд прослеживается в изменениях ключевой ставки за последние 2 года?".

В рамках этой категории мы предполагаем, что вопрос кроме временного промежутка может также содержать требуемую гранулярность: месяц/квартал и так далее,

за которую необходимо посчитать статистики и на их основе сделать вывод о тренде. Поэтому из вопроса мы извлекаем не только временной интервал, но и требуемую гранулярность (если ее в вопросе нет - берем значение по умолчанию). Для текущей категории вопросов меняется стратегия работы с clue-queries. Теперь мы генерируем вспомогательный запрос для каждой пары [гранулярная отметка, весь временной промежуток]. Например, для вопроса "какой тренд прослеживается в изменении ключевой ставки за 2023 год?" мы генерируем 12 clue-queries (требуемой гранулярности в вопросе нет, но мы используем значение по умолчанию - месяц): "что было с ключевой ставкой в январе 2023?", "что было с ключевой ставкой в феврале 2023 года?" и так далее. Затем для каждого сгенерированного вспомогательного запроса запускается обход графа. Поскольку нам нужно сделать вывод о тренде, мы хотим получить максимально полную историю факта. Для этого используем снова WaterCircles (hyper_num=35-40, chain_triplets_num=35-50, strict_filter=False) и лучевой поиск (max_depth=10-15, max_paths=50-70), но выставяем гиперпараметры для более полного поиска. После происходит стадия фильтрации. Она в точности повторяет этап для предыдущей категории. Далее мы последовательно резюмируем ответы на эти clue-queries (проходя снизу-вверх), чтобы не столкнуться с проблемой lost in the middle: сначала суммаризируем вспомогательные запросы за каждый квартал, затем делаем финальный вывод за запрашиваемый год. Чтобы избежать большого числа запросов к LLM гранулярность разбиения подбирается адаптивно, в зависимости от временного промежутка, указанного в вопросе. Если пользователь просит проследить тренд за последние 10 лет, то логично брать года в качестве разбиения, а не месяцы.

В итоге данная категория накладывает изменения на большинство этапов получения ответа на вопрос из плана поиска: генерация вспомогательных запросов, обход графа, фильтрация, суммаризация запросов.

3.2.4 Universal QA

К Universal QA относятся вопросы, которые не попали в предыдущие категории. У данных вопросов может быть либо конкретная временная метка, либо интервал, либо временная составляющая может полностью отсутствовать: "какие условия по вкладу 'Лучший'?", "когда последний раз температура поднималась выше 30 градусов?". Стратегия работы с clue-queries здесь классическая. В качестве обхода графа выбран универсальный метод, позволяющий найти релевантную информацию, не затрачивая много времени и ресурсов. Используется комбинация WaterCircles (hyper_num=25, chain_triplets_num=25) и наивного поиска триплетов. Если в вопросе не было временной метки, то стадия фильтрации такая же, что и в обычном QA-пайплайне. Если же временная метка есть, то фильтрация происходит по аналогии с тем, что спроектировано для категории Snapshot QA.

3.2.5 Выводы

Разработанный QA-пайплайн расширяет базовый конвейер за счет явной работы с абсолютными и относительными временными метками, их нормализации к единому формату и учета времени при извлечении знаний. Классификация шагов плана поиска на четыре категории (Snapshot QA, Timeline/Range, Trend/Aggregation, Universal QA) и привязка к ним специализированных стратегий, включающих в себя выбор обхода графа, схему генерации clue-queries и правила фильтрации триплетов, позволяет адаптировать глубину и характер поиска под задачу: от точечного ответа "на дату" до реконструкции истории изменений и анализа трендов.

Временная фильтрация кандидатов (покрытие отметок/интервалов, учет инвалидированных фактов, обрезка по границам интервалов), а также явные инструкции LLM по разрешению конфликтов по приоритетам "временная релевантность -> актуальность -> полнота" и сохранение разрывов без интерполяции делают ответы устойчивыми к противоречивым и неполным данным. В совокупности это обеспечивает корректную обработку широкого класса временно-зависимых вопросов, повышая точность и надежность ответов без радикального изменения структуры базового QA-пайплайна.

4 Направления развития предложенного решения

Дальнейшее развитие предложенного и реализованного решения может осуществляться в разных направлениях. Во-первых, очевидно, текущее решение не лишено недостатков и имеются места, которые можно проработать детальнее и улучшить. Во-вторых, можно добавлять новые компоненты и расширять уже реализованный функционал. В данном разделе представлены идеи и возможные следующие инкременты: модели памяти, алгоритма сохранения информации в памяти (Memorize-пайплайн) и алгоритма поиска информации (QA-пайплайн).

4.1 Технические улучшения

4.1.1 Семантическое entity-resolution

Одним из ключевых направлений развития является внедрение механизмов семантического разрешения сущностей (semantic entity-resolution), позволяющих обнаруживать и устранять неявные дубликаты знаний в модели памяти. В текущей реализации отдельные узлы могут представлять одну и ту же сущность, обозначенную разными именами или формулировками. Предстоит разработать методы унификации таких узлов – например, сопоставление новых фактов с уже существующими на основе сходства эмбедингов или контекстных определений. Решение этой задачи повысит целостность знаний: система будет избегать ситуаций, когда одна и та же реальная сущность представлена несколькими узлами, что снижает избыточность и устраняет противоречия. В научной литературе предлагаются подходы типа Extract-Define-Canonicalize [11], где LLM-модель последовательно извлекает факты, дает им определения и выполняет канонизацию сущностей. Интеграция подобных техник в конвейер построения графа позволит автоматически выявлять дублирующиеся вершины и сливать их, а также различать омонимичные упоминания. Семантическое слияние узлов на основе их атрибутов и связей укрепит связность графа и упростит последующее извлечение релевантных знаний.

4.1.2 Дообучение и легковесные LLM

Текущее решение опирается на большие языковые модели, поэтому важным техническим шагом станет возможность дообучения этих компонентов на специфических данных графовой памяти и задачах пользователя. В предложенном решении LLM решают множество задач (в том числе и задачи бинарной классификации). Дообучение моделей под соответствующие задачи поможет повысить точность ответов системы. Парал-

тельно рассматривается переход к легковесным LLM: использование моделей меньшего размера или оптимизированных версий, которые обеспечивают более быстрое и экономичное выполнение без существенной просадки ключевых метрик. Это может включать применение моделей с меньшим числом параметров, а также техники квантования (например, 4-битные веса) и дистилляции. Легковесные модели облегчают локальный запуск решения и масштабирование числа агентов – вместо одной крупной модели несколько компактных агентов смогут работать параллельно, сохраняя суммарную емкость знаний. Адаптация больших языковых моделей с помощью эффективных методов дообучения и сжатия моделей – важное и актуальное на сегодняшний день направление. Так авторы работы [33] представляют QLoRA – метод дообучения с 4-битной квантизацией, позволяющий существенно снизить требования к ресурсам при сохранении качества модели. Это демонстрирует ценность легковесных подходов (квантизация, дистилляция, low-rank адаптация) для локального дообучения LLM на специфичных доменах. Кроме того, комбинация дообучения и меньших моделей открывает путь к персонализации: модель можно адаптировать под стиль общения конкретного пользователя или домен задачи, не вызывая внешние API крупной модели. В целом, эти технические улучшения направлены на повышение эффективности системы: устранение дублей в памяти и оптимизация моделей снизят требования к ресурсам и улучшат точность ответов за счет более релевантного контекста.

4.1.3 Критерии оценки улучшения планирования в QA-пайплайне

Еще одним техническим аспектом является выработка формальных критериев, по которым можно измерять и сравнивать качество плана поиска и рассуждений в QA-конвейере. В многошаговом qa-пайплайне система строит план – набор шагов (поисковых запросов) с целью извлечения их графа знаний нужной информации. Оценка улучшения плана необходима, чтобы объективно судить, приводит ли новая версия алгоритма планирования к лучшим результатам. В роли критериев могут выступать эффективность (например, число шагов или вызовов модели до получения ответа – меньшее число указывает на более оптимальный план), полнота (доля охваченных необходимых фактов – высокий охват означает, что план не упустил важную информацию), а также точность результата (релевантность ответа вопросу). Можно также вводить специальные метрики согласованности плана: например, сравнивать логическую правильность порядка шагов или проверять отсутствие лишних посещений узлов графа. Еще один подход – привлечение самой LLM в роли критика: модель-"судья" может оценивать обоснованность цепочки рассуждений, выявлять ненужные переходы или факты вне контекста. В дальнейшем такие оценки могут использоваться для обучения системы планированию (через обучение с подкреплением: вознаграждая за планы, ведущие к корректному и краткому ответу). Благодаря формализации критериев оценки плана появится возможность итеративно улучшать модуль планирования и объективно

сравнивать различные стратегии.

Определение формальных метрик для оценки планирования критично для итеративного улучшения системы. В обзоре по возможностям планирования современных LLM-моделей ([34]) отмечается отсутствие согласованных метрик и предлагается использовать такие характеристики, как полнота, эффективность и оптимальность плана. Это подчеркивает необходимость формализации критериев оценки.

4.2 Поддержка диалогового взаимодействия

4.2.1 Архитектура многошагового взаимодействия с пользователем (диалоговая система)

Следующим направлением развития является преобразование системы из режима "один вопрос – один ответ" в полноценный диалоговый режим с возможностью многошагового уточнения запросов. Для этого требуется архитектура, позволяющую агенту запрашивать дополнительную информацию у пользователя, если исходный вопрос неполон, неоднозначен или слишком сложен для прямого ответа. В рамках такой архитектуры каждая сессия взаимодействия с пользователем будет хранить контекст диалога, а агент сможет задавать уточняющие вопросы прежде, чем приступить к поиску ответа. Например, если пользователь спрашивает расплывчато ("расскажи про достижения компании X"), система должна иметь возможность ответить: "Уточните, интересует исторический период или конкретная область достижений?". Очевидно, механизм многошагового уточнения позволит повысить точность и полноту ответов системы. Система MemGPT ([15]) иллюстрирует вариант "диалоговой" архитектуры. В рамках этой системы реализуется агент, который способен запоминать, размышлять и эволюционировать в процессе длительного взаимодействия с пользователем. Это подтверждает важность архитектур, поддерживающих многошаговый диалог и управление состоянием — агент может задавать уточняющие вопросы и сохранять контекст на протяжении всего общения.

Кратко разберем основные компоненты архитектуры такой задаче-ориентированной диалоговой системы (рис. 4.2.1). Сначала запрос пользователя попадает в модуль предобработки, где происходит нормализация входного текста, устранение шума, перефразирование и (при необходимости) первичная декомпозиция на под-запросы. Результатом является один или несколько нормализованных запросов, удобных для последующей интерпретации. Далее следует Dialogue Manager, отвечающий за ведение диалогового графа (узлы - реплики пользователя и системы, ребра - временные и семантические связи) и агрегацию информации, собранной в ходе цикла уточнения. Благодаря Dialogue Graph система будет обуславливаться не на последнюю user-реплику (utterance), а на весь контекст. Затем на основе предобработанного запроса и контекста из диалогового графа Intent Detection определяет

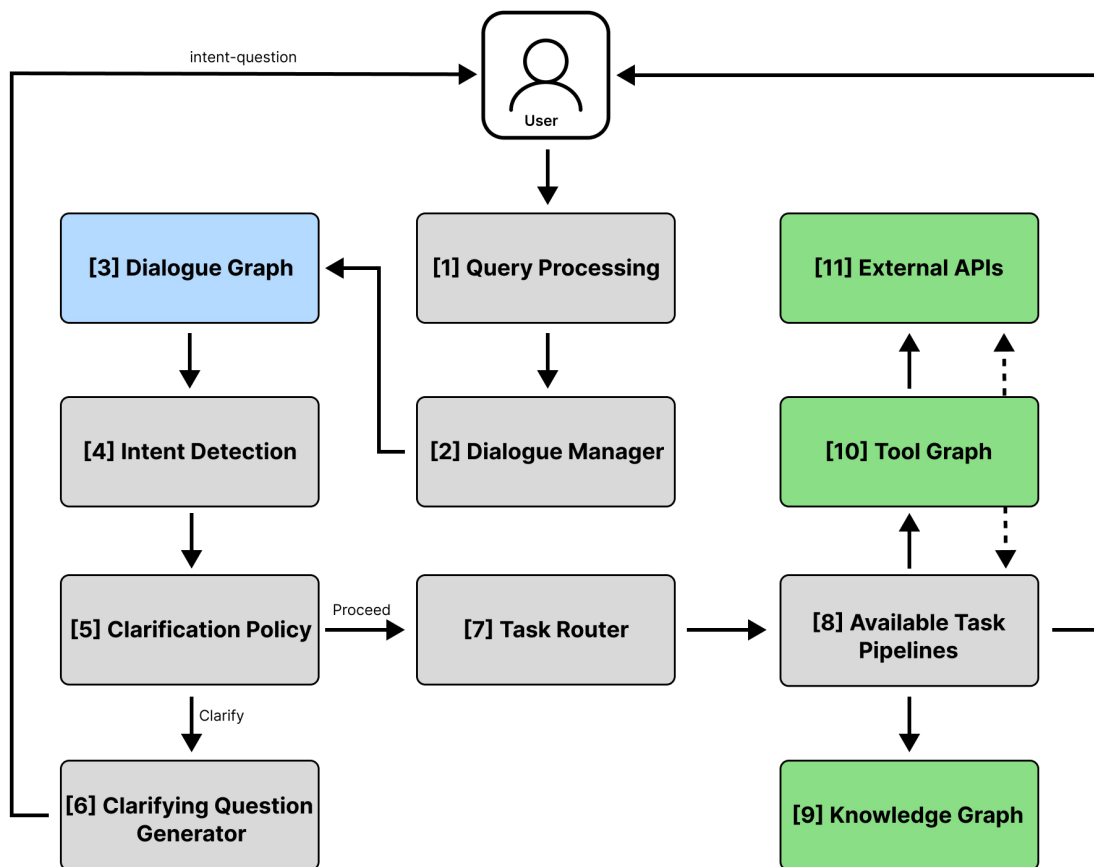


Рисунок 4.2.1 — Высокоуровневая архитектура задача-ориентированной диалоговой системы. Синим цветом обозначены структуры данных для временного хранения поступающей информации. Зеленым цветом обозначены компоненты для постоянного хранения сценариев поведения системы и персональных данных пользователя

один или несколько кандидатов intent'ов, оценивает уверенность в принадлежности запросов к этим intent'ам и выявляет недостающие параметры задачи, без которых нельзя запустить соответствующий task-конвейер. Результат диагностики передается в Clarification Policy. В данной компоненте принимается решение о том, следует ли задавать уточняющий вопрос. Если уверенность в intent ниже порога, либо количество незаполненных параметров задачи превышает порог, либо обнаружены противоречия - сгенерировать уточняющий вопрос; иначе - перейти к запуску task-конвейера. К этим условиям могут добавиться и другие: стоимость дополнительного вопроса, предпочтения пользователя (например, короткие/длинные ответы, склонность к дополнительным пояснениям), временные ограничения (например, не более N уточнений подряд). Также в Clarification Policy определяется какой именно тип вопроса стоит задать и когда цикл уточнения нужно завершить. Если было принято решение о необходимости уточняющего вопроса, то конвейер переходит к блоку Clarifying Question Generator. Используя LLM и информацию от Clarification Policy, генератор формирует текст уточняющего вопроса на естественном языке. При этом учитывается текущий диалоговый контекст (Dialogue Graph), чтобы вопросы были непротиворечивыми и локально связными. Сам уточняющий запрос записывается в Dialogue Graph. После каждого шага уточнения

производится повторная детекция intent'ов и оценка заполненности параметров задачи. Цикл прекращается, если:

- уверенность в одном из intent'ов превышает верхний порог;
- все критичные параметры для выбранной user-проблемы заполнены и не противоречат графу знаний;
- достигнуто ограничение по числу уточнений (защита от бесконечных диалогов);
- пользователь явно сигнализирует о завершении или отказе.

Затем на следующем этапе выполняется сопоставление извлеченных user-проблем с поддерживаемым набором task-конвейеров, каждый из которых предназначен для решения конкретной узкоспециализированной задачи. При выполнении, каждый из доступных task-конвейеров может взаимодействовать с пользователем в режиме диалога для уточнения/сбора необходимой информации с целью решения заданной проблемы. В результате выполнения каждого из сопоставленных task-конвейеров пользователю отправляется финальное сообщение с информацией о статусе/решении его user-проблем. Диалоговая сессия завершается. Важно, что task-конвейер может использовать граф знаний и граф инструментов (Tool Graph) для получения данных и взаимодействия с внешними системами. Сами инструменты (tools) являются API-сервисами с конкретным/предметным функционалом, взаимодействие с которыми осуществляется по фиксированным протоколам (например, в формате json). С помощью графовой структуры можно определять зависимости между имеющимися инструментами и управлять последовательностью их вызова. Данный механизм позволяет декомпозировать логику системы: функционал по взаимодействию с внешними ресурсами по детерминированному протоколу вынести в компонент с инструментами, а конвейеры по решению узкоспециализированных задач на базе мультиагентных методов, которые требуют наличия механизма понимания естественного языка (Natural Language Understanding), вынести в компоненту с доступными task-конвейерами. Таким образом, task-конвейеры могут пользоваться имеющимися инструментами для отправки/получения необходимой информации с целью решения поставленной задачи, при этом связность данных компонент достаточно низкая и может разрабатываться/поддерживаться параллельно независимыми командами программистов.

Многошаговое уточнение тесно связано с существующими пайплайнами:

4.2.1.1 Memorize-пайплайн

Ответы пользователя в ходе уточнения могут содержать новые факты (например, уточненные параметры договора, новые даты, условия). После завершения цикла эти факты могут быть преобразованы в триплеты и добавлены в граф знаний с привязкой к эпизодической вершине, описывающей текущую диалоговую сессию. При этом применяются уже реализованные механизмы темпоральной разметки и инвалидации устаревших фактов.

4.2.1.2 QA-пайплайн

Уточненный запрос поступает в модуль извлечения соответствующих сущностей и триплетов. При обходе графа учитывается структурированная информация из Dialogue Graph. Модуль регенерации плана также может учитывать историю уточнений: например, если один план привел к необходимости дополнительного уточнения, это сигнал, что исходный план был недостаточно детализирован или неверно интерпретировал intent.

В итоге благодаря описанной архитектуре появляется возможность многоуровневого уточнения. На глобальном уровне уточнения используются для выбора intent'a и базовой постановки задачи (какой task-конвейер запускать). А на локальном каждый task-конвейер может иметь собственный под-цикл уточнения (например, уточнение параметров продукта, периода, типа клиента). Кроме того, возможна персонализация стратегии уточнений. По мере накопления взаимодействий полезно адаптировать Clarification Policy под конкретного пользователя (например, уменьшать частоту уточнений, если пользователь предпочитает краткие ответы, или, наоборот, увеличивать, если пользователь склонен к подробным обсуждениям); а также сохранять в персональном сегменте графа знаний мета-информацию о предпочтениях пользователя и использовать ее при формировании вопросов.

4.2.2 Поддержка диалоговых структур и контекста

Поскольку клиент и менеджер взаимодействуют именно в формате диалога, то важно учитывать эту структуру при извлечении триплетов. Если объединять все сообщения в один текст и далее извлекать данные из него, то мы рискуем потерять важные детали и вместе с тем, наоборот, найти факты, которые не подразумеваются в исходной переписке. Поэтому важно учитывать диалоговую структуру и принимать во внимание временные метки сообщений. Как вариант можно добавить resolution стадию объединения связанных сообщений перед тем как сохранять информацию в модель памяти. Кроме того, текущая архитектура графа знаний может быть расширена для хранения диалогового контекста – например, добавлением специальных типов узлов для реплик пользователя и системы. Это позволит привязывать факты или уточнения к конкретным шагам диалога. Каждое сообщение можно рассматривать как текстовый узел, связанный с упоминаемыми в нем сущностями и предыдущими репликами. Таким образом, сохранение связей между сообщениями будет обеспечиваться как на уровне линейной истории (последовательность диалога), так и на уровне семантических отношений (например, связь "уточнение к вопросу №2" или "контекст для ответа №3"). Большая языковая модель, получая на вход не только последний запрос, но и релевантные узлы этого диалогового графа, сможет правильно интерпретировать местоимения, вопросы и отсылки к ранее данным ответам. Также описанная структура облегчает реализацию

долгосрочных диалогов: даже если контекст выходит за окно памяти модели, агент может извлекать необходимые прошлые факты из графа (по семантическому сходству или ссылкам в диалоговом графе) и тем самым не забывать важных деталей беседы. В дополнении к этому представляется возможным обеспечить управление состоянием диалога: похожим образом на граф состояний в классических чат-ботах, система может отслеживать фазу диалога (уточнение вопроса, поиск ответа, валидация ответа и так далее) и переключаться между ними по заранее заданной логике. Поддержка полноценной диалоговой структуры расширит сферу применения ассистента – от одиночных фактологических вопросов к интерактивному помощнику, ведущему осмысленный разговор.

В работе [31] демонстрируется необходимость поддержки структуры диалога и контекста (например, иерархической памяти). Авторы предлагают древовидную структуру памяти для LLM, организующую историю диалога по иерархии, аналогичной человеческим когнитивным схемам. Это решение значительно улучшает обработку многошаговых запросов, что подтверждается ростом метрик на соответствующих бенчмарках.

4.3 Концептуальные расширения системы

4.3.1 Мультиагентные архитектуры

Дополнительное внедрение агентов в систему сможет повысить ее надежность и воспроизводимость. Если один агент делает промежуточный вывод, другой может его сверить с данными графа или попытаться найти противоречия. Механизмы вроде "majority voting" уменьшают вероятность ошибки или галлюцинации: даже если один агент ошибется, остальные скорректируют итог. В статье *Debate on Graph* [23] описан подход, в котором команда агентов поочередно выдвигает и проверяет гипотезы, что обеспечивает надежность процесса рассуждений. Результаты показывают, что архитектуры с несколькими агентами (например, генератор, критик, валидатор) превосходят одиночные. Для реализации подобных схем взаимодействия следует использовать современные фреймворки. LangGraph – перспективный open-source инструмент, позволяющий задавать явно рабочие процессы агентов в виде графов: разработчик определяет узлы-агенты и ребра коммуникации между ними, формируя тем самым схему координации (например, цепочка, звезда с центральным оркестратором, или даже произвольный граф взаимодействий). Эта гибкость особенно ценна для экспериментального сравнения архитектур: можно будет настроить как централизованную схему (один главный агент планирует и распределяет задачи вспомогательным), так и децентрализованную (агенты обмениваются сообщениями свободно, на основе событий). Развитие проекта в сторону мультиагентности также позволит интегрировать идеи из свежих научных разработок: например, концепция GPTSwarm предлагает рассматривать си-

стему агентов как оптимизируемый граф (swarm), где узлы – это операции или модели, а ребра – обмен данными; система обучается настраивать этот граф для улучшения результата. Внедрение мультиагентной архитектуры усложнит систему, но и сделает ее более гибкой и мощной. Необходимо будет решить задачу организации их коллаборации: определить протоколы общения (например, обмен сообщениями на общем языке или через разделяемую память), выбрать тип архитектуры (централизованная с одним менеджером или иерархическая с мета-агентом, координирующим группы ниже). В целом, мультиагентный подход заложит основу для масштабирования системы: можно наращивать число агентов для более сложных задач, постепенно добавляя новых "умений" (агентов-инструментов, агентов-переводчиков, агентов-визуализаторов и т.д.) без радикальной переработки архитектуры.

4.3.2 Мультимодальность

Если рассматривать настоящий проект применительно именно к банковской сфере, то естественным образом возникает необходимость добавить возможность работы с данными разных модальностей. Действительно, достаточно часто пользователю требуется не просто задать отдельный вопрос, но и дополнить его соответствующим контекстом. Нередко в качестве такого контекста выступает картинка. Например, клиент может отправить копию своего паспорта, или заполнить нормативный документ и попросить проверить корректность этого заполнения. Вполне вероятно, что пользователь захочет что-то спросить по какому-нибудь графику (динамике изменения ключевой ставки) или по таблице. Наконец, если запрос достаточно сложный и клиент ограничен во времени, он может сформулировать его голосом - в этом случае важно, чтобы модель смогла принять и такой формат данных. Если система получит возможность принимать на вход не только текст, но и фото/аудио, это значительно расширит границы ее применимости и поможет удовлетворить больше пользовательских запросов.

4.3.3 Развитие системы в сторону цифрового наставника

Долгосрочное видение проекта предусматривает трансформацию из просто ответчика на вопросы в полноценного цифрового наставника, способного не только выдавать справочную информацию, но оценивать работу менеджера, корректировать ее и предлагать рекомендации по улучшению. Для этого требуются концептуальные расширения по нескольким направлениям. Первое – встроенная валидация и коррекция ответов: система должна уметь оценивать корректность высказываний менеджера. Например, если консультант предлагает свое решение задачи, ассистент-наставник проверит его на основании графа знаний и укажет на возможные ошибки или подтвердит правильность. Достичь этого поможет сочетание методов: фактическая проверка по

графу (нахождение контрпримеров, сверка утверждений с известными фактами) и использование дополнительного агента-эксперта для критического разбора ответа. Второе – персонализированные рекомендации: наставник не ограничивается выдачей прямого ответа, а на основе данных диалога (и возможно предыдущих коммуникаций клиента) учитывает его особенности. Граф знаний можно обогатить персональными узлами, отражающими предпочтения пользователя, а LLM-агент – дообучить на данных взаимодействия с конкретным человеком. Третье направление – объяснимость и прозрачность. Наставник обязан не только давать ответ, но и объяснять его (как учитель ученику). Поэтому система должна генерировать обоснование, ссылаясь на узлы графа: например, "Я рекомендую этот ответ, потому что в базе знаний нашлись такие-то факты, они приведены в приложении". Здесь вновь пригодится мультиагентность: один агент может формировать решение, а второй – генерировать к нему понятное объяснение, либо краткое резюме ключевых знаний из графа для поддержки ответа. Кроме того, цифровой наставник может выполнять превентивную коррекцию: если вопрос пользователя некорректен или базируется на ложном предположении, ассистент мягко укажет на это и переформулирует проблему. Например: "Ваш вопрос предполагает, что X является Y, однако, согласно нашим данным, это не так. Возможно, Вы имеете в виду...". Реализация роли наставника потребует сочетания всех вышеперечисленных улучшений: диалогового взаимодействия (для проведения разъяснительной беседы), валидации по графу (для фактологической корректности), мультиагентных проверок (для надежности ответов) и дообучения на взаимодействиях (для адаптации к стилю пользователя). Успех в этом направлении сделает систему уникальной: она станет не просто отвечать на запросы, а активно помогать клиенту достигать целей, обучать менеджера, давать ему советы и независимо оценивать его работу.

4.4 Оценка качества и расширение бенчмарков

4.4.1 Новые бенчмарки с учетом темпоральности и доменной вариативности

С развитием функциональности системы необходимо расширять и систему ее оценки. Помимо текущих тестов на фактические вопросы (таких как HotpotQA для и RuBQ), разумно добавить бенчмарки, учитывающие временные зависимости и другие предметные области. Темпоральный аспект особенно актуален, поскольку граф знаний в проекте поддерживает временные метки. Будут полезны датасеты наподобие TimeQA или TemporalWiki – вопросы, где правильный ответ зависит от времени (например, "Кто был президентом Франции в 1995 году?" или "Как изменилось население города X с 2010 по 2020?"). Проверяя систему на таких задачах, можно измерить, насколько эффективно реализована временная память: правильно ли учитываются обновления знаний, не путает ли агент факты из разных эпох. В целом оценка систем на данных, отражающих изменение знаний во времени, становится все более актуальной. Авторы

работы [14] представляют T-GRAG, архитектуру QA с учетом темпоральных графов знаний, и бенчмарк Time-LongQA, основанный на эволюционирующих корпоративных данных, специально для оценки таких задач. Результаты показывают, что учет временной динамики заметно улучшает точность.

Интересно также проверить робастность системы к данным из разных доменов: например, медицинские QA датасеты, где требуется использовать соответствующий граф знаний, или технические форумы (QA по программированию), где свой жаргон и специфические факты. Расширение системы оценки предполагает не только подбор новых датасетов, но и, при необходимости, разработку собственных тестовых сценариев, отражающих целевые условия эксплуатации системы. Например, можно создать набор из синтетических диалогов, где пользователь постепенно уточняет свой запрос, или включить проверку наставнических функций (пользователь дает неправильный ответ, а система должна его поправить). Важным шагом будет внедрение метрики комплексной успешности диалога: сочетание понимания, корректности ответа, уместности уточняющих вопросов и удовлетворенности пользователя. Это более сложная верификация, чем простой Exact Match, но она ближе к реальной работе цифрового помощника.

4.4.2 Оценка улучшений плана и reasoning

Как отмечалось, модуль планирования требует своих критериев оценки. В рамках тестирования новой версии QA-пайплайна на расширенных бенчмарках полезно собирать статистики, отражающие качество reasoning-процесса. Один из подходов – сравнение trace (траектории) рассуждений системы с эталонными или с логикой, заложенной авторами вопросов. Например, в HotpotQA часто доступны supporting facts – набор предложений, необходимых для вывода ответа. Система должна не просто дать верный ответ, но и найти эти ключевые факты в графе. Метрика Coverage может измерять, сколько из требуемых фактов было извлечено. Другая метрика – Redundancy: напротив, насколько лишние узлы графа были посещены (идеально, план охватывает только релевантные области памяти). Снижение Redundancy при стабильной или возросшей Accuracy будет признаком более целевого плана. Кроме автоматических метрик, целесообразно проводить анализ цепочек рассуждений: экспертами или с помощью LLM-судьи. Авторы статьи [26] подтверждают необходимость метрик, оценивающих именно reasoning-процесс (логическая связность, соответствие фактам), а не только результат. В работе предложен метод Knowledge-Driven CoT, в котором LLM самостоятельно проверяет и корректирует промежуточные шаги reasoning, используя внешние знания. Это позволяет устранить неверные или недостоверные промежуточные рассуждения и улучшить итоговые ответы.

4.5 Безопасность и надежность

Современные исследования [35] показывают, что текущие LLM остаются уязвимыми к "джейлбрейкам" и fine-tuning атакам. Только недавно начали появляться более глубокие стратегии alignment'a, способствующие большей устойчивости системы. Поэтому развитие возможностей предложенного подхода должно сопровождаться усилением мер безопасности, чтобы персональный AI-ассистент оставался надежным и не нарушал этических норм. Безопасность здесь включает две ключевые стороны: защиту чувствительной информации и противодействие некорректным или провокационным запросам. Во-первых, поскольку граф знаний может содержать личные или конфиденциальные данные пользователя, крайне важно обеспечить их защиту. Как вариант можно реализовать многоуровневый контроль доступа к узлам графа: некоторые факты будут помечены как приватные и использоваться моделью лишь в ответах непосредственно владельцу данных, но не раскрываться внешним образом. Например, если в персональном графе сохранены номера счетов, ассистент не должен выдавать их без соответствующего запроса и авторизации. Хранение графовой базы знаний должно быть защищено, возможно, с шифрованием, а доступ LLM-агента к ней – реализован через фильтрующий слой, который исключает передачу в промпт явно личной информации (вместо этого подставляя абстракции или обезличенные метки). Во-вторых, система будет оборудована механизмами обнаружения и отклонения провокационных запросов. Сюда относятся попытки пользователя заставить модель выдать заведомо неправильный или этически неприемлемый ответ, инструкции, обходящие политики (jailbreak-промпты), и прочие виды ввода, способные привести к токсичным или небезопасным выводам. Для этого целесообразно внедрить специальный фильтр контента: перед тем как передать запрос в основную модель, он будет проверяться классификатором или вспомогательной LLM-моделью на предмет запрещенных тем (экстремизм, личные данные, оскорбления, приватные сведения и так далее). Если запрос нарушает политику, ассистент либо откажется отвечать, либо выдаст вежливое предупреждение. Аналогично, перед отправкой ответа пользователю, финальную информацию можно прогонять через модуль модерации, чтобы убедиться в отсутствии утечки данных или нежелательной лексики. В целом в мультиагентной конфигурации можно выделить отдельного агента-модератора, отвечающего за цензурирование и безопасность: основной агент генерирует черновой ответ, модератор его корректирует или маскирует чувствительные фрагменты, затем только одобренный ответ уходит пользователю. С интеграцией все более сложных возможностей, доверие пользователя будет зависеть от того, насколько система справляется с провокациями и не допускает утечек, поэтому безопасность остается приоритетным направлением на всех этапах развития проекта.

4.6 Выводы

Предложенные направления развития нацелены на превращение текущей QA-системы в надёжного и эффективного цифрового ассистента. Перечень ключевых аспектов: (1) технические улучшения ядра системы (унификация сущностей/отношений при построении графа знаний, оптимизация и дообучение используемых нейросетевых моделей), (2) поддержка диалогового взаимодействия пользователя с системой, (3) расширение списка поддерживаемых модальностей данных и (4) расширение списка механизмов/техник по контролю эффективности и производительности, а также обеспечению надёжности и безопасности системы. Совокупность предлагаемых модификаций направлена на создание эффективного инструмента для менеджеров различных отраслей экономики, включая банковскую сферу, способных поддерживать длительные диалоги, предоставлять точные и безопасные ответы на сложные вопросы, учитывая временное изменение информации и обеспечивая глубокую персонализацию рекомендаций и советов. Описанные нововведения позволят вывести решение на качественно новый уровень, открывая перспективы практического внедрения технологии для широкого круга профессиональных приложений.

ЗАКЛЮЧЕНИЕ

В ходе выполнения настоящей научно-исследовательской работы разработаны методы и подходы для представления внешней памяти LLM-агентов в виде графа знаний. Создана архитектура графовой базы знаний, объединяющая как структурированные, так и неструктурированные данные, а также алгоритмы ее пополнения и актуализации. Разработан поисковый QA-пайплайн, позволяющий извлекать из графа релевантные сведения для ответа на пользовательские запросы. Предложенные решения интегрированы в прототип LLM-агента, память которого основана на графе знаний. Более того, были спроектированы и реализованы варианты пайплайнов, учитывающие изменчивость данных с течением времени, что позволило решить проблему утраты актуальности информации и точнее отвечать на вопросы, связанные с датами и временными промежутками.

Все задачи, поставленные в рамках технического задания, решены полностью. Выполнен обзор современных методов построения и обхода графов знаний (GraphRAG-подход), разработана целевая архитектура хранения знаний (text-attributed knowledge graph) с поддержкой динамического обновления информации, реализован алгоритм контекстного извлечения информации из графа; данный функционал оформлен в виде программной PersonalAI-библиотеки. Проведена количественная оценка качества разработанного функционала на открытых данных в соответствии с условиями технического задания. В частности посчитаны метрики Exact Match и LLM-as-a-Judge на датасетах HotpotQA и RuBQ и для разных LLM (в том числе GigaChat Max). Таким образом, требования ТЗ выполнены в полном объеме, а полученные результаты отражены в данном отчете.

Эксперименты подтвердили, что LLM-агент с внешней памятью на основе графа знаний способен более точно отвечать на сложные составные (multi-hop) вопросы по сравнению с классической поисково-дополненной генерацией (RAG). В частности, на датасете HotpotQA (англоязычные вопросы с привлечением нескольких фактов) достигнут показатель Exact Match порядка 61%, что превосходит результаты известных аналогичных методов (Think-on-Graph, Reasoning on Graph, Think-on-Graph 2.0, Debate-on-Graph и др., с диапазоном 36–46% EM) примерно на 15%. Кроме того, по метрике LLM-as-a-Judge разработанный подход превзошел базовый метод PersonalAI на 3% на HotpotQA, подтвердив повышение точности и фактической корректности ответов. Также в рамках сравнительного анализа были проведены эксперименты с моделью GigaChat Max, результаты которой на HotpotQA составили 79%, а на RuBQ - 87% (LLM-as-a-Judge).

Общий научно-технический уровень проведенного исследования соответствует современному состоянию дел (SOTA) в области: в работе использованы и развиты самые актуальные подходы к построению и использованию графа знаний с участием больших языковых моделей.

Технико-экономическая эффективность внедрения заключается в возможности использования разработанного графа знаний в качестве внешней памяти для интеллектуальных агентов в финансовой сфере и образовании. В частности, он может стать основой для цифрового наставника (Tutor AI) – виртуального тренажера для клиентских менеджеров банка. Такой наставник будет анализировать диалоги менеджера с клиентом, проверять факты и данные на основе собственной базы знаний (графа) о продуктах банка и генерировать рекомендации по ведению беседы. Реализация подобного инструмента облегчит работу менеджеров, снизит отток клиентов и повысит их лояльность, что в итоге положительно скажется на бизнес-показателях Индустриального партнера. С экономической точки зрения внедрение Tutor AI позволит сократить затраты на очное обучение и сопровождение сотрудников, уменьшить время вывода новых менеджеров на целевые показатели, а также снизить количество ошибок и некорректных консультаций, ведущих к прямым и косвенным финансовым потерям. Ожидается, что совокупный эффект от уменьшения издержек при одновременном росте конверсии продаж, увеличении доли удержанных клиентов и повышении объема кросс-продаж приведет к устойчивому росту прибыли и улучшению ключевых финансово-экономических показателей Индустриального партнера. При этом разработанное решение обладает высоким потенциалом тиражирования на различные подразделения и продуктовые линии, что дополнительно усиливает его экономическую привлекательность. Кроме того, методы, разработанные в рамках проекта, могут быть адаптированы для агентов, обобщающих обратную связь от клиентов с целью улучшения продуктов (резюмирования обращений, анализ жалоб и предложений), а также для создания персональных AI-ассистентов и умных информационных хранилищ в различных предметных областях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Edge D., Trinh H., Cheng N., Bradley J., Chao A. et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. – 2024. – arXiv:2404.16130.
2. Gutiérrez B. J., Shu Y., Qi W., Zhou S. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. – 2025. – arXiv:2502.14802.
3. OpenAI et al. GPT-4 Technical Report. – 2024. – arXiv:2303.08774.
4. Yang A., Yang B., Hui B., Zheng B., Yu B. et al. Qwen2 Technical Report. – 2024. – arXiv:2407.10671.
5. Grattafiori A., Dubey A., Jauhri A., Pandey A., Kadian A. et al. The Llama 3 Herd of Models. – 2024. – arXiv:2407.21783.
6. Liu L., Yang X., Lei J., Shen Y., Wang J. et al. A Survey on Medical Large Language Models: Technology, Application, Trustworthiness, and Future Directions. – 2024. – arXiv:2406.03712.
7. Li Y., Wang S., Ding H., Chen H. Large Language Models in Finance: A Survey // Proceedings of the Fourth ACM International Conference on AI in Finance (ICAIF 2023). – Association for Computing Machinery, 2023. – P. 374–382. – doi:10.1145/3604237.3626869.
8. Ghimire A., Prather J., Edwards J. Generative AI in Education: A Study of Educators' Awareness, Sentiments, and Influencing Factors // Proceedings of the 2024 IEEE Frontiers in Education Conference (FIE 2024). – IEEE, 2024. – P. 1–9. – doi:10.1109/FIE61694.2024.10892891.
9. Huang L., Yu W., Ma W., Zhong W., Feng Z. et al. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions // ACM Transactions on Information Systems. – 2025. – Vol. 43, Iss. 2. – P. 1–55. – doi:10.1145/3703155.
10. Liu N. F., Lin K., Hewitt J., Paranjape A., Bevilacqua M. et al. Lost in the Middle: How Language Models Use Long Contexts // Transactions of the Association for Computational Linguistics. – 2024. – Vol. 12, Iss. 5. – P. 157–173. – doi:10.1162/tacl_a_00638.
11. Zhang B., Soh H. Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction // Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024). – Association for Computational Linguistics, 2024. – P. 9820–9836.
12. Rasmussen P., Paliychuk P., Beauvais T., Ryan J., Chalef D. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. – 2025. – arXiv:2501.13956.
13. Bai J., Fan W., Hu Q., Zong Q., Li C. et al. AutoSchemaKG: Autonomous Knowledge

- Graph Construction through Dynamic Schema Induction from Web-Scale Corpora. – 2025. – arXiv:2505.23628.
14. Li D., Niu Y., Ai Y., Zou X., Qi B. et al. T-GRAG: A Dynamic GraphRAG Framework for Resolving Temporal Conflicts and Redundancy in Knowledge Retrieval // Proceedings of the 33rd ACM International Conference on Multimedia (MM 2025). – Association for Computing Machinery, 2025.
 15. Packer C., Fang V., Patil S. G., Lin K., Wooders S. et al. MemGPT: Towards LLMs as Operating Systems. – 2023. – arXiv:2310.08560.
 16. Sun J., Xu C., Tang L., Wang S., Lin C. et al. Think-on-Graph: Deep and Responsible Reasoning of Large Language Models on Knowledge Graph // Proceedings of the International Conference on Learning Representations (ICLR 2023), 2023.
 17. Luo L., Li Y., Haffari G., Pan S. Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning. – 2024. – arXiv:2310.01061.
 18. Liu J., Zhou T., Chen Y., Liu K., Zhao J. Enhancing Large Language Models with Pseudo- and Multisource- Knowledge Graphs for Open-Ended Question Answering // Proceedings of the 2025 IEEE 41st International Conference on Data Engineering Workshops (ICDEW 2025). – IEEE, 2025. – doi:10.1109/ICDEW67478.2025.00019.
 19. Sanmartin D. KG-RAG: Bridging the Gap Between Knowledge and Creativity. – 2024. – arXiv:2405.12035.
 20. Hu Y., Lei Z., Zhang Z., Pan B., Ling C. et al. GRAG: Graph Retrieval-Augmented Generation // Findings of the Association for Computational Linguistics (NAACL 2025). – Association for Computational Linguistics, 2025. – P. 4145–4157.
 21. Mavromatis C., Karypis G. GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning // Findings of the Association for Computational Linguistics (ACL 2025). – Association for Computational Linguistics, 2025. – P. 16682–16699.
 22. Ma S., Xu C., Jiang X., Li M., Qu H. et al. Think-on-Graph 2.0: Deep and Faithful Large Language Model Reasoning with Knowledge-Guided Retrieval Augmented Generation // International Conference on Learning Representations (ICLR 2024), 2024.
 23. Ma J., Gao Z., Chai Q., Sun W., Wang P. et al. Debate on Graph: A Flexible and Reliable Reasoning Framework for Large Language Models // Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2024). – Association for the Advancement of Artificial Intelligence, 2024. – Vol. 38. – P. 24768–24776.
 24. Luo L., Zhao Z., Haffari G., Li Y., Gong C. et al. Graph-Constrained Reasoning: Faithful Reasoning on Knowledge Graphs with Large Language Models. – 2025. – arXiv:2410.13080.
 25. Sun L., Wang X., Li Y. Pyramid-Driven Alignment: Pyramid Principle Guided Integration of Large Language Models and Knowledge Graphs. – 2024. – arXiv:2410.12298.
 26. Wang K., Duan F., Wang S., Li P., Xian Y. et al. Knowledge-Driven CoT: Exploring

- Faithful Reasoning in LLMs for Knowledge-Intensive Question Answering. – 2023. – arXiv:2308.13259.
27. Brown T. B., Mann B., Ryder N., Subbiah M., Kaplan J. et al. Language Models Are Few-Shot Learners // Advances in Neural Information Processing Systems 33 (NeurIPS 2020). – Curran Associates, Inc., 2020. – P. 1877–1901.
 28. He X., Tian Y., Sun Y., Chawla N. V., Laurent T. et al. G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering // Advances in Neural Information Processing Systems 37 (NeurIPS 2024). – Curran Associates, Inc., 2024. – P. 132876–132907.
 29. Li B., Fei H., Li F., Wu Y., Zhang J. et al. DiaASQ: A Benchmark of Conversational Aspect-Based Sentiment Quadruple Analysis // Findings of the Association for Computational Linguistics (ACL 2023). – Association for Computational Linguistics, 2023. – P. 13449–13467.
 30. Andoni A., Indyk P., Razenshteyn I. Approximate Nearest Neighbor Search in High Dimensions // Proceedings of the International Congress of Mathematicians 2018 (ICM 2018). – World Scientific, 2018.
 31. Rezazadeh A., Li Z., Wei W., Bao Y. From Isolated Conversations to Hierarchical Schemas: Dynamic Tree Memory Representation for LLMs. – 2024. – arXiv:2410.14052.
 32. Menshikov M., Kharitonov A., Kotyga M., Porvatov V., Zhukovskaya A. et al. Position of Uncertainty: A Cross-Linguistic Study of Positional Bias in Large Language Models. – 2025. – arXiv:2505.16134.
 33. Dettmers T., Pagnoni A., Holtzman A., Zettlemoyer L. QLoRA: Efficient Finetuning of Quantized LLMs. – 2023. – arXiv:2305.14314.
 34. Wei H., Zhang Z., He S., Xia T., Pan S. et al. PlanGenLLMs: A Modern Survey of LLM Planning Capabilities // Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025). – Association for Computational Linguistics, 2025. – P. 19497–19521.
 35. Qi X., Panda A., Lyu K., Ma X., Roy S. et al. Safety Alignment Should Be Made More Than Just a Few Tokens Deep // International Conference on Learning Representations (ICLR 2025), 2025.